OneCare Clinic Management System

Submitted By

Thushajini Yogalingam
(SE/2021/019)
Vipusa Sriharan
(SE/2021/043)
Ramachandran Sweeskaran
(SE/2021/046)
Kenuja Sivakumar
(SE/2021/050)

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honors in Software Engineering

Name of the Supervisor: Dr Nalin Warnajith

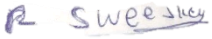
Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka
2026

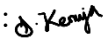
Declaration

We hereby certify that this project and the all the artifacts associated with it is our own work and it has not been submitted before nor is currently being submitted for any other degree program.

Full name : Thushajini Yogalingam
Student No : SE/2021/019
Signature :  Date : 2026/04/30

Full name : Vipusa Sriharan
Student No : SE/2021/043
Signature :  Date : 2026/04/30

Full name : Ramachandran Sweeskaran
Student No : SE/2021/046
Signature :  Date : 2026/04/30

Full name : Kenuja Sivakumar
Student No : SE/2021/050
Signature :  Date : 2026/04/30

Name of the supervisor(s): .....
Signature of the supervisor:..... Date.....

Acknowledgement

We would like to extend our heartfelt gratitude to all our lecturers for their unwavering support and guidance throughout the SENG 31242 System Design Project. We are especially thankful to the lecturers of the Software Engineering Teaching Unit (SETU), whose instruction and mentorship provided us with the foundational knowledge necessary to effectively address the complex design challenges we faced during the development of the OneCare Clinic Management System.

We are also deeply grateful to our client and the medical professionals who collaborated with us during the requirement elicitation and design phases. Their invaluable insights into healthcare workflows and real-world clinical challenges significantly influenced the functional design and usability of the OneCare platform.

In addition, we would like to acknowledge the dedication and teamwork of our entire project team. Each member's effort from system analysis to architectural design was integral to the success of this project, reflecting our shared commitment, determination, and collaborative spirit in achieving our objectives.

Project Team:

- Thushajini Yogalingam (SE/2021/019)
- Vipusa Sriharan (SE/2021/043)
- Ramachandran Sweeskaran (SE/2021/046)
- Kenuja Sivakumar (SE/2021/050)

Abstract

The OneCare Clinic Management System is a specialized software solution designed to digitize and optimize the daily operations of a private medical clinic with pharmacy. By transitioning the clinic from manual, paper-based processes to a centralized digital framework, the platform addresses specific operational bottlenecks such as inefficient record-keeping, disorganized appointment scheduling, and manual pharmacy tracking. Key features include a role-based access control system for clinic staff, an automated patient queue management module, and high-fidelity UI/UX designs tailored to the specific workflows of private practice.

The system architecture is built on a robust layered model using Sprin Boot and React, ensuring high performance and data integrity for sensitive medical records. A significant focus of the design is the integration of a real-time Pharmacy Inventory system and a dedicated Doctor Module, which together minimize clinical errors and reduce patient wait times. By providing a secure, user centric interface, the OneCare system enhances the productivity of the clinic's staff while improving the overall quality of care provided to patients. Through this targeted digital transformation, OneCare establishes a reliable and scalable design standard for modern private clinical management.

Table of Contents

DECLARATION I

ACKNOWLEDGEMENT II

ABSTRACT III

TABLE OF CONTENTS IV

LIST OF FIGURES VI

LIST OF TABLES VII

ABBREVIATIONS VIII

CHAPTER 1: INTRODUCTION 1

1.1 Client Details 1
1.2 Problem Statement 1
1.3 Proposed Solution Overview 2
1.4 Stakeholders 2
1.5 Client Agreement Letter 3

CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION 6

CHAPTER 3: SYSTEM DESIGN SPECIFICATION (SDS) 7

CHAPTER 4: PROTOTYPE & CLIENT VALIDATION 8

4.1 Prototype (Figma navigation flow) 8
4.2 Client feedback 24
4.3 Client validation 24
4.4 Ethical considerations 24
4.5 Communication reflection 28

CHAPTER 5: CONCLUSION 29

5.1 Degree of Objectives Met 29
5.2 Usability, Accessibility, Reliability and Friendliness 31
5.3 Limitations and Drawbacks 32
5.4 Timeline 33
5.5 Future Modifications, Improvements and Extensions Possible 35

REFERENCES 37

ANNEX 1: SOFTWARE REQUIREMENTS SPECIFICATION 38

1.1 Project Scope And Stakeholders 38
1.2 Functional Requirements 39
1.3 Non-Functional Requirements 44
1.4 Business Rules, Constraints And Assumptions 52
1.5 Use Case Diagrams 55
1.6 Use Case Descriptions 60
1.7 Activity Diagrams 90
1.8 External Interface Requirements 96
1.9 Data Requirements 99
1.10 Alternative Solutions Considered 106
1.11 Feasibility Study 108

1.12 RISK ANALYSIS 112

ANNEX 2: SYSTEM DESIGN SPECIFICATION 118

2.1 ARCHITECTURAL DESIGN 118
2.2 DATABASE DESIGN 123
2.3 CLASS DIAGRAM 127
2.4 SEQUENCE DIAGRAM 130
2.5 SECURITY DESIGN 143
2.6 DEPLOYMENT DESIGN 156

ANNEX 3: PROOFREADING CERTIFICATION 162

ANNEX 4: CLIENT FEEDBACK REPORT 165

List of Figures

Figure 1 Client Agreement Letter Page 01 3
Figure 2 Client Agreement Letter Page 02 4
Figure 3 Client Agreement Letter Page 03 5
Figure 4: Medical system login page 8
Figure 5 Patient Webpage 8
Figure 6: Booking Confirmed Page 9
Figure 7: Patient Records Page 10
Figure 8: Clinic configuration page 15
Figure 9: Access Control page 15
Figure 10: Audits & Report page 16
Figure 11: Pharmacy Dashboard Page 16
Figure 12: Add New Medicine Page 17
Figure 13: Update Medicine Page 17
Figure 14: External Dispensing Entry 18
Figure 15: Inventory Management Page 18
Figure 16: Dispensing Queue Page 19
Figure 17: Admin Dashboard Page 19
Figure 18: Admin Queue Management Page 20
Figure 19: Register Patient Page 20
Figure 20:Admin Book Appointment Page 21
Figure 21: Admin Clinic Report Page 21
Figure 22: Admin Billing Page 22
Figure 23: Appointment Hub 22
Figure 24: use case diagrams 55
Figure 25 : Activity Diagrams 90
Figure 26 :Cloud Native Diagram 120
Figure 27: ER Diagram 123
Figure 28: Sequence Diagrams 130
Figure 29: User Flow Diagram 141
Figure 30: Authentication Flow Diagram 145
Figure 31: Security Architecture Diagram 155
Figure 32: Network Diagram 157
Figure 33: Environment Specification 158
Figure 34: CI-CD Diagram 161
Figure 35: Proof Reading Certificate - Page 01 162
Figure 36: Proof Reading Certificate - Page 02 163
Figure 37: Proof Reading Certificate - Page 03 164
Figure 38: Client Feedback Report - Page 01 165
Figure 39 Client Feedback Report - Page 02 166
Figure 40: Client Feedback Report - Page 03 167

List of Tables

TABLE 1 PROJECT TIMELINE PLANNED VS. ACTUAL STATUS 33
TABLE 2 FUNCTIONAL REQUIREMENTS 39
TABLE 3: NON-FUNCTIONAL REQUIREMENTS 44
TABLE 4: BUSINESS RULES 52
TABLE 5 : USE CASE DESCRIPTION 60
TABLE 6: DATA ENTITIES 99
TABLE 7: ALTERNATIVE SOLUTIONS 106
TABLE 8: FEASIBILITY STUDY 108
TABLE 9: PERMISSION MATRIX 148

Abbreviations

SRS – Software Requirements Specification

SDS – System Design Specification

UML – Unified Modeling Language

FR – Functional Requirement

NFR – Non-Functional Requirement

BR – Business Rule

CON – Constraint

ASM – Assumption

API – Application Programming Interface

REST – Representational State Transfer

JSON – JavaScript Object Notation

CRUD – Create, Read, Update, Delete

SQL – Structured Query Language

UI – User Interface

UX – User Experience

RBAC – Role-Based Access Control

JWT – JSON Web Token

2FA – Two-Factor Authentication

HTTPS – HyperText Transfer Protocol Secure

TLS – Transport Layer Security

AES – Advanced Encryption Standard

RSA – Rivest-Shamir-Adleman

SHA – Secure Hash Algorithm

HMAC – Hash-based Message Authentication Code

XSS – Cross-Site Scripting

CSRF – Cross-Site Request Forgery

HIPAA – Health Insurance Portability and Accountability Act

ER – Entity-Relationship

1NF – First Normal Form

2NF – Second Normal Form

3NF – Third Normal Form

AWS – Amazon Web Services

CHAPTER 1: INTRODUCTION

The OneCare Clinic Integrated Pharmacy Management System is a digital solution designed to modernize clinic operations by integrating patient management, appointment scheduling, and pharmacy management into a single platform.

Currently, the clinic manages patient records manually while operating the pharmacy as a separate system. This fragmented approach causes delays, increases the risk of human error, and limits efficient inventory tracking.

The proposed system centralizes the management of patient records, appointments, prescriptions, and pharmacy operations, enabling improved efficiency, accuracy, and overall service quality.

1.1 Client Details

The client for this project is a clinic management authority responsible for overseeing the daily operations of a small to medium-scale healthcare facility. The clinic provides general medical consultation services along with an integrated pharmacy unit.

Currently, the clinic operates with partially manual processes and lacks a fully integrated digital system to manage its operations. Therefore, the client requires a centralized software solution to improve efficiency, accuracy, and service quality.

The client plays a critical role throughout the project by providing system requirements, validating functionalities, and approving the final system deliverables. The client also represents the primary end-user environment in which the system will be deployed.

1.2 Problem Statement

Many small and medium-scale clinics continue to rely on manual or semi-digital systems for managing patient records, appointments, and pharmacy operations. These practices introduce several operational challenges.

Firstly, manual record keeping leads to inefficiencies in storing and retrieving patient information. Secondly, the absence of integration between different functional areas, such as appointment management and pharmacy services, results in delays and inconsistencies. Additionally, human errors in data entry and prescription handling can negatively impact patient safety and service quality.

Furthermore, inventory management within the pharmacy is often poorly monitored, leading to issues such as stock shortages or expired medicines. The lack of real-time reporting and monitoring capabilities also limits the ability of clinic management to make informed decisions.

Therefore, there is a need for a comprehensive system that can address these limitations and streamline clinic operations.

1.3 Proposed Solution Overview

To address the identified challenges, this project proposes the development of the OneCare Clinic Integrated Pharmacy Management System, a web-based application designed to centralize and automate clinic operations.

The system integrates multiple functional modules, including patient management, appointment scheduling, doctor consultation, prescription handling, pharmacy management, inventory tracking, and billing. In addition, the system provides role-based access control to ensure secure and efficient usage by different users such as administrators, doctors, and pharmacists.

The proposed solution aims to enhance operational efficiency by reducing manual tasks, improving data accuracy, and enabling seamless communication between different system components. It also provides features such as reporting, audit logging, and patient self-service capabilities, which contribute to better decision-making and improved patient experience.

Overall, the system is expected to provide a scalable, secure, and user-friendly platform for managing clinic and pharmacy operations effectively.

1.4 Stakeholders

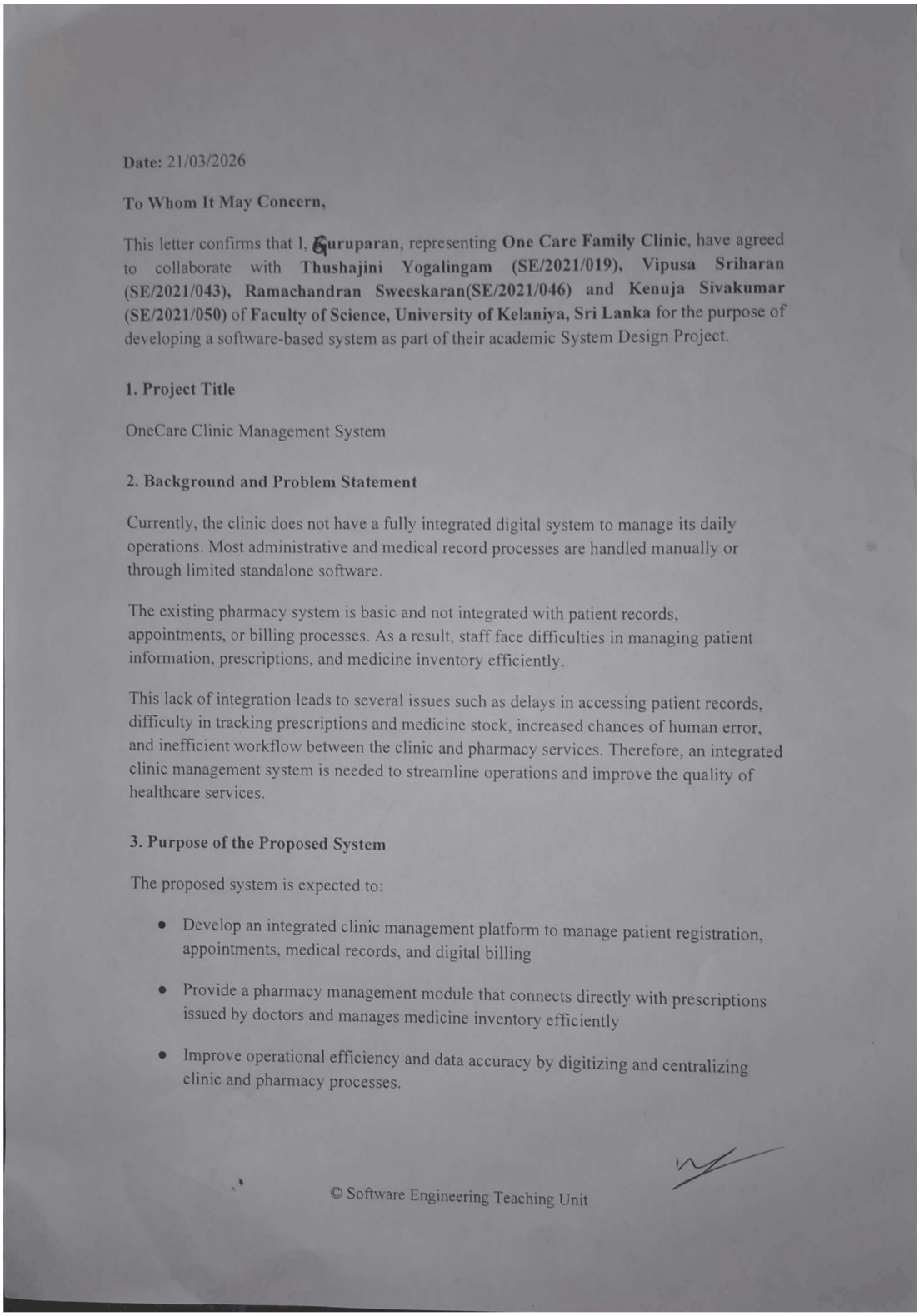
The successful development and operation of the system involve multiple stakeholders, each with specific roles and responsibilities.

The primary stakeholder is the clinic owner or management, who acts as the project sponsor and decision-maker. Medical professionals, including doctors, use the system to manage patient consultations and generate prescriptions. Pharmacy staff are responsible for handling medicine dispensing and maintaining inventory records. Administrative staff manage appointments, patient registration, and billing processes.

In addition, a super administrator is responsible for managing system configurations, user roles, and access permissions. Patients also act as external stakeholders, as they interact with the system for services such as appointment booking and receiving medical care.

The development team is responsible for designing, implementing, and maintaining the system, while the project supervisor provides academic and technical guidance throughout the project lifecycle.

1.5 Client Agreement Letter



Date: 21/03/2026

To Whom It May Concern,

This letter confirms that I, **Guruparan**, representing **One Care Family Clinic**, have agreed to collaborate with **Thushajini Yogalingam (SE/2021/019), Vipusa Sriharan (SE/2021/043), Ramachandran Sweeskaran(SE/2021/046) and Kenuja Sivakumar (SE/2021/050)** of **Faculty of Science, University of Kelaniya, Sri Lanka** for the purpose of developing a software-based system as part of their academic System Design Project.

1. Project Title

OneCare Clinic Management System

2. Background and Problem Statement

Currently, the clinic does not have a fully integrated digital system to manage its daily operations. Most administrative and medical record processes are handled manually or through limited standalone software.

The existing pharmacy system is basic and not integrated with patient records, appointments, or billing processes. As a result, staff face difficulties in managing patient information, prescriptions, and medicine inventory efficiently.

This lack of integration leads to several issues such as delays in accessing patient records, difficulty in tracking prescriptions and medicine stock, increased chances of human error, and inefficient workflow between the clinic and pharmacy services. Therefore, an integrated clinic management system is needed to streamline operations and improve the quality of healthcare services.

3. Purpose of the Proposed System

The proposed system is expected to:

- Develop an integrated clinic management platform to manage patient registration, appointments, medical records, and digital billing
- Provide a pharmacy management module that connects directly with prescriptions issued by doctors and manages medicine inventory efficiently
- Improve operational efficiency and data accuracy by digitizing and centralizing clinic and pharmacy processes.

© Software Engineering Teaching Unit

CamScanner

Figure 1 Client Agreement Letter Page 01

4. Scope of Collaboration

We agree to:

- Provide necessary information related to current processes
- Participate in requirement discussions and clarification meetings
- Review and validate the Software Requirement Specification (SRS)
- Provide feedback on the proposed design and prototype

The student developers agree to:

- Maintain confidentiality of all proprietary or sensitive information disclosed by the client
- Use client-provided information solely for the purposes of this academic project
- Provide regular progress updates and notify the client of any material changes to the project scope

5. Nature of the Project

This project is undertaken solely for academic purposes. We understand and agree that:

- **To the maximum extent permitted by applicable law, the university of Kelaniya and the student developers expressly disclaim all warranties, whether express, implied, statutory or otherwise, including any implied warranties of merchantability, fitness for a particular purpose, or non-infringement. The university and student developers shall have no liability whatsoever for any direct, indirect, incidental, special, consequential or punitive damages, data loss, or financial impact arising from or related to the use or inability to use this software.**

6. Confidentiality (If Applicable)

- **"Confidential Information"** means any non-public data, documents, trade secrets, or business information disclosed by the client in connection with this project. The student developers and the University shall: (a) use Confidential Information solely for the purposes of this academic project; (b) not disclose it to any third party without the client's prior written consent; and (c) apply reasonable measures to protect it from unauthorised access. These obligations shall survive for a period of two (2) years following the completion or termination of the project. Confidential Information does not include information that is or becomes publicly known through no breach of this agreement, or that was already in the receiving party's possession prior to disclosure. Upon written request, all Confidential Information shall be returned or securely destroyed.

© *Software Engineering Teaching Unit*

CamScanner

Figure 2 Client Agreement Letter Page 02

7. Intellectual Property

- All intellectual property rights in the software, code, designs, documentation, and other deliverables created by the student developers as part of this project ("OneCare Clinic Management System") shall vest in and remain the property of the student developers and/or the University of Kelaniya. The client is granted a non-exclusive, non-transferable, royalty-free licence to use the Project OneCare Clinic Management System solely for internal evaluation and testing purposes. The client shall not reproduce, distribute, sublicense, modify, or commercially exploit the Project IP without the prior written consent of the University. The client acknowledges that any data, logos, trademarks, or pre-existing materials provided by the client remain the client's exclusive property.

8. Term and Termination

- This agreement shall commence on the date of signing and shall remain in effect until the final submission of the project deliverables to the Faculty of Science, University of Kelaniya, or upon written notice of termination by either party. Either party may terminate this agreement by providing fourteen (14) days' written notice. Upon termination, the student developers shall return or destroy any Confidential Information in their possession, and the licence granted under Section 7 shall cease immediately.

I/We hereby confirm that the above project request is genuine and that we require the proposed solution as described.

Client Name: DR V. GURUPARAN.

Designation: Medical Officer.

Organization: OneCare Family Clinic

Contact Information: 0770563046.

Signature: _____

Official Seal: Dr.V.Guruparan MD (UKR) FID (UK) SLMC No. 38242 OneCare Family Clinic

Date of Signing: 25/03/2026.

© Software Engineering Teaching Unit

CamScanner

Figure 3 Client Agreement Letter Page 03

CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION

This chapter presents the Software Requirement Specification (SRS) developed during Stage 2 of the project. It systematically documents the requirements elicited from the client, provides an analysis of the existing system, and defines both the functional and non-functional requirements of the proposed system. These specifications establish the foundation for the system design described in the subsequent chapter.

The complete SRS document is included in Appendix 1 for further reference.

In conclusion, this chapter has detailed the comprehensive requirements of the proposed system, including use case specifications, feasibility analysis, and risk assessment. The following chapter, Chapter 3, presents the System Design Specification derived from the approved requirements.

CHAPTER 3: SYSTEM DESIGN SPECIFICATION (SDS)

This chapter presents the System Design Specification (SDS) developed during Stage 3 of the project. Building upon the approved Software Requirement Specification outlined in Chapter 2, this chapter provides a comprehensive description of the system's design, including the architectural framework, database design, class structure, sequence diagrams, user interface design, security considerations, and deployment strategy for the proposed system.

The complete SDS document is included in Appendix 2 for reference.

In summary, this chapter has outlined the overall design of the proposed system, encompassing architectural design, database schema, class structures, interaction flows, user interface design, security mechanisms, and deployment planning. The subsequent chapter, Chapter 4, presents the prototype implementation and the client validation process.

CHAPTER 4: PROTOTYPE & CLIENT VALIDATION

4.1 Prototype (Figma navigation flow)

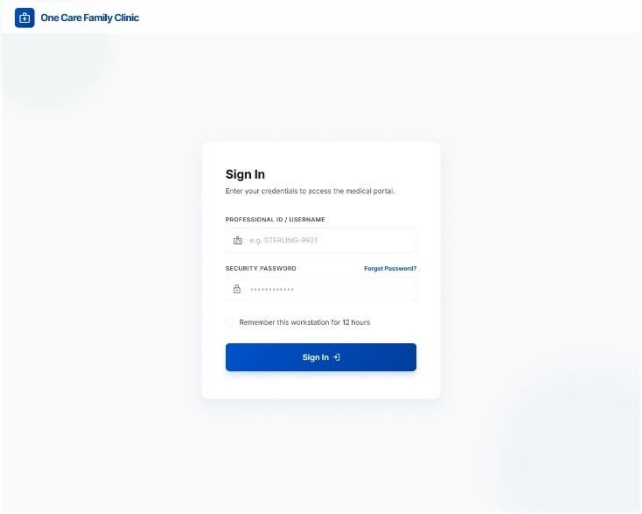


Figure 4: Medical system login page

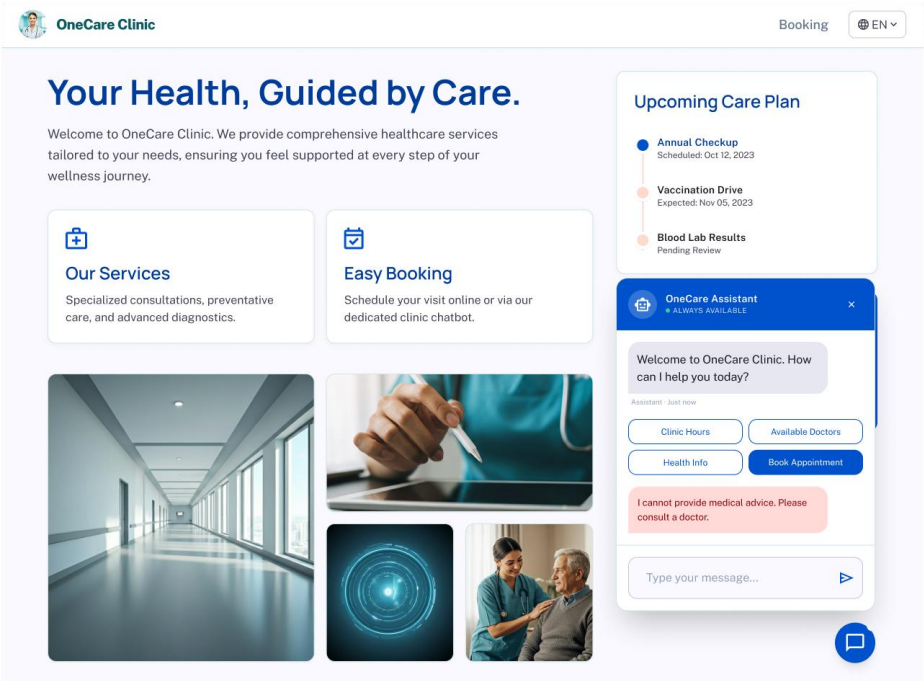


Figure 5 Patient Webpage

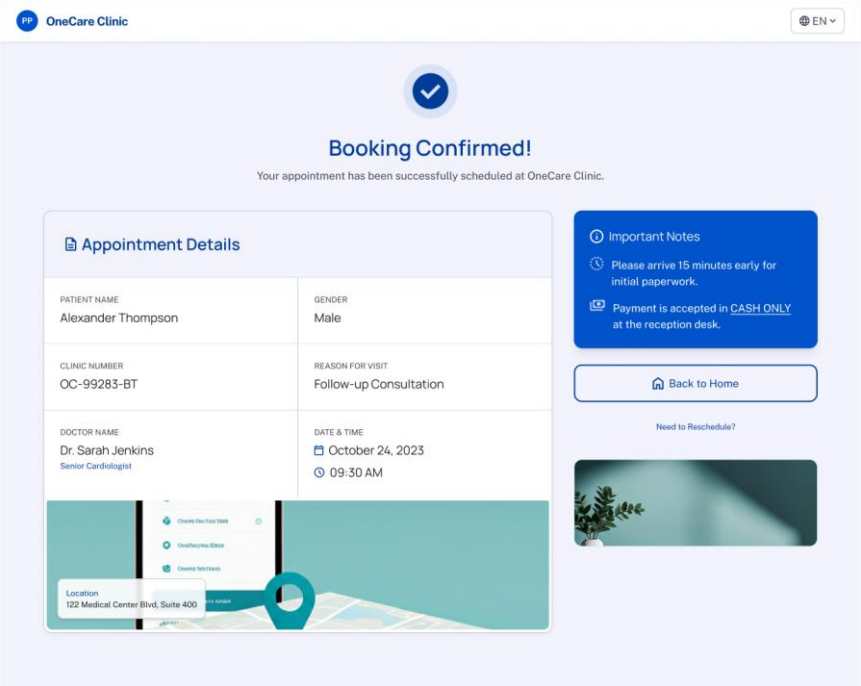


Figure 6: Booking Confirmed Page

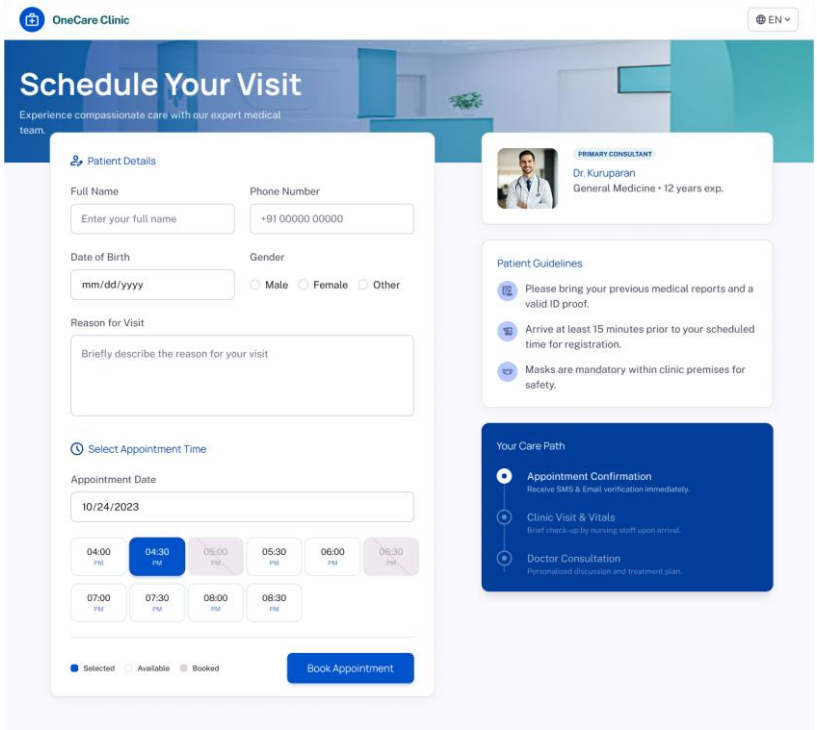


Figure 5: Patient Appointment Page

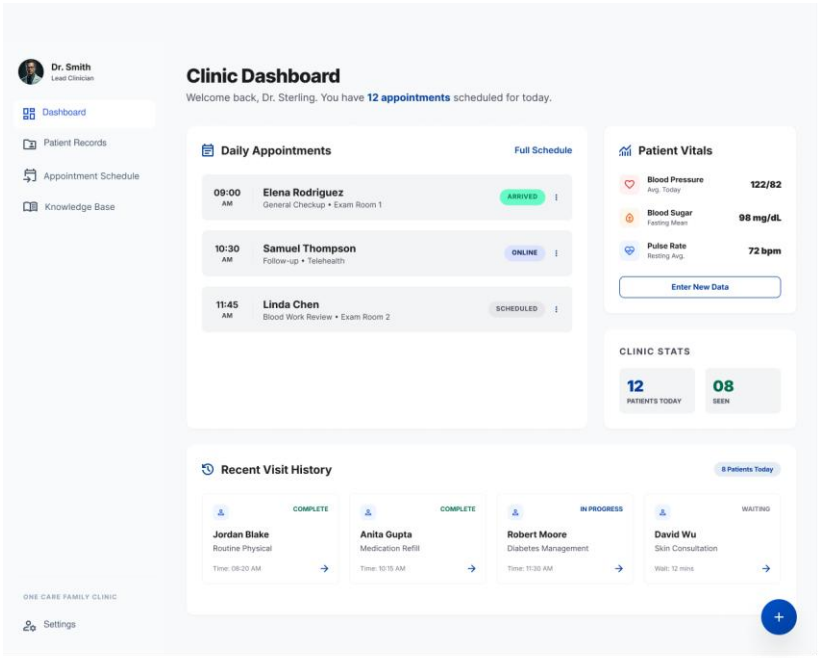


Figure 6: Doctor Dashboard

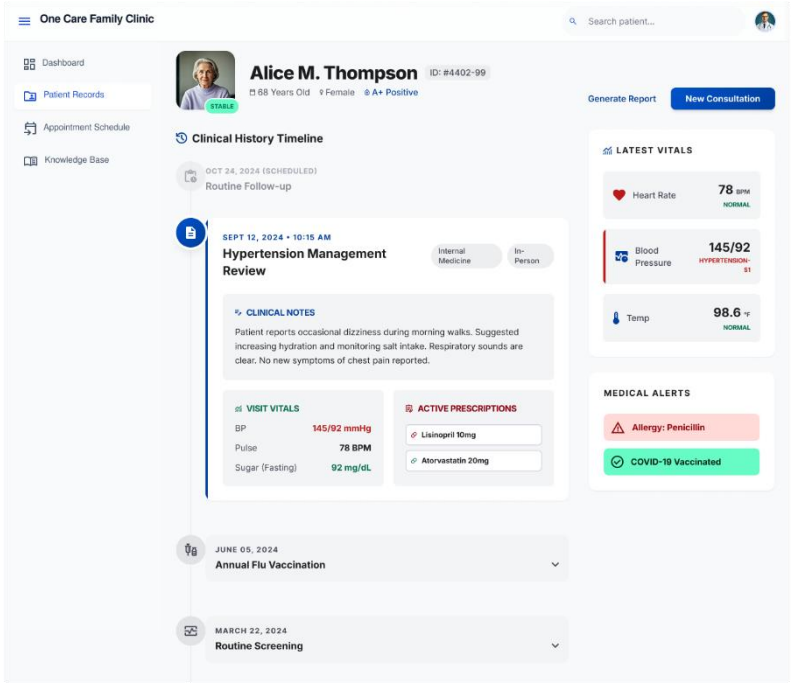


Figure 7: Patient Records Page

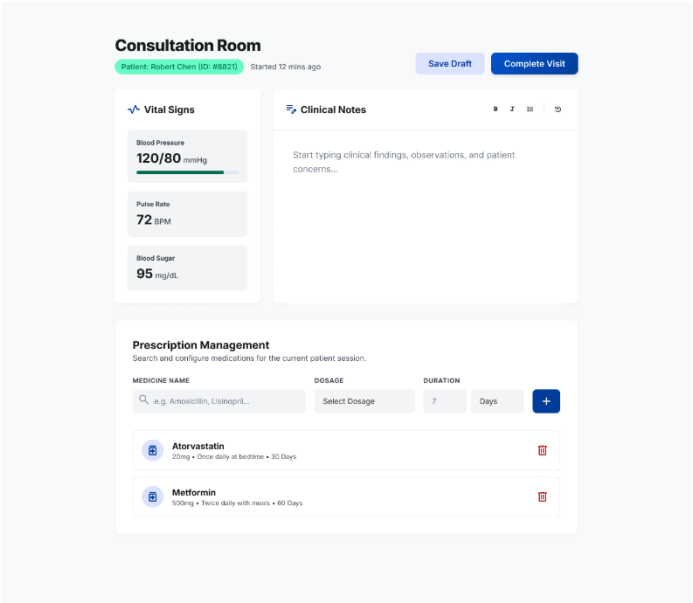


Figure8: Consultation Page

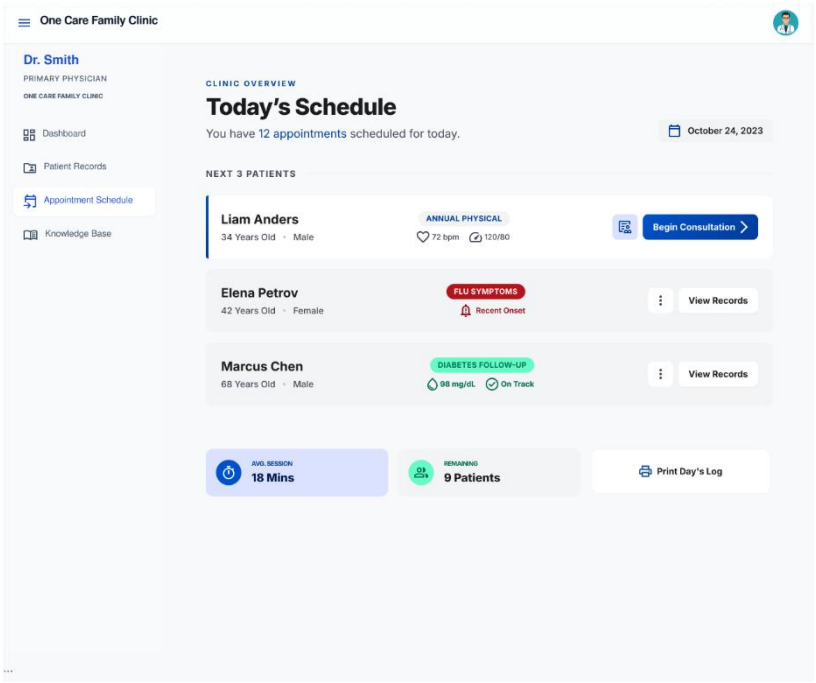


Figure 9: Today schedule Page

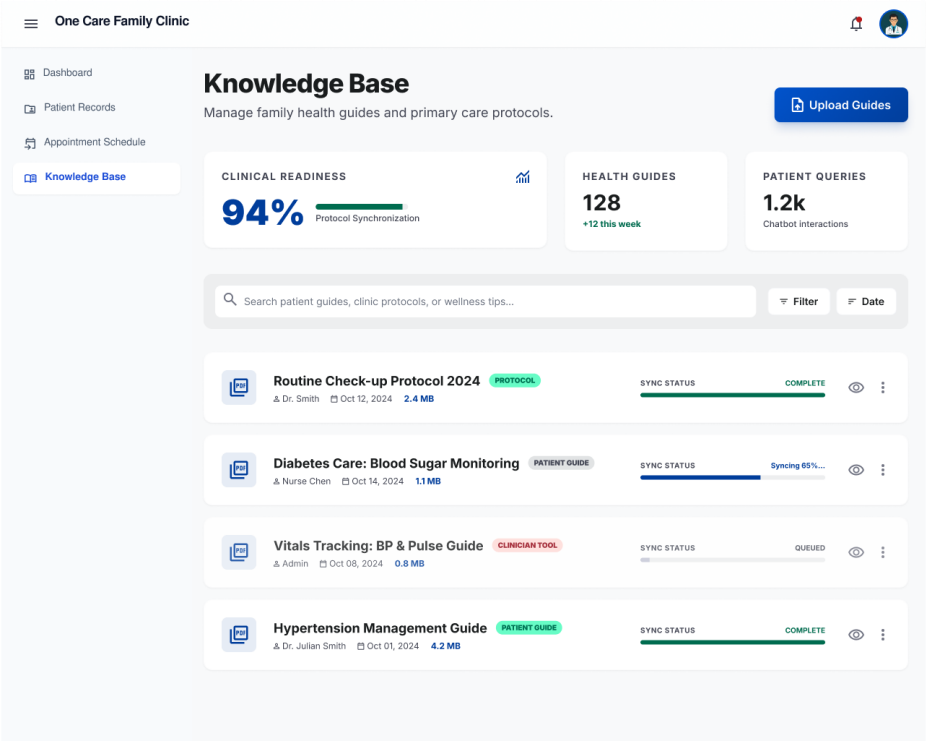


Figure 10: Knowledgebase Page

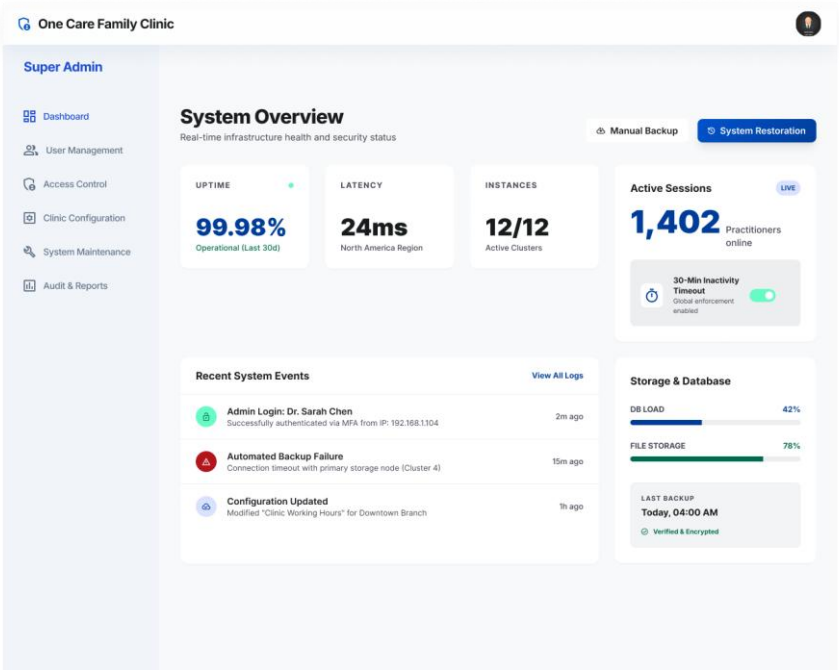


Figure 11: Super Admin Dashboard page

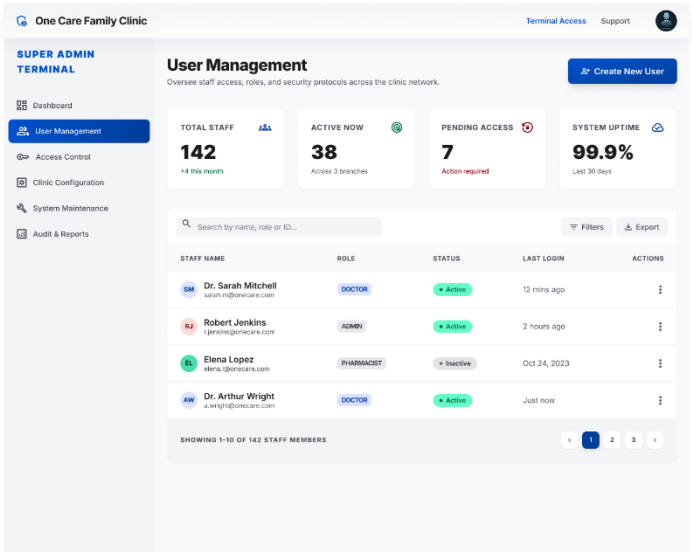


Figure 12: User management page

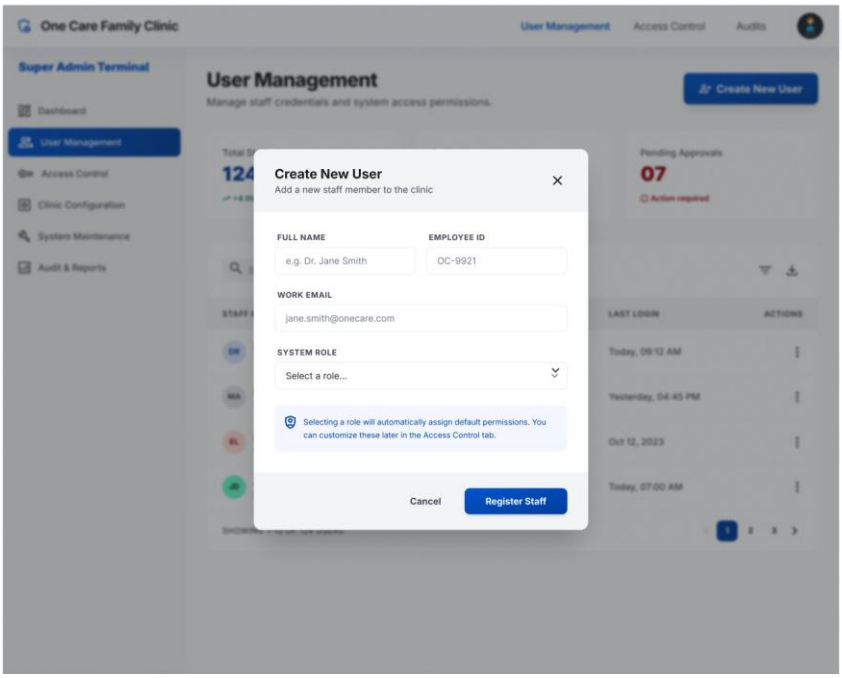


Figure 13: Create user page

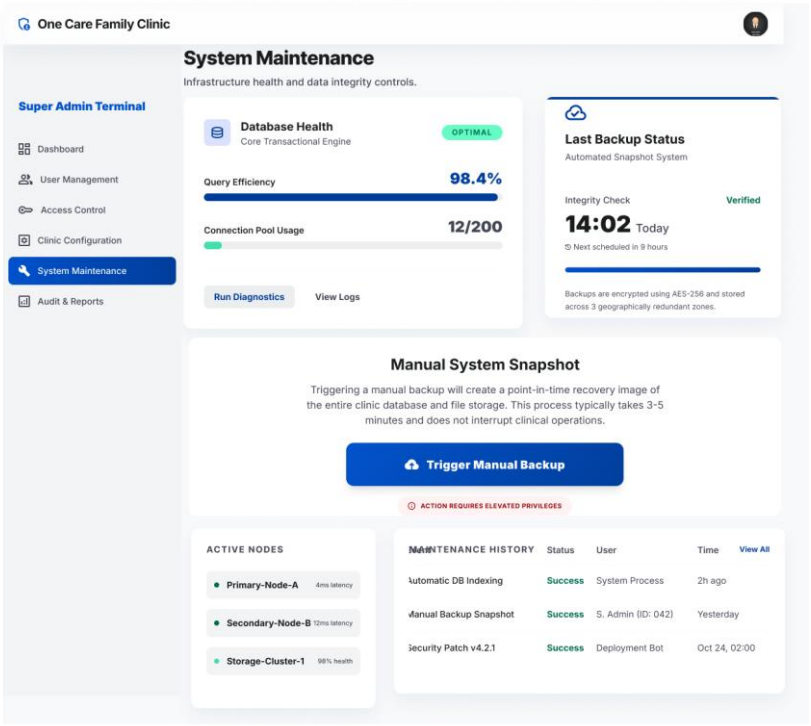


Figure 14: System maintenance page

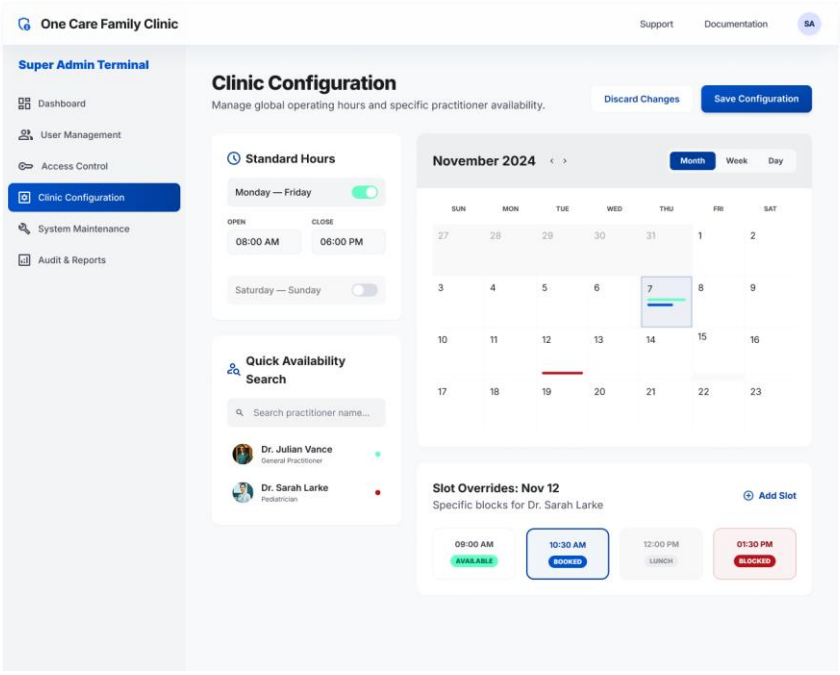


Figure 8: Clinic configuration page

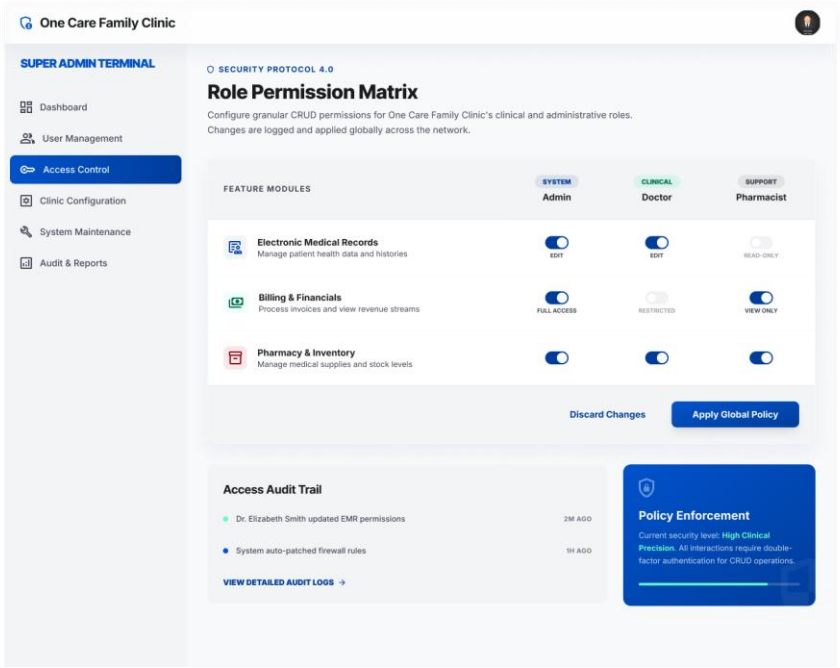


Figure 9: Access Control page

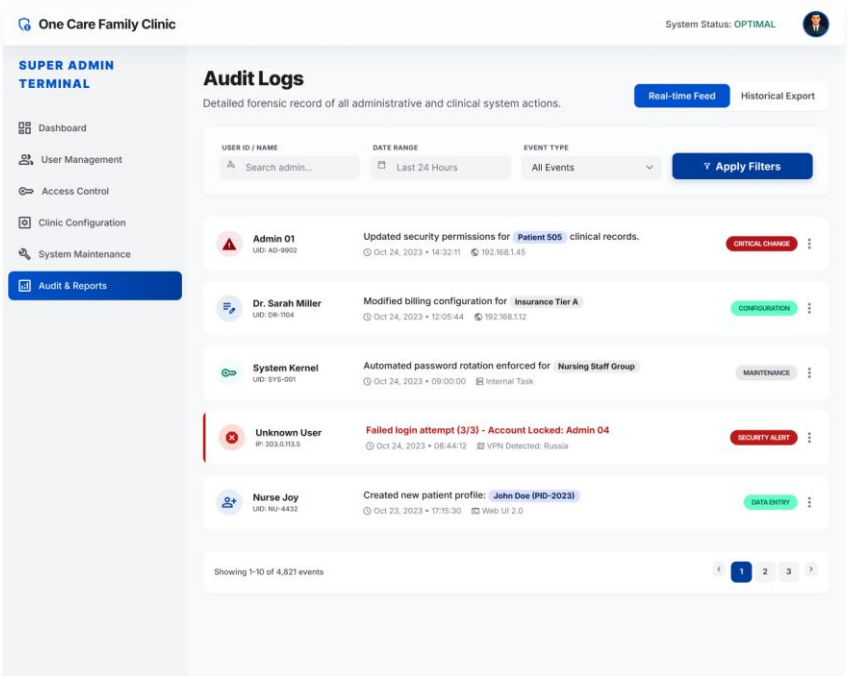


Figure 10: Audits & Report page

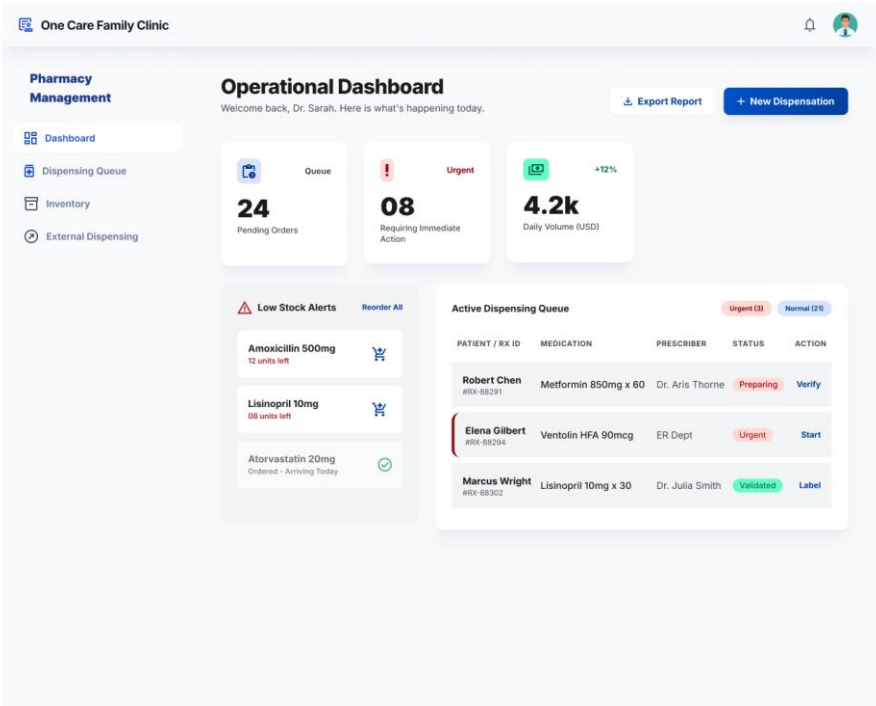


Figure 11: Pharmacy Dashboard Page

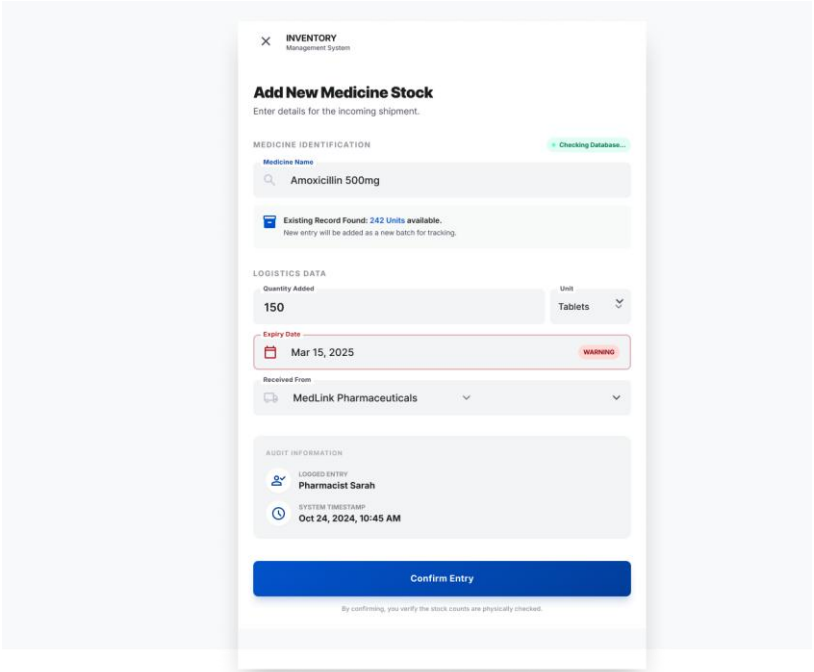


Figure 12: Add New Medicine Page

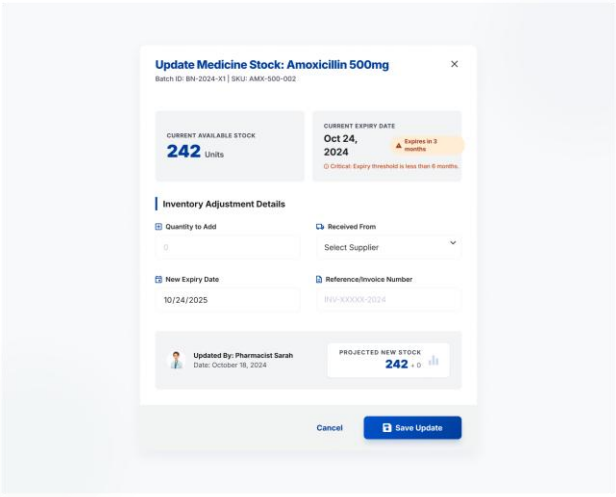


Figure 13: Update Medicine Page

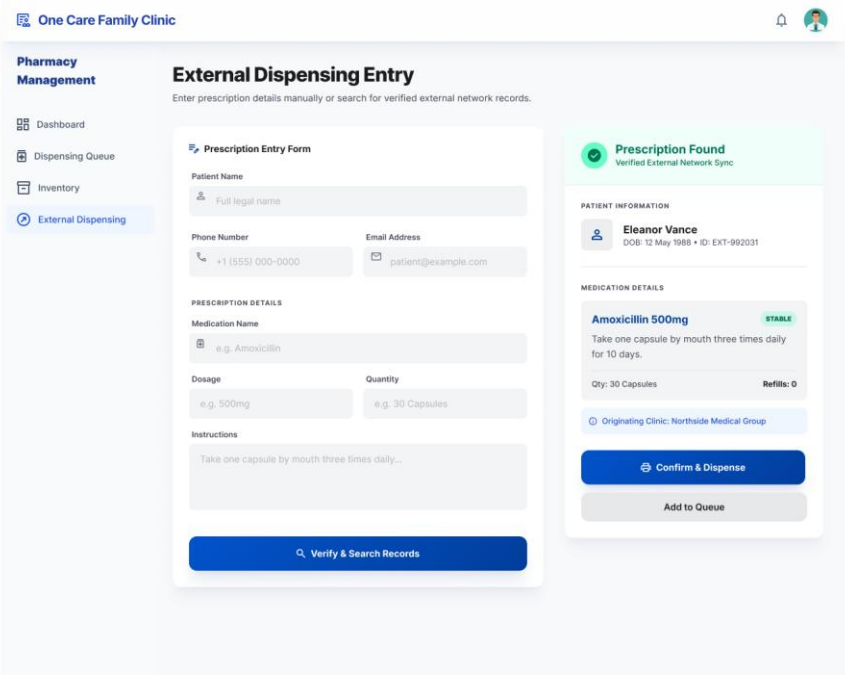


Figure 14: External Dispensing Entry

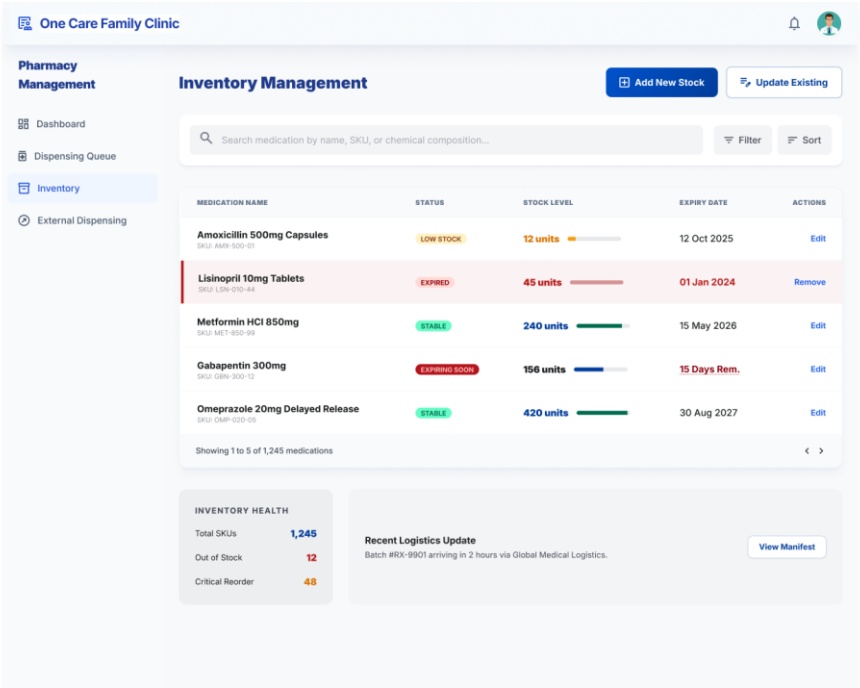


Figure 15: Inventory Management Page

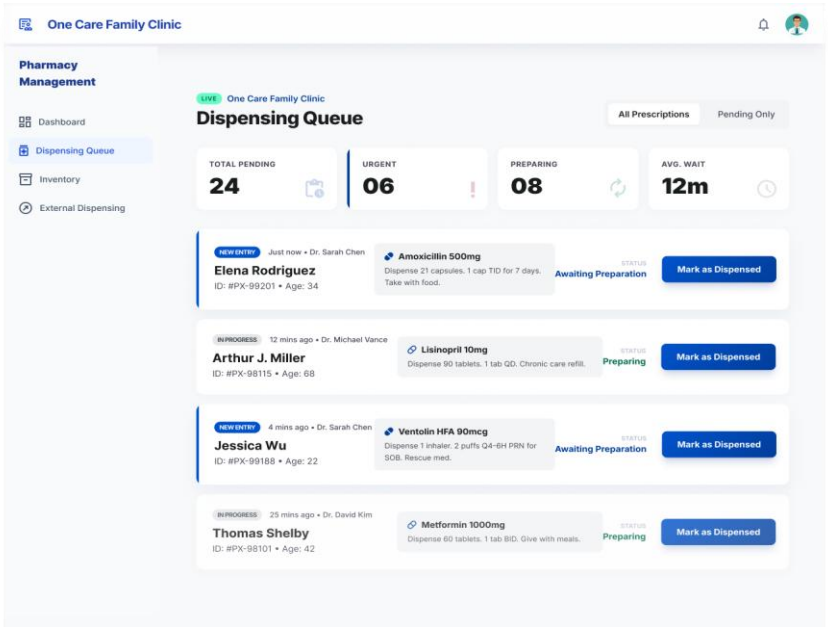


Figure 16: Dispensing Queue Page

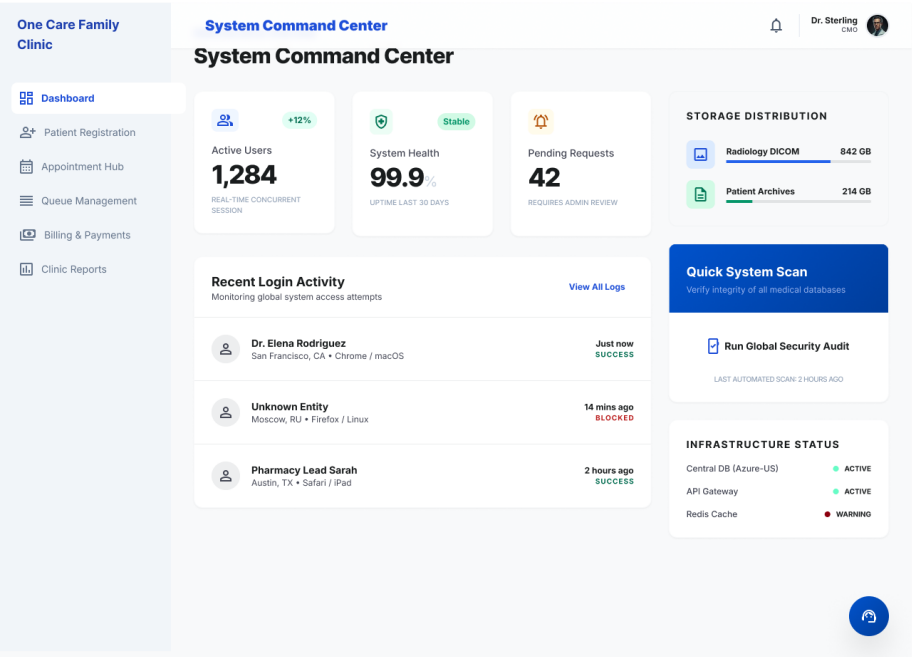


Figure 17: Admin Dashboard Page

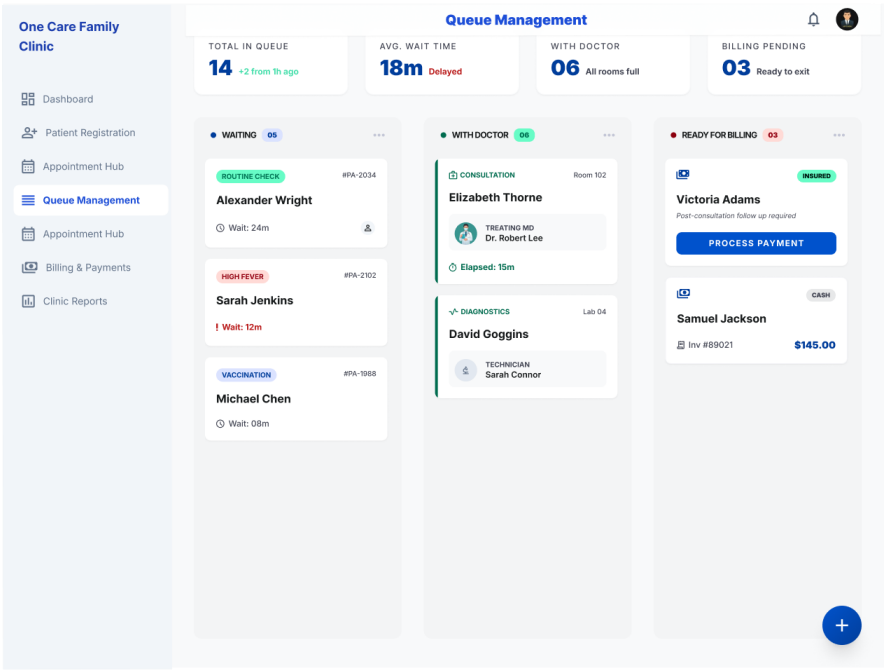


Figure 18: Admin Queue Management Page

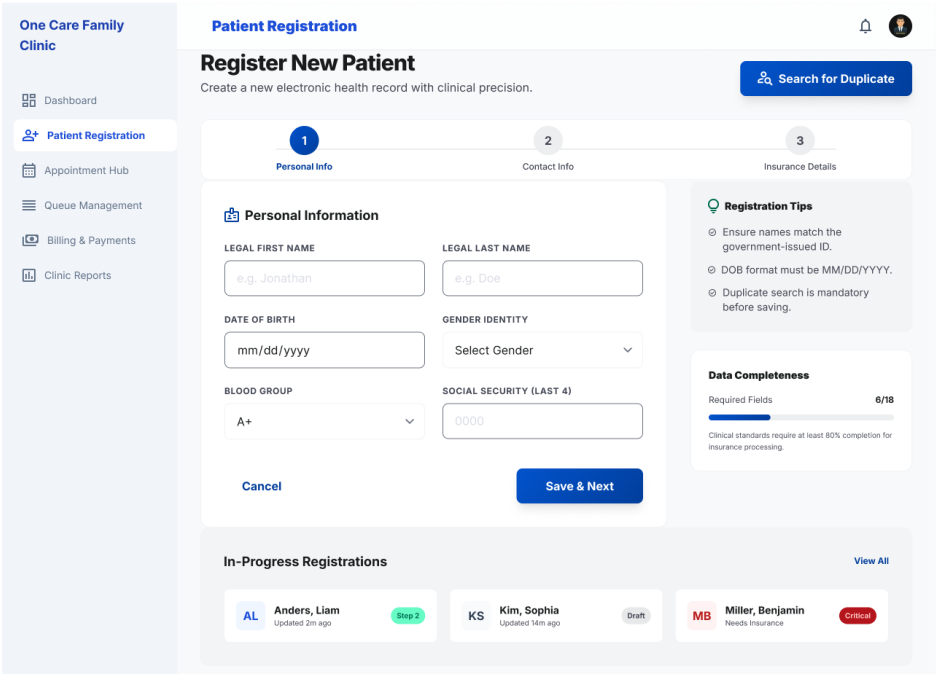


Figure 19: Register Patient Page

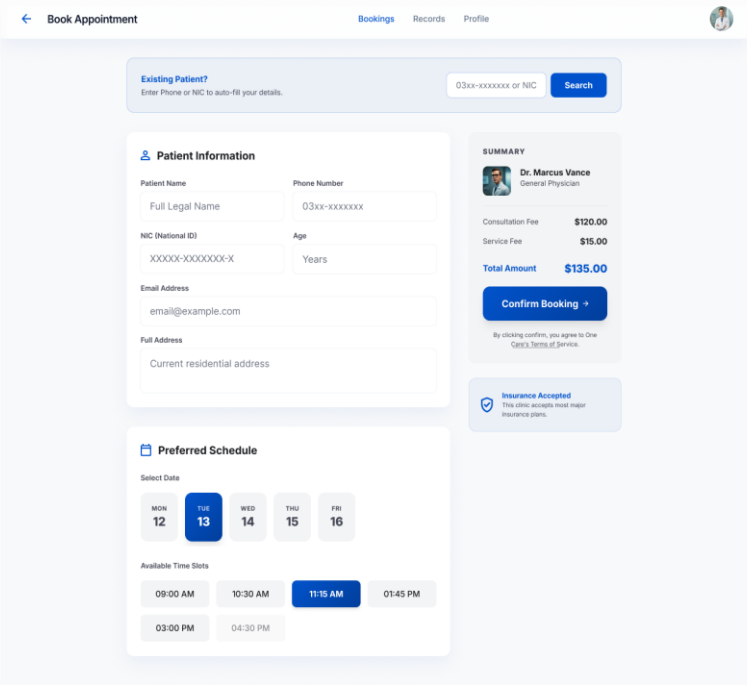


Figure 20:Admin Book Appointment Page

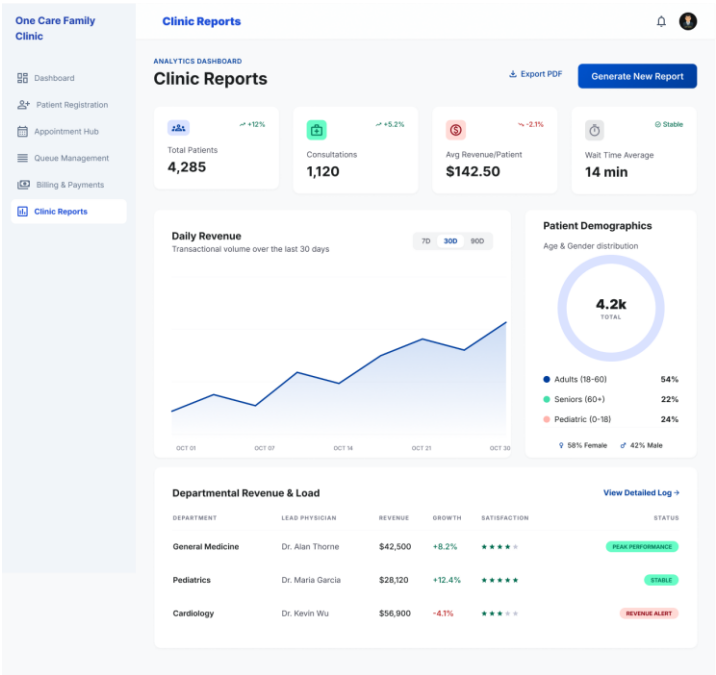


Figure 21: Admin Clinic Report Page

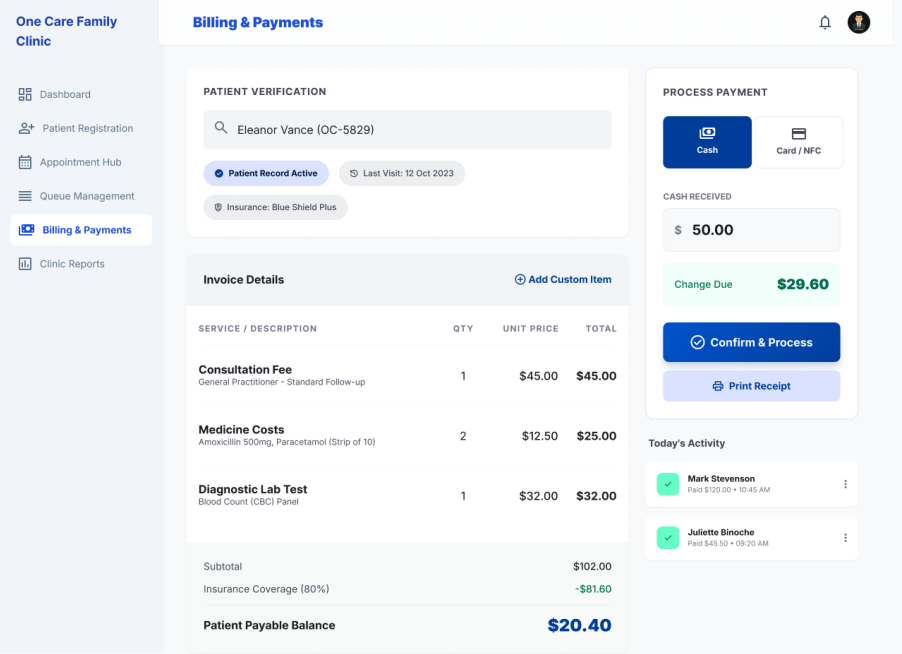


Figure 22: Admin Billing Page

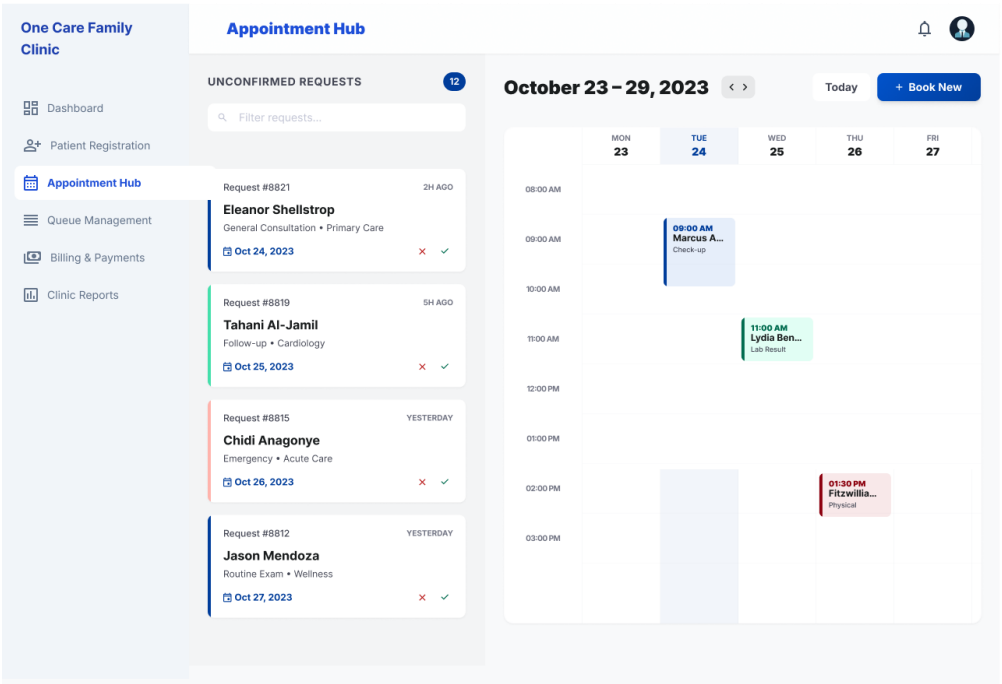


Figure 23: Appointment Hub

UI/UX Design – Prototype

The system UI/UX design has been developed using Figma to demonstrate the complete user interface flow and interaction design. The prototype includes navigation between key modules such as authentication, dashboard, patient management, and related system functionalities.

The interactive prototype can be accessed using the following link: prototype link

4.2 Client feedback

A client visit was conducted for the clinical system project, during which the Figma design and supporting documentation were presented. The client reviewed the materials and provided several constructive feedback points for improvement.

1. It was suggested that the system logo be repositioned within the designated red rounded region to improve visual alignment and branding consistency.
2. It was recommended that the font size be increased to enhance readability, particularly to support users of all age groups, including elderly users.
3. The client advised that guidance hints or instructional messages be added within the chatbot interface to assist first-time users in navigating the system effectively.
4. It was suggested that additional medical-related symbols be incorporated into the web interface to improve user recognition of the healthcare context and to enhance the professional appearance of the system. The client provided sample symbols as reference for implementation.
5. The client inquired about future system scalability, particularly the possibility of supporting multiple doctors within the clinical system as the clinic expands. It was agreed that system requirements will be revised to accommodate future scalability and client growth needs.

4.3 Client validation

The proposed system design, including the Figma prototype and documentation, was reviewed and validated by the client. The client expressed satisfaction with the overall design direction and confirmed that the system aligns with operational requirements of the clinical environment.

The validation process confirmed that the proposed solution is acceptable at the current design stage, while minor improvements and refinements were identified for implementation in subsequent iterations.

4.4 Ethical considerations

The One Care Medical Clinic System is designed as a web-based platform that facilitates interaction among doctors, pharmacists, administrators, and super administrators for managing patient records, appointments, and pharmacy-related activities. As the system

processes sensitive medical data, strict ethical considerations are required to ensure patient safety, data privacy, and responsible system usage.

The key ethical objectives include:

- Ensuring patient safety
- Protecting patient privacy and confidentiality
- Promoting responsible and authorized system usage

Ethical policies are enforced throughout the system lifecycle, from user authentication to final system usage.

Patient Privacy and Confidentiality

Principle: All patient medical histories, prescriptions, and personal information (e.g., contact details) must be kept confidential and accessible only to authorized personnel.

Implementation:

- Role-Based Access Control (RBAC) is enforced to restrict access based on user roles. Clinical data are accessible only to doctors, while administrative users access non-clinical information.
- Patient records are accessible solely by authorized medical staff.
- Each patient is assigned a unique identifier (e.g., Patient_ID) to prevent duplication and ensure continuity of medical records across visits.

Data Security

Principle: All sensitive healthcare data must be protected from unauthorized access, breaches, and cyber threats.

Implementation:

- User passwords are securely stored using hashing mechanisms.
- Accounts are temporarily locked after multiple failed login attempts.
- Communication between client and server is secured using HTTPS with TLS 1.2 or higher.
- Data are securely maintained within the database environment.

Role-Based Responsibility

Principle:

System users are restricted to actions appropriate to their professional roles to prevent misuse of authority and sensitive data.

Implementation:

- Doctors are permitted to create prescriptions but are restricted from modifying pharmacy inventory.
- Pharmacists are allowed to dispense medications but are restricted from accessing or modifying patient medical records.
- Administrative users are limited to non-clinical operations.
- The Super Administrator manages system-level permissions and user roles.

Prescription Integrity and Safety

Principle**:**

Prescriptions must be accurate, secure, and generated only by qualified professionals.

Implementation**:**

- Digital prescriptions are created exclusively by authenticated doctors.
- Prescriptions cannot be modified by unauthorized personnel.
- The system prevents re-dispensing of completed prescriptions.
- Prescription statuses (e.g., Pending, Fully Dispensed, Partially Dispensed) are tracked.

Accountability and Auditability

Principle**:**

All system activities must be traceable to ensure accountability and detect misuse or errors.

Implementation**:**

- System actions (e.g., login, prescription creation, dispensing, payments) are recorded in audit logs.
- Logs include timestamps, user identifiers, and activity details.
- Audit logs can be reviewed and exported by the Super Administrator for monitoring and compliance.

Data Accuracy and Reliability

Principle**:**

All system data must be accurate and validated to minimize medical errors.

Implementation**:**

- Input validation is enforced during patient registration, prescription entry, and billing processes.
- Duplicate patient records are prevented through validation mechanisms.
- Medication stock levels and expiration dates are monitored to prevent unsafe dispensing.

Safe Use of AI / Chatbot

Principle**:**

Automated systems must not provide misleading or unsafe medical advice.

Implementation**:**

- The chatbot is restricted to providing general medical information only.
- Information generated by the system is subject to verification by medical professionals.
- If symptoms are not recognized, users are prompted to consult a qualified doctor.

Legal and Regulatory Compliance

Principle**:**

The system must comply with healthcare data protection laws and ethical standards.

Implementation**:**

- Patient data are handled in accordance with healthcare privacy regulations.
- Access to medical records is strictly limited to authorized personnel.
- Data retention policies are enforced to define storage duration and archival procedures.

Data Backup and Continuity

Principle**:**

System and patient data must be protected against loss to ensure uninterrupted healthcare services.

Implementation:

- Backup and restore mechanisms are implemented within the system.
- Regular data backups are maintained.
- Recovery procedures are established to ensure system continuity in case of failures.

Patient Safety in Dispensing

Principle**:**

Medications must be dispensed safely and only to the correct patient.

Implementation**:**

- Patient identity verification is required prior to dispensing.
- Multiple verification methods (e.g., ID, phone number, QR code) are supported.
- Dispensing activities are logged with staff identifiers and timestamps.

4.5 Communication reflection

Communication

Effective communication was maintained throughout the client interaction. The system design was presented clearly, and key functionalities were explained in a structured and professional manner during the meeting with the doctor and clinical team, including pharmacists, administrative staff, and the super administrator.

Prior to the meeting, the availability of the doctor was confirmed due to patient engagement in the clinic waiting area. Accordingly, the meeting was conducted at the scheduled time to ensure minimal disruption to clinical operations.

The use of visual prototypes (Figma design) facilitated a clear understanding of the system workflow and user interface. The client actively participated in the discussion and provided valuable feedback based on real-world clinical requirements and comparisons with existing systems. The client emphasized only the inclusion of necessary and practical features aligned with actual operational needs.

Furthermore, the onsite visit enabled direct interaction with the client, allowing immediate clarification of requirements and efficient collection of feedback. This ensured that the development team accurately captured the client's expectations.

Reflection

The client meeting was highly valuable in validating the system design and identifying practical improvements. The feedback highlighted important usability considerations, particularly regarding accessibility for elderly users and the need for improved user guidance within the system.

The suggestion to support multiple doctors emphasized the importance of designing a scalable system architecture to accommodate future expansion. This insight has been incorporated into the refinement of system requirements.

In addition, the positive response from the client indicated that the project is progressing in the correct direction. The appreciation of the Figma design and documentation reflects the effectiveness of the proposed solution. The client's statement that the system exceeded expectations demonstrates the strength of the design approach.

Overall, the communication process strengthened the alignment between client expectations and system development objectives, enabling improved clarity and direction for the subsequent stages of the project lifecycle.

CHAPTER 5: CONCLUSION

The OneCare Clinic Integrated Pharmacy Management System was designed and developed as a comprehensive digital solution to modernize the operations of a private medical clinic with an in-house pharmacy. The primary aim of this project was to replace the clinic's fragmented, paper-based processes with a unified, web-based platform capable of managing patient records, appointment scheduling, queue management, doctor consultations, prescription generation, pharmacy dispensing, inventory tracking, external dispensing, billing, and management reporting. Built on a layered architecture using Spring Boot and React, the system was designed to serve multiple roles Super Admin, Admin, Doctor, and Pharmacist with role-based access control ensuring data security and operational clarity.

Throughout the course of this project, the team navigated real-world challenges including aligning system design with the practical workflows of healthcare professionals, managing complex interdependencies between modules, and ensuring that the prototype met both functional and usability expectations of the client. The following sections reflect on how well the project's objectives were realized, the quality attributes of the system, limitations encountered, timeline performance, and potential avenues for future improvement.

5.1 Degree of Objectives Met

The project sets out a series of well-defined functional and non-functional objectives at the outset. The table below evaluates the extent to which each key objective was achieved during the design and prototype phases of the OneCare system.

Objective	**Degree Met**	**Explanation**
Develop a role-based access control system supporting Super Admin, Admin, Doctor, and Pharmacist roles	Fully Met	A complete role-permission matrix was designed and enforced throughout all system modules. Each role operates within clearly defined access boundaries as specified in the SRS.
Digitize patient record management to replace paper-based processes	Fully Met	Comprehensive digital patient records including personal details, medical history, and consultation logs were designed. The admin module supports

Objective	Degree Met	Explanation
		full CRUD operations and patient search by name, phone number, or clinic ID.
Implement an automated appointment scheduling and queue management module	Fully Met	The appointment module supports scheduling, rescheduling, and cancellation with double-booking prevention. The daily queue is generated automatically based on scheduled appointments.
Design a Doctor Module for recording clinical notes and issuing digital prescriptions	Fully Met	The Doctor Module was fully designed with features for viewing appointments, accessing patient histories, recording clinical notes, and generating digital prescriptions with dosage, duration, and instructions.
Build a Pharmacy Module for prescription dispensing and real-time inventory management	Fully Met	The Pharmacy Module handles prescription viewing, dispensing validation, status tracking (Fully/Partially Dispensed), inventory management, expiry tracking, and low-stock alert generation.
Implement a billing and invoicing system for consultation and medicine charges	Partially Met	Billing design was completed to support cash-only invoice generation and payment recording. Online payment processing was intentionally excluded by business rule BR-011 and project scope boundaries.
Implement an External Dispensing Module with patient identity verification	Fully Met	The External Dispensing Module was designed to support off-site prescription fulfilment, identity verification via

Objective	Degree Met	Explanation
		phone number, clinic ID, or QR code, and immediate inventory deduction with audit logging.
Provide management reporting, audit logging, and a patient chatbot	Fully Met	Clinic and pharmacy reports, filterable audit logs, and a chatbot restricted to predefined doctor-approved content were all included in the system design and prototype.

Overall, many of the defined objectives were fully realized. The only partially met objective relates to the billing module, where online payment processing was intentionally excluded from scope in consultation with the client and project supervisor, as documented in business rule BR-011. This decision-maintained project feasibility while preserving the integrity of the core clinical workflows.

5.2 Usability, Accessibility, Reliability and Friendliness

Usability

The OneCare system was designed with a strong emphasis on usability. The high-fidelity Figma prototype was developed following established UI/UX principles, incorporating clear navigation hierarchies, consistent visual language, and role-specific dashboards that present only relevant information to each user. During the client validation phase, medical staff confirmed that the interface reflected their actual clinical workflows. Iterative client feedback was incorporated into the prototype across review cycles. In accordance with non-functional requirement NFR-U002, a new staff member shall be able to perform basic tasks such as patient registration, queue updates, and invoice generation within 30 minutes of initial use. Common tasks are designed to require no more than three navigation steps, as specified in NFR-U003, minimizing friction for clinic staff.

Accessibility

Accessibility was considered throughout the design process. The system is web-based and accessible through modern desktop browsers, ensuring compatibility across Chrome, Firefox, Edge, and Safari as specified in NFR-PO001. The interface supports responsive layouts on desktops and tablets, with correct functioning ensured for screen widths of 360px

and above, as per NFR-U004. The system interface supports both English and Tamil for all patient-facing pages in accordance with NFR-U001, addressing the multilingual requirement of the clinic's patient base. Readable fonts, appropriate color contrast, and inline form validation messages further support ease of use for staff of varying technical proficiency.

Reliability

Reliability was a core non-functional requirement embedded in the system architecture. The system is designed to achieve a minimum uptime of 99% per month, excluding planned maintenance, as defined in NFR-R001. Offline operational continuity is supported for queue updates, appointment logging, and billing for up to five minutes, with automatic synchronization upon connectivity restoration (NFR-R002, NFR-R003). The layered Spring Boot backend provides a modular and maintainable foundation, while automated daily database backups (NFR-R004) and system recovery within four hours of failure (NFR-R005) ensure data integrity. Invent alertsagement features, including expiry tracking and low-stock alerts, further contribute to operational reliability by preventing clinical errors arising from outdated or depleted medicine stocks.

User-Friendliness

The system was designed to be intuitive for non-technical healthcare staff. Each user role — Doctor, Pharmacist, Admin, and Super Admin — is presented with a contextually appropriate dashboard focused on their required tasks. The patient self-service chatbot module provides patients with an automated support channel for querying clinic schedules and services, reducing administrative burden on clinic staff. Chatbot responses are restricted to predefined, doctor-approved content as specified in business rules BR-009 and BR-010, ensuring clinical safety. The client validation phase confirmed that the prototype's user flows were well received, reflecting positively on the system's overall friendliness and readiness for adoption.

5.3 Limitations and Drawbacks

Despite the considerable achievements of the OneCare project, several limitations and drawbacks were identified during the design and prototyping phases.

Single-Clinic Scope: The system has been designed exclusively for a single-doctor, single-pharmacy clinic environment as specified in CON-006. The architecture does not currently support multi-clinic deployments or centralized management across multiple branches, limiting scalability for clinic owners who may wish to expand operations.

No Online Payment Integration: The project scope explicitly excludes online payment processing by business rule BR-011, restricting the billing module to cash-only transactions. This is a recognized limitation in the context of modern healthcare digitization, where patients increasingly expect digital payment options. While the rationale — accessibility for patients who do not use digital payment services — is documented in the alternative solutions analysis, it remains an acknowledged gap in the current system.

No Telemedicine Support: The system does not support telemedicine or remote consultation features, which have become increasingly relevant in contemporary healthcare. Patients requiring remote consultations must use external platforms, resulting in a fragmented experience outside the OneCare system.

Design-Phase Constraints: As this project was primarily design and prototype exercise, full system implementation and production-level testing were not carried out. Performance under real-world load conditions, edge-case handling, and integration with external health systems could not be empirically validated within the project scope.

Limited Chatbot Capability: The chatbot module is scoped to handle basic patient queries using predefined knowledge base documents. Advanced natural language understanding, appointment booking through conversational interfaces, or AI-driven triage capabilities were not included, limiting the chatbot's practical utility compared to modern healthcare virtual assistants.

No Insurance Claims Processing: The system does not support insurance claim submission or tracking, which is a significant operational need for clinics serving insured patients. Manual processing of insurance claims remains necessary, adding administrative overhead outside the scope of the OneCare platform.

5.4 Timeline

The project was structured across eight phases as defined in the Schedule Feasibility assessment (SRS Section 12.4.1), with a total estimated timeline of approximately 28 weeks. The table below presents the planned duration, milestone, and actual status for each phase against the original plan.

Table 1 Project Timeline Planned vs. Actual Status

Phase	Description	Planned Duration	Actual Status
1	Requirements Analysis & SRS Documentation	3 weeks	Completed on time

Phase	Description	Planned Duration	Actual Status
2	System Design (Architecture, Database, UI Mockups)	3 weeks	Completed on time
3	Frontend Development (React.js, core modules)	6 weeks	Completed on time
4	Backend Development (APIs, Auth, DB integration)	6 weeks	Not yet started (design phase only)
5	Integration Testing & Bug Fixing	3 weeks	Not yet started
6	User Acceptance Testing (UAT) with clinic staff	2 weeks	Not yet started
7	Deployment, Data Migration & Training	2 weeks	Not yet started
8	Buffer Time	3 weeks	—

Overall, the project progressed in line with the planned schedule through the design phases. Phases 1 and 2, covering requirements analysis, SRS documentation, and system design, were completed on time, supported by structured requirements elicitation sessions with the client and effective team collaboration. Phase 3, the frontend UI prototype developed using Figma and React, was also completed within the planned duration, incorporating client validation feedback across iterative review cycles.

Phases 4 through 7 covering backend development, integration testing, user acceptance testing, and deployment have not yet commenced, as this project is currently at the design phase. These phases are planned for execution in subsequent project stages. Phase 8, the buffer period, has been reserved for contingency and remains available. If the project were to be repeated, the team would initiate Figma prototyping in parallel with the SRS phase to reduce the overall duration and would allocate additional time in Phase 4 to account for the complexity of backend API integration and database implementation.

5.5 Future Modifications, Improvements and Extensions Possible

The OneCare Clinic Management System has been designed with scalability and extensibility in mind. Several enhancements and extensions are envisioned for future iterations of the platform.

Online Payment Integration: Incorporating digital payment gateways such as credit or debit card processing or mobile payment platforms would significantly modernize the billing module and improve patient convenience. This was excluded from the current scope based on the cash-payment practice of the target clinic environment but remains the highest-priority future enhancement as patient payment preferences evolve.

Multi-Clinic Support: Extending the architecture to support multiple clinic branches under a centralized administration layer would allow clinic groups to manage staff, patients, and inventory across locations from a single platform. This would require architectural changes to the database schema and API layer to introduce multi-tenancy capabilities.

Telemedicine Module: A dedicated telemedicine feature enabling video consultations between doctors and patients would greatly expand the system's utility, particularly for follow-up consultations and patients with mobility limitations. Integration with WebRTC-based communication frameworks or third-party telehealth APIs could facilitate this capability without requiring significant changes to the existing system architecture.

AI-Enhanced Chatbot: The current chatbot module could be enhanced with natural language processing capabilities to enable intelligent, conversational appointment booking, symptom triage, and proactive health reminders. This would substantially improve patient engagement while maintaining the safety constraints established in business rules BR-009 and BR-010 through appropriate content filtering and supervision.

Insurance and Claims Integration: Future versions of the system could incorporate insurance claim submission and tracking functionality, enabling the clinic to handle insured patients more efficiently and reduce manual claim processing overhead. This would require integration with relevant insurance provider APIs and compliance with applicable healthcare data exchange standards.

Mobile Application: Developing a companion mobile application for patients would allow appointment booking, prescription viewing, notification receipt, and chatbot interaction directly from smartphones. This would complement the existing web-based system and improve patient accessibility, particularly given the high mobile usage rates in the clinic's patient demographic.

Advanced Analytics Dashboard: A more sophisticated reporting and analytics layer incorporating visual dashboards, trend analysis across patient volumes and pharmacy throughput, and predictive inventory management would provide clinic management with actionable operational insights beyond the standard reports currently designed.

In conclusion, the OneCare Clinic Integrated Pharmacy Management System represents a significant step toward the digital transformation of private clinical practice. The project has provided the team with invaluable experience in requirements engineering, system architecture, user-centered design, client communication, and project management. Academic modules in Software Requirements Engineering, System Design, Human-Computer Interaction, Database Management, and Project Management directly informed the methodological approach applied throughout. The challenges navigated from reconciling technical design with real-world healthcare workflows, to managing scope boundaries and iterative client feedback have strengthened the team's capability to deliver complex, client-facing software solutions in a professional engineering context. The experience gained through this project will be directly applicable to future career development in the software engineering field.

REFERENCES

[1] React Official Documentation, “React – A JavaScript library for building user interfaces,” Available: https://react.dev/

[2] Spring Boot Official Documentation, “Spring Boot Reference Guide,” Available: https://docs.spring.io/spring-boot/

[3] Spring Security Documentation, “Authentication and Authorization with Spring Security,” Available: https://docs.spring.io/spring-security/reference/

[4] Auth0, “JSON Web Token (JWT) Introduction and Best Practices,” Available: https://auth0.com/docs/secure/tokens/json-web-tokens

[5] IETF, “RFC 7519: JSON Web Token (JWT) Specification,” Available: https://datatracker.ietf.org/doc/html/rfc7519

[6] MDN Web Docs, “HTTP Authentication and Security Concepts,” Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication

[7] Docker Documentation, “Docker Overview and Containerization Concepts,” Available: https://docs.docker.com/get-started/

[8] GitHub Actions Documentation, “Automating CI/CD Workflows,” Available: https://docs.github.com/en/actions

[9] Scribd, “Ethical Considerations in System Design Document,” Available: https://www.scribd.com/document/873626736/DOC-20250215-WA0009

[10] Taylor & Francis Online, “Ethical and Legal Considerations in Healthcare Systems,” *International Journal of General Medicine*, Available: https://www.tandfonline.com/doi/full/10.2147/IJGM.S368262

[11] PubMed Central, “Ethical Issues in Healthcare Information Systems,” Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC8720823/

[12] The Auditorium Blog, “Reflection in Action: Living a Project Management Theory,” Available: https://theauditorium.blog/2024/10/31/reflection-in-action-living-a-project-management-theory/

ANNEX 1: SOFTWARE REQUIREMENTS SPECIFICATION

1.1 Project Scope And Stakeholders

1.1.1 Project Scope

The project aims to develop a web-based integrated system that improves clinical efficiency and pharmacy management.

Key goals:

- Reduce manual paperwork
- Improve patient experience
- Automate prescription processing
- Maintain accurate medicine inventory
- Provide management reports

Support for approximately 25 patients per day shall be provided by the system managed by one doctor and one pharmacy.

1.1.2 Stakeholders

Stakeholder	**Role**
Clinic Owner	Decision maker and project sponsor
Doctor	Creates prescriptions and manages patient records
Pharmacist	Dispenses medicine and manages stock
Super Admin	Manage system users, permissions, and reports
Admin	Manage queues, appointments
Patients	Book appointments and receive notifications
Development Team	Build and maintain the system

1.2 Functional Requirements

1.2.1 User & Access Management

Table 2 Functional Requirements

ID	Description	Priority	Source
FR-001	The system shall support the roles: Super Admin, Admin, Doctor, and Pharmacist, and access shall be controlled using a role permission matrix.	High	System
FR-002	User accounts shall be created, updated, deactivated, and assigned roles by Super Admin.	High	Super Admin
FR-003	Role-based permissions shall be configurable and enforceable across all system modules.	High	Super Admin
FR-004	Users shall be authenticated using a valid username and password.	High	System
FR-005	Users shall be redirected to role-specific dashboards after successful authentication.	High	System
FR-006	User accounts shall be locked for 15 minutes after five consecutive failed login attempts.	High	System
FR-007	Password reset shall be performed with a time limited email link.	High	System
FR-008	User sessions shall be terminated after 30 minutes of inactivity.	High	System

1.2.2 Clinic Configuration

ID	Description	Priority	Source
FR-009	Doctor schedules including working days and time slots shall be configured by the Super Admin.	High	Super Admin

FR-010	System data backup and restore operations shall be performed by Super Admin.	Medium	Super Admin
FR-011	System-wide reports and audit logs shall be accessible to Super Admin.	High	Super Admin

1.1.1 Patient Management

ID	Description	Priority	Source
FR-012	Patient records shall be created by capturing personal and contact details.	High	Admin
FR-013	Non-medical patient information shall be updated by Admin users.	High	Admin
FR-014	Patient records shall be searchable using name, phone number, or clinic number.	High	Admin

1.2.3 Appointment Management

ID	Description	Priority	Source
FR-015	Appointments shall be created by assigning patient, doctor, date, and time slot and preventing double booking.	High	Admin
FR-016	Existing appointments shall be rescheduled.	High	Admin
FR-017	Appointments shall be cancelled when required.	High	Admin
FR-018	Appointment statuses shall be maintained as Scheduled, Completed, Cancelled, or No-show.	High	System

1.2.4 Queue Management

ID	Description	Priority	Source

FR-019	A daily patient queue shall be generated automatically based on appointments.	High	System
FR-020	Queue status shall be updated when a consultation is marked as completed by the Doctor.	High	System
FR-021	Patient arrival and consultation completion shall be recorded by Admin.	High	Admin

1.2.5 Doctor Module

ID	Description	Priority	Source
FR-022	Scheduled appointments shall be viewable by the Doctor.	High	Doctor
FR-023	Patient medical history and past consultations shall be accessible to the Doctor.	High	Doctor
FR-024	Clinical notes shall be recorded during consultations.	High	Doctor
FR-025	Digital prescriptions including medicine, dosage, duration, and instructions shall be created.	High	Doctor
FR-026	Prescriptions shall be marked as “Pending Dispense” after creation.	High	System

1.2.6 Pharmacy Module

ID	Description	Priority	Source
FR-027	Prescriptions with “Pending Dispense” status shall be viewable by Pharmacist.	High	Pharmacist
FR-028	Medicines shall be dispensed after validating prescription details.	High	Pharmacist

FR-029	Prescription status shall be updated as Fully Dispensed or Partially Dispensed.	High	System
FR-030	Duplicate dispensing of completed prescriptions shall be prevented.	High	System

1.1.2 Inventory Management

ID	Description	Priority	Source
FR-031	Medicine records shall be created, updated, and deactivated by Pharmacist.	High	Pharmacist
FR-032	Stock levels shall be maintained and updated after dispensing.	High	Pharmacist
FR-033	Expiry dates shall be tracked and expired medicines shall be marked unavailable.	High	Pharmacist
FR-034	Low stock alerts shall be generated when thresholds are reached.	Medium	Pharmacist

1.2.7 Billing

ID	Description	Priority	Source
FR-035	Invoices shall be generated including consultation and medicine charges.	High	Admin
FR-036	Only cash payments shall be accepted as per BR-011.	High	Admin
FR-037	Invoice status shall be maintained as Paid or Pending.	High	Admin
FR-038	Historical invoices shall be retrievable.	Medium	Admin

1.2.8 Reporting & Audit

ID	Description	Priority	Source

FR-039	Clinic reports including patient counts and consultations shall be generated.	Medium	System
FR-040	Pharmacy reports including stock usage and expiry shall be generated.	Medium	Pharmacist
FR-041	User activities including login, data changes, and dispensing actions shall be logged.	High	System
FR-042	Audit logs shall be viewable with filtering capabilities.	High	Super Admin

1.2.9 Patient Self-Service & Chatbot

ID	Description	Priority	Source
FR-043	Appointments shall be booked by patients without logging in using basic personal details.	High	Patient
FR-044	A chatbot interface shall be provided for querying clinic details such as schedules and services.	Medium	Patient
FR-045	Chatbot responses to medicine-related queries shall be limited to predefined approved information.	Medium	System
FR-046	Health-related responses shall be generated only from doctor-approved PDF knowledge sources.	Medium	Doctor
FR-047	PDF documents used for chatbot knowledge shall be uploaded and managed by authorized users.	Medium	Doctor

1.2.10 External Dispensing Module

ID	Requirement	Priority	Source

FR-048	The system shall allow Pharmacist to mark a prescription for external dispensing to an offsite patient.	High	Pharmacist
FR-049	The system shall require verification of patient identity using phone number, clinic ID, or QR code before dispensing medicines externally.	High	Pharmacist
FR-050	External dispensing shall update the inventory immediately and mark the prescription status as “Dispensed (External)”.	High	System
FR-051	A delivery record shall be generated including patient ID, staff ID, date/time, and payment method.	High	Pharmacist
FR-052	The system shall maintain audit logs for all external dispensing activities.	High	Compliance

1.3 Non-Functional Requirements

1.3.1 Performance

Table 3: Non-Functional Requirements

ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-P-001	Dashboard Performance	Dashboard pages shall be loaded within ≤3 seconds for 95% of requests under ≤50 concurrent users.	High	Admin

NFR-P-002	API Performance	95% of API responses shall be returned within ≤2 seconds.	High	Dev Team
NFR-P-003	Booking Performance	Appointment booking shall be completed within ≤2 seconds.	High	Patient
NFR-P-004	Queue Update Latency	Queue updates shall be reflected within ≤2 seconds after consultation completion is recorded.	High	Staff
NFR-P-005	Notification Time	SMS notifications shall be sent within ≤10 seconds of triggering events.	High	Patient
NFR-P-006	Report Generation	Reports up to 1 year of data shall be generated within ≤10 seconds.	Medium	Admin

1.1.3 Reliability

ID	**Requirement**	**Measurable Acceptance Criterion**	**Priority**	**Source**
NFR-R001	Availability	System uptime shall be ≥99% per month excluding maintenance.	High	Admin

NFR-R002	Offline Capability	Queue updates, appointment logging, and billing shall be allowed to be performed for up to 5 minutes during offline operations; data shall be synchronized automatically after connectivity is restored.	High	Staff
NFR-R003	Data Sync	Data created during offline operation shall be automatically synchronized with the central database within 5 minutes after connectivity is restored.	High	Dev Team
NFR-R004	Backup	Automated backups shall be performed every 24 hours.	High	Admin
NFR-R005	Recovery	System recovery shall be completed within ≤4 hours after failure.	High	Management

1.3.2 Usability

NFR ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-U-001	Multilingual interface	English and Tamil shall be supported by the system interface for all patient facing pages.	High	Staff

NFR-U-002	Ease of learning	Basic tasks (patient registration, queue updates, and invoice generation) shall be able to be performed by a new staff member within 30 minutes of initial use.	High	Staff
NFR-U-003	Task efficiency	Common operational tasks (confirm booking, updating stock, and generate report) shall be required to be completed within no more than three navigation steps.	High	Admin
NFR-U-004	Mobile responsiveness	Correct functioning shall be ensured by the system interface on desktop, tablet, and mobile devices with screen widths ≥360px.	High	Staff / Patients
NFR-U-005	Form validation	Invalid form inputs shall be detected, and inline validation messages shall be displayed before submission.	Medium	Staff

1.3.3 Security

NFR ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-S-001	Password policy	Password complexity rules shall be met, and passwords shall be securely hashed.	High	Admin

NFR-S-002	Session management	Automatic expiration of user sessions shall occur after 30 minutes of inactivity.	High	Dev Team
NFR-S-003	Role-based access control	Role-Based Access Control (RBAC) shall be enforced by the system to restrict access to authorized modules only.	High	Admin
NFR-S-004	Secure communication	All client-server communications shall be encrypted using HTTPS with TLS 1.2 or higher.	High	Dev Team
NFR-S-005	Data protection	Patient and prescription data shall be accessible only to authorized users according to defined access permissions.	High	Compliance
NFR-S-006	Audit logging	Logs for critical actions, including user login, patient record updates, prescription creation, and	High	Admin
		stock changes, shall be recorded by the system.		
NFR-S-007	Input validation	All user inputs shall be validated to prevent SQL injections, cross-site scripting (XSS), and other injection attacks.	High	Dev Team

NFR-S-008	CSRF Protection	Cross-Site Request Forgery (CSRF) protection shall be implemented by the system for all state changing HTTP requests (POST, PUT, DELETE).	High	Dev Team
NFR-S-009	Rate Limiting	Authentication requests shall be limited by the system to a maximum of 10 attempts per minute per IP address to prevent brute-force attacks.	High	Dev Team
NFR-S-010	Data Encryption at Rest	Sensitive data, including patient records and authentication credentials, shall be encrypted at rest using industry-standard encryption (e.g., AES256).	High	Compliance

1.3.4 Scalability

NFR ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-SC001	Concurrent user capacity	Support for at least 50 concurrent users shall be ensured by the system without exceeding the response times defined in NFR-P-001 and NFR-P-002.	High	Dev Team

NFR-SC002	Database growth	Support for up to 20,000 patient records and 5 years of transaction history shall be provided by the system database without performance degradation.	High	Dev Team

1.3.5 Maintainability

NFR ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-M-001	Modular architecture	A modular architecture separating frontend, backend, and database layers shall be followed by the system.	High	Dev Team
NFR-M-002	API documentation	All backend APIs shall be documented using Open API / Swagger standards.	High	Dev Team
NFR-M-003	Code quality	At least 70%-unit test coverage shall be achieved for the backend source code.	Medium	Dev Team
NFR-M-004	Configuration management	Environment-specific configurations shall be stored outside the source code using environment variables or secure configuration files.	High	Dev Team
NFR-M-005	Logging	Application logs shall be stored in a centralized logging system for a minimum of 90 days.	Medium	Dev Team

1.3.6 Portability

NFR ID	Requirement	Measurable Acceptance Criterion	Priority	Source
NFR-PO001	Browser compatibility	Correct functioning shall be ensured by the web application on Chrome, Firefox, Edge, and Safari browsers.	High	Dev Team
NFR-PO002	Container deployment	The application shall be deployable using Docker containers.	High	Dev Team
NFR-PO003	Operating system independence	Operation shall be supported by the system on any operating system that supports Docker containers.	High	Dev Team
NFR-PO004	Database portability	The database schema shall be deployable on MySQL version 8.0 or higher.	High	Dev Team
NFR-PO005	Language extensibility	Additional languages should be provided using separate translation files without modifying the application source code.	Low	Dev Team

1.4 Business Rules, Constraints And Assumptions

1.4.1 Business Rules

Table 4: Business Rules

ID	Rule Description
BR-001	Defined complexity rules shall be complied with by user passwords, and secure hashing mechanisms shall be used for their storage.
BR-002	User accounts shall be temporarily locked after a defined number of consecutive failed login attempts.
BR-003	Each patient shall be assigned a unique patient identifier within the system.
BR-004	Doctors shall be permitted to create prescriptions but shall not be permitted to modify pharmacy inventory records.
BR-005	Pharmacist shall be permitted to dispense medicines but shall not be permitted to modify patient medical records.
BR-006	Administrative users shall be restricted to accessing and modifying non-clinical patient information only.
BR-007	Super Admin users shall be responsible for managing user accounts, permissions, and system-level reports.
BR-008	Password reset operations for system users shall be performed only by the Super Admin, and users shall be required to change the password upon first login after reset.
BR-009	Chatbot responses shall be restricted to general clinic information and predefined health knowledge approved by a qualified doctor.
BR-010	Chatbot shall not provide diagnosis, prescription, or medical advice beyond approved informational content.
BR-011	All patient payments shall be made in cash; no electronic or card payments shall be supported by the system

1.4.2 Constraints

1.4.2.1 Technology Constraints

ID	Constraint
CON-001	The system shall be implemented as a web-based application accessible via modern web browsers (Chrome, Edge, Firefox, Safari).
CON-002	The system shall be compatible with Windows-based systems used within the clinic environment.

1.4.2.2 Legal Constraints

ID	Constraint
CON-003	Patient medical data shall be handled in compliance with applicable health data privacy regulations.
CON-004	Access to patient medical records shall be restricted to authorized medical personnel only.
CON-005	Patient data shall be securely stored and retained in accordance with defined medical record retention policies.

1.4.2.3 Operational Constraints

ID	Constraint
CON-006	Operations for a single-doctor clinic environment shall be supported by the system.
CON-007	Operational continuity shall be maintained by the system under moderate internet connectivity conditions.

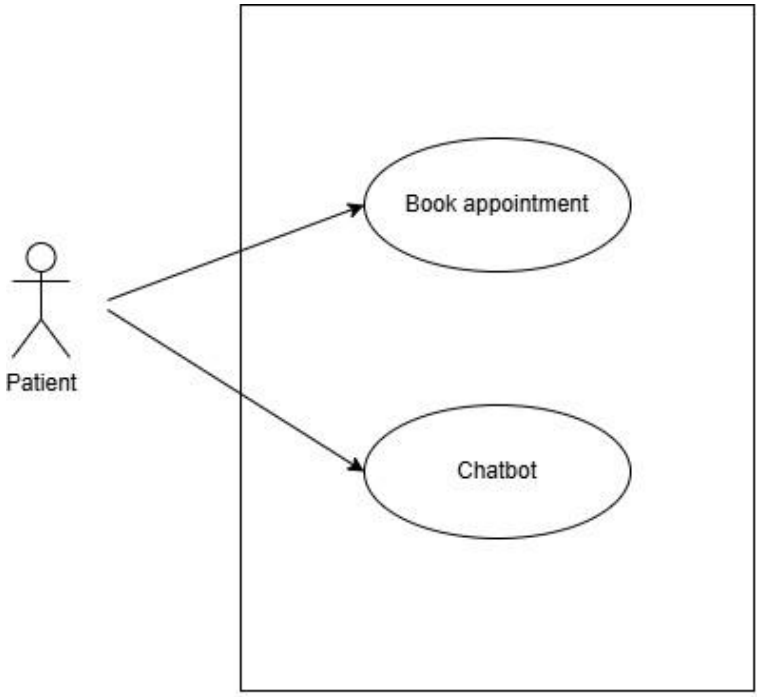
1.4.3 Assumptions

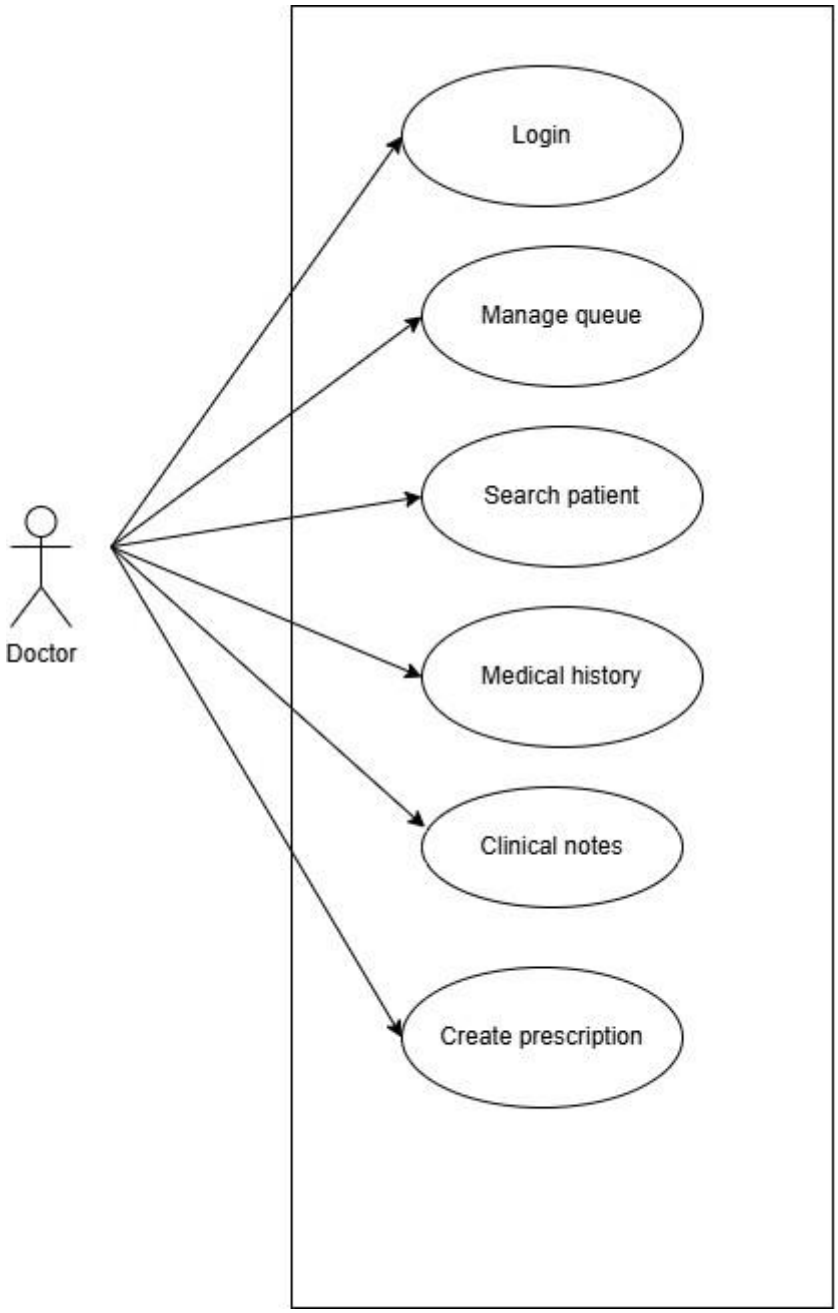
ID	Assumption Description
ASM-001	Staff are assumed to possess basic computer literacy to operate the system effectively.

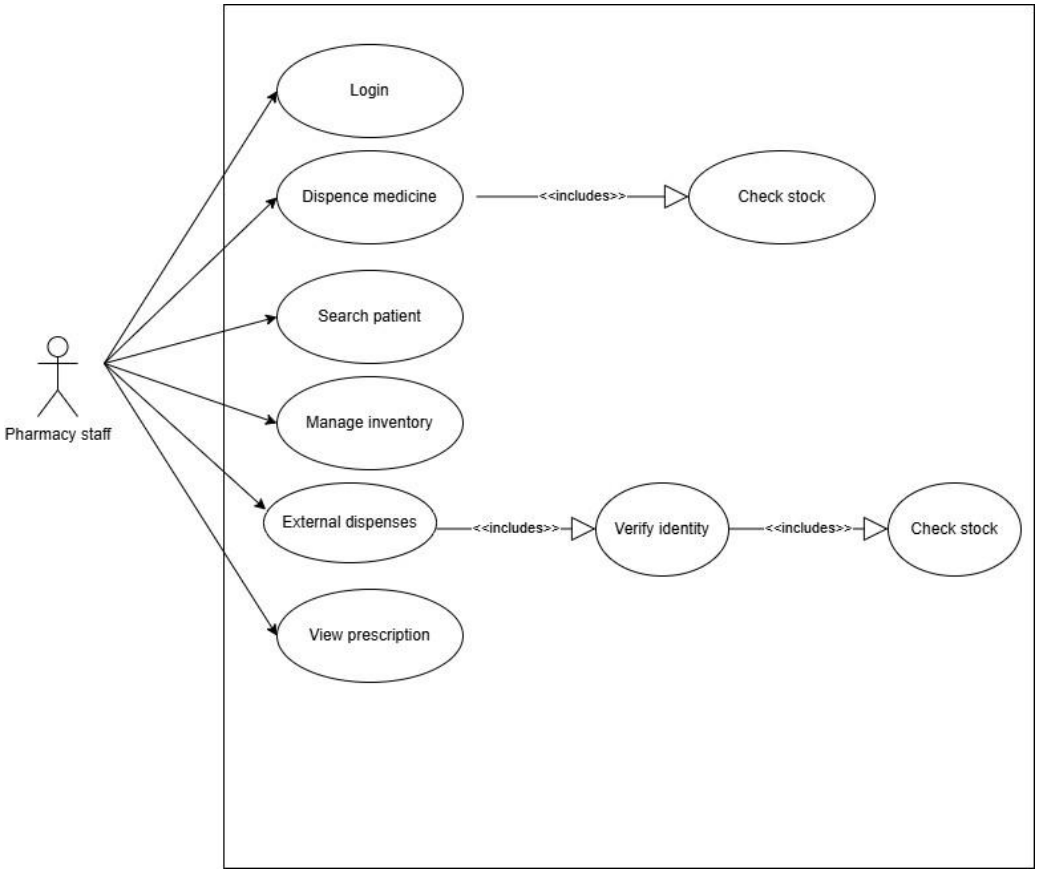
ASM-002	Existing patient records and medicine inventory are assumed to be manually entered into the system during the initial setup phase.
ASM-003	Reliable internet connection is assumed to be available for accessing the cloud-hosted system.
ASM-004	Regular system backups are assumed to be performed to safeguard patient and pharmacy data.
ASM-005	Adequate computer devices are assumed to be provided for all staff required to operate the system.

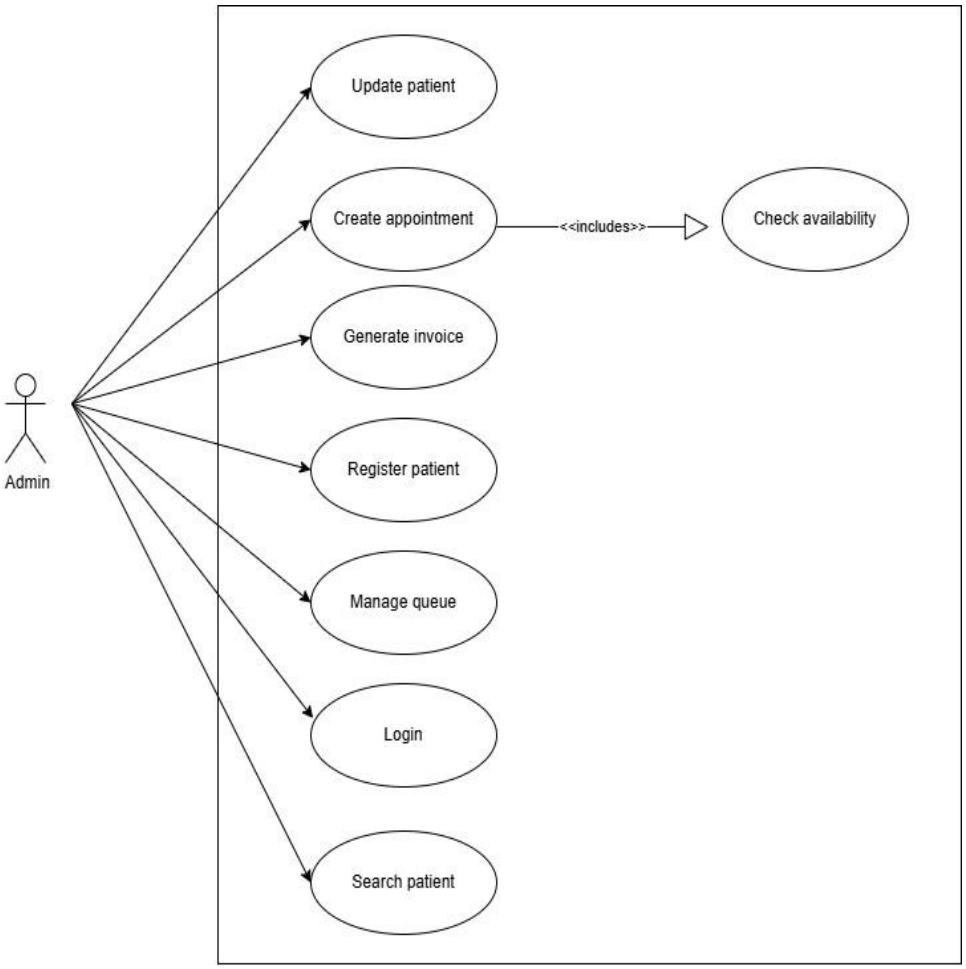
1.5 Use Case Diagrams

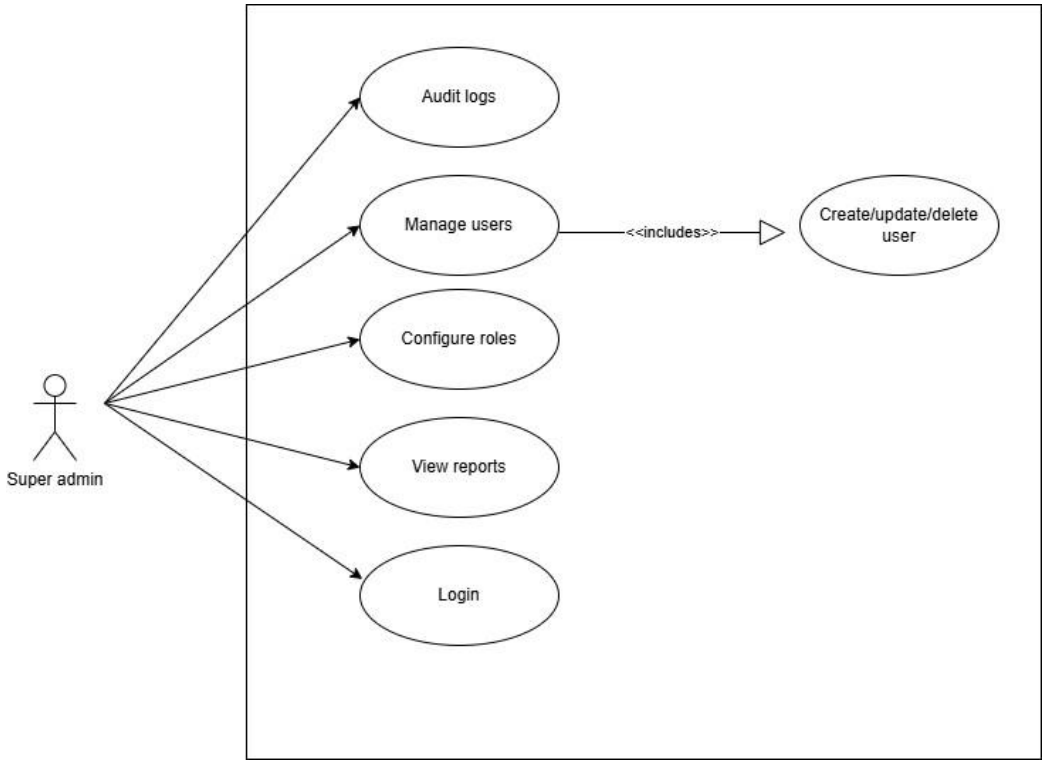
Figure 24: use case diagrams




Login
Manage queue
Search patient
Medical history
Clinical notes
Create prescription
Doctor


Login
Dispence medicine
<<includes>>
Check stock
Search patient
Manage inventory
Pharmacy staff
External dispenses
<<includes>>
Verify identity
<<includes>>
Check stock
View prescription


Update patient
Create appointment
<<includes>>
Check availability
Generate invoice
Admin
Register patient
Manage queue
Login
Search patient


Audit logs
Manage users
<<includes>>
Create/update/delete user
Configure roles
View reports
Login
Super admin

1.6 Use Case Descriptions

The following case descriptions define the interaction sequences between system actors and the OneCare Clinic Management System. Each use case is derived from the validated scenario specifications agreed upon during requirements elicitation.

Table 5 : Use Case Description

Use Case ID	1
Use Case Name	User Login
Description	Handles secure authentication of users (Doctor, Admin, Pharmacy Staff, Super Admin). It validates credentials, manages sessions, logs activities, and ensures account security
Actor	Doctor, Pharmacy Staff, Admin, Super Admin
Pre-Conditions	A user account exists in the system. The user possesses valid credentials.
Post Conditions	The authenticated user accesses the role-specific dashboard. A login event is recorded in the audit log with a timestamp and client IP address. A session with a 30-minute inactivity timeout is active.
Main Flow	1. The user navigates to the login page. 2. The system prompts the user to enter their username/email and password. 3. The system validates the credentials. 4. Upon successful validation, the user is logged in and redirected to their dashboard.
Alternative Flows	Invalid credentials: The system displays: 'Invalid username or password.' After five consecutive failed attempts, the account is locked for 15 minutes. The system logs the lockout event in the audit log. The system displays a lockout message with unlock instructions.

Use Case ID	2
Use Case Name	Manage user accounts
Description	Allows the Super Admin to create, update, deactivate, and reset user accounts. Ensure proper user lifecycle management and maintain audit logs.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated and on the User Management module
Post Conditions	The user account is created, updated, or deactivated as requested. The relevant audit log entry is recorded.
Main Flow	1. The Super Admin navigates to the User Management section. 2. The system displays a list of existing user accounts. 3. The Super Admin selects Create New User. 4. The system displays the user creation form. 5. The Super Admin enters the user's name, email, role, and initial password. 6. The Super Admin submits the form. 7. The system validates the input and creates the account. 8. The system sends a welcome email prompting the user to change the password on first login. 9. The system logs the account creation event in the audit log.
Alternative Flows	Update User: The Super Admin selects an existing user and clicks Edit. The Super Admin modifies allowable fields (name, role, status). The Super Admin saves the changes. The system updates the record and logs the change. Deactivate User: The Super Admin selects the user and clicks Deactivate. The system prompts confirmation. Upon confirmation, the account is deactivated and the user is prevented from logging in. The system logs the deactivation event.

	Reset Password: The Super Admin selects the user and clicks Reset Password. The system generates a temporary password or sends a reset link. The user is required to change the password on next login. The system logs the password reset event

Use Case ID	3
Use Case Name	Configure Role-Based Permissions
Description	Enables the Super Admin to assign and control access permissions for different roles, ensuring secure and role-based system access.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated. Roles exist in the system (Super Admin, Admin, Doctor, Pharmacy Staff).
Post Conditions	Updated permissions are enforced across all system modules for the selected role. The permission change is recorded in the audit log.
Main Flow	1. The Super Admin navigates to the Role & Permissions configuration section. 2. The system displays the role-permission matrix. 3. The Super Admin selects a role to configure. 4. The system displays the current permissions assigned to the role. 5. The Super Admin enables or disables specific module permissions. 6. The Super Admin saves the configuration. 7. The system applies the updated permissions and logs the change in the audit log
Alternative Flows	The Super Admin reviews permissions without modifying. The Super Admin cancels, and no changes have persisted

Use Case ID	4
Use Case Name	Configure Doctor Schedule
Description	Allow the Super Admin to define doctors' working days and time slots, making them available for appointment booking.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated. Doctor user accounts exist in the system.
Post Conditions	The doctor's schedule is saved and available for appointment creation. The configuration change is recorded in the audit log
Main Flow	1. The Super Admin navigates to the Clinic Configuration section. 2. The Super Admin selects the doctor whose schedule is to be configured. 3. The system displays the current schedule of the selected doctor. 4. The Super Admin sets working days and time slots. 5. The Super Admin saves the schedule. 6. The system validates the configuration and persists in the schedule. 7. The system makes the doctor's schedule available for appointment booking
Alternative Flows	The system detects a conflict in the defined time slots. The system displays a validation error indicating conflict. The Super Admin corrects the time slot and resubmits

Use Case ID	5
Use Case Name	Backup and Restore System Data

Description	Ensures data safety by allowing system backups and restoration in case of failures or data loss.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated.
	The system is operational
Post Conditions	A backup file is created and stored, or data is restored from a selected backup. The backup or restore event is logged in the audit log.
Main Flow	1. The Super Admin navigates to the System Administration section. 2. The Super Admin selects Backup Data. 3. The system initiates a backup of all system data. 4. The system confirms successful backup completion and records the timestamp. 5. The Super Admin can download or store the backup file securely
Alternative Flows	The Super Admin selects Restore Data. The Super Admin uploads or selects a backup file. The system prompts for confirmation and warning of data overwritten. Upon confirmation, the system restores data from the backup. The system confirms successful restoration

Use Case ID	6
Use Case Name	View System-Wide Reports and Audit Logs
Description	Provides access to system reports and logs for monitoring, auditing, and decision-making.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated. System activity data and audit logs exist.

Post Conditions	The requested report or audit log is displayed. Exported files are available for download.
Main Flow	1. The Super Admin navigates to the Reports & Audit Logs section. 2. The system displays report categories and audit log filters.
	3. The Super Admin selects the report type or applies filters (date range, user, action type). 4. The system generates and displays the requested report or filtered audit log. 5. The Super Admin can export the report or log as a file
Alternative Flows	The system returns no results for the applied filters. The system displays: 'No records found for the selected criteria.' The Super Admin adjusts the filters and retries

Use Case ID	7
Use Case Name	Registering New Patient
Description	Enables Admin to create new patient records with unique clinic IDs.
Actor	Admin
Pre-Conditions	The admin is authenticated. The patient is not already registered in the system.
Post Conditions	A new patient record is created with a unique clinic number. The patient is searchable in the system

Main Flow	1. The admin navigates to the Patient Management module. 2. The admin selects Register New Patient. 3. The system displays the patient registration form. 4. The admin enters the patient's personal and contact details. 5. The admin submits the form. 6. The system validates the input fields. 7. The system assigns a unique clinic number (patient ID) to the patient. 8. The system creates the patient record and saves it. 9. The system displays confirmation with the assigned clinic number
Alternative Flows	The system detects a potential duplicate based on name and phone number. The system warns the admin of the potential duplicate. The admin reviews the existing record and either proceeds with creation or cancels.

Use Case ID	8
Use Case Name	Update Patient Information
Description	Allows Admin to update patient details such as contact information and address.
Actor	Admin
Pre-Conditions	The admin is authenticated. The patient record exists in the system
Post Conditions	The patient's non-medical information is updated. The update is recorded in the audit log.

Main Flow	1. The admin searches for the patient by name, phone number, or clinic number. 2. The system displays matching patient record. 3. The admin selects the patient and clicks Edit. 4. The system displays editable non-medical fields (contact details, address, etc.). 5. The admin modifies the required fields. 6. The admin saves the changes. 7. The system validates and persists the updated record. 8. The system logs the update in the audit log
Alternative Flows	No matching patient record is found. The system displays: 'No patient found matching the search criteria.' The admin refines the search or registers the patient

Use Case ID	9
Use Case Name	Search for Patient Records
Description	Allows users (Admin, Doctor, Pharmacy Staff) to quickly locate patient records using search criteria.
Actor	Admin, Doctor, Pharmacy Staff
Pre-Conditions	The user is authenticated. Patient records exist in the system.
Post Conditions	The patient record is located and displayed to the user

Main Flow	1. The user navigates to the Patient Search section. 2. The user enters a search term (name, phone number, or clinic number). 3. The user submits the search. 4. The system queries the database and returns matching results. 5. The user selects the desired patient record from the results. 6. The system displays the patient's details appropriate to the user's role
Alternative Flows	The system returns no results. The system displays: 'No matching patient records found.' The user revises the search term or registers the patient

Use Case ID	10
Use Case Name	Create Appointment
Description	Allows Admin to schedule appointments by selecting patient, doctor, and available time slots.
Actor	Admin
Pre-Conditions	The admin is authenticated. The patient record exists. The doctor's schedule is configured
Post Conditions	The appointment is created with status 'Scheduled'. The appointment appears in the daily queue.

Main Flow	1. The admin navigates to the Appointment Management module. 2. The admin selects Create New Appointment. 3. The system displays the appointment creation form. 4. The admin searches for and selects the patient. 5. The admin selects the doctor, date, and available time slot. 6. The admin submits the form. 7. The system validates availability and absence of conflicts. 8. The system creates the appointment with status 'Scheduled'. 9. The system adds the appointment to the doctor's schedule. 10. The system sends an appointment confirmation notification to the patient
Alternative Flows	The selected time slot is already booked. The system displays available alternative slots. The admin selects an available slot and resubmits.

Use Case ID	11
Use Case Name	Reschedule Appointment
Description	Enables Admin to modify existing appointment dates and times.
Actor	Admin
Pre-Conditions	The admin is authenticated. An existing appointment with status 'Scheduled' exists
Post Conditions	The appointment is updated with the new date and time. The patient receives a reschedule notification

Main Flow	1. The admin searches for and selects the existing appointment. 2. The admin clicks Reschedule.
	3. The system displays available time slots for the same or different date. 4. The admin selects a new date and time slot. 5. The admin confirms the reschedule. 6. The system updates the appointment with the new date and time. 7. The system notifies the patient of the updated appointment details
Alternative Flows	No available slots exist for the requested date. The system prompts the admin to select a different date

Use Case ID	12
Use Case Name	Cancel Appointment
Description	Allow Admin to cancel scheduled appointments and free up time slots.
Actor	Admin
Pre-Conditions	The admin is authenticated. An existing appointment with status 'Scheduled' exists.
Post Conditions	The appointment status is set to 'Cancelled'. The time slot is available for new bookings

Main Flow	1. The admin searches for and selects the appointment. 2. The admin clicks Cancel Appointment. 3. The system prompts cancellation confirmation. 4. The admin confirms the cancellation. 5. The system updates the appointment status to 'Cancelled'. 6. The system releases the time slot for other bookings. 7. The system notifies the patient of the cancellation
Alternative Flows	The appointment status is 'Completed' or 'No-show'. The system prevents cancellation and displays an appropriate message

Use Case ID	13
Use Case Name	View and Manage Daily Queue
Description	Manages daily patient flow by organizing appointments into a queue and tracking statuses (Arrived, Completed, No-show)..
Actor	Admin, Doctor
Pre-Conditions	The user is authenticated. Appointments are scheduled for the current day.
Post Conditions	Queue statuses are updated in real time. Completed consultations are reflected in the patient's history

Main Flow	1. The system automatically generates the daily patient queue based on scheduled appointments at the start of each day. 2. The Admin or Doctor navigates to the Queue Management module. 3. The system displays the ordered list of patients with their queue number and appointment details. 4. The admin marks a patient as 'Arrived' when the patient checks in. 5. The Doctor selects the next patient in the queue to begin consultation. 6. Upon completing the consultation, the Doctor marks the consultation as 'Completed'. 7. The system updates the queue status and advances to the next patient. 8. The system records the consultation completion timestamp
Alternative Flows	The patient does not arrive within the expected timeframe. The admin marks the appointment status as 'No-show'. The system moves to the next patient in the queue

Use Case ID	14
Use Case Name	View Scheduled Appointments
Description	Allow doctor to view and filter their appointments.
Actor	Doctor
Pre-Conditions	The Doctor is authenticated. Appointments are scheduled for the current or future dates
Post Conditions	The Doctor views the scheduled appointments and can proceed to consultation

Main Flow	1. The Doctor navigates to the Appointments section of the dashboard. 2. The system displays all appointments scheduled for the current day by default. 3. The Doctor can filter appointments by date range. 4. The system displays patient name, appointment time, and queue status. 5. The Doctor selects an appointment to view detailed patient information
Alternative Flows	No appointments exist for the selected date. The system displays: 'No appointments scheduled for the selected date.'

Use Case ID	15
Use Case Name	Access Patient Medical History
Description	Enables doctors to review past consultations, notes, and prescriptions.
Actor	Doctor
Pre-Conditions	The Doctor is authenticated. The patient record exists with prior consultation history
Post Conditions	The Doctor has reviewed the patient's medical history
Main Flow	1. The Doctor selects a patient from the appointment queue or via search. 2. The system displays the patient's profile. 3. The Doctor selects the Medical History tab.
	4. The system retrieves and displays all past consultations, clinical notes, and prescriptions. 5. The Doctor reviews the history to inform the current consultation

Alternative Flows	The patient has no prior consultations. The system displays: 'No medical history on record for this patient’

Use Case ID	16
Use Case Name	Record Clinical Notes
Description	Allow doctors to document diagnosis, symptoms, and treatment plans.
Actor	Doctor
Pre-Conditions	The Doctor is authenticated. A consultation session is in progress for the selected patient.
Post Conditions	Clinical notes are saved and associated with the patient's consultation record. The notes are accessible in future medical history reviews.
Main Flow	1. The Doctor opens the active consultation for the patient. 2. The system displays the clinical notes entry form. 3. The Doctor enters symptoms, examination findings, diagnosis, and treatment plan. 4. The Doctor saves the clinical notes. 5. The system stores the notes linked to the current consultation record. 6. The system timestamps the record with the date and doctor ID
Alternative Flows	The Doctor decides not to save the notes. The Doctor clicks Discard. The system prompts confirmation. Upon confirmation, the unsaved notes are discarded

Use Case ID	17
Use Case Name	Create Digital Prescription
Description	Enables doctors to generate prescriptions linked to consultations and send them to the pharmacy.
Actor	Doctor
Pre-Conditions	The Doctor is authenticated. A consultation session is in progress. Medicine records exist in the inventory.
Post Conditions	The prescription is saved with status 'Pending Dispense'. The Pharmacy Staff can view the prescription for dispensing.
Main Flow	1. During an active consultation, the Doctor selects Create Prescription. 2. The system displays the prescription form. 3. The Doctor searches for and adds medicines, specifying dosage, duration, and instructions for each. 4. The Doctor reviews the complete prescription. 5. The Doctor submits the prescription. 6. The system saves the prescription linked to the consultation. 7. The system automatically sets the prescription status to 'Pending Dispense'. 8. The prescription becomes visible in the Pharmacy module.
Alternative Flows	The searched medicine does not exist in the inventory. The system displays: 'Medicine not found in inventory.' The Doctor adds a free-text note for the Pharmacy Staff

Use Case ID	18
Use Case Name	View Pending Prescriptions

Description	Allow pharmacy staff to view prescriptions awaiting dispensing.
Actor	Pharmacy Staff
Pre-Conditions	The Pharmacy Staff is authenticated. Prescriptions with status 'Pending Dispense' exist.
Post Conditions	The Pharmacy Staff has access to all pending prescription details.
Main Flow	1. The Pharmacy Staff navigate to the Pharmacy module. 2. The system displays all prescriptions with status 'Pending Dispense'. 3. The Pharmacy Staff selects a prescription to view details. 4. The system displays patient information, prescribed medicines, dosage, duration, and instructions
Alternative Flows	No pending prescriptions exist. The system displays: 'No prescriptions pending dispensing.'

Use Case ID	19
Use Case Name	Dispense Medicines
Description	Handles medicine dispensing, stock deduction, and updating prescription status.
Actor	Pharmacy Staff
Pre-Conditions	The Pharmacy Staff is authenticated. A prescription with status 'Pending Dispense' is selected. Sufficient stock exists in inventory.
Post Conditions	The medicines are dispensed and inventory is updated. The prescription status is updated to 'Fully Dispensed' or 'Partially Dispensed'. The dispensing event is recorded in the audit log

Main Flow	1. The Pharmacy Staff selects a pending prescription. 2. The system displays prescription details and current stock levels for each medicine. 3. The Pharmacy Staff validates the prescription details.
	4. The Pharmacy Staff confirms dispensing of all medicines. 5. The system deducts dispensed quantities from the inventory. 6. The system updates the prescription status to 'Fully Dispensed'. 7. The system records the dispensing action with staff ID and timestamp in the audit log. 8. The system prevents re-dispensing of the completed prescription
Alternative Flows	One or more medicines have insufficient stock. The system highlights the items with low or zero stock. The Pharmacy Staff dispenses available items and marks the prescription as 'Partially Dispensed'. The remaining items are flagged for follow-up.

Use Case ID	20
Use Case Name	Manage Medicine Records
Description	Allow pharmacy staff to add, update, or deactivate medicine records.
Actor	Pharmacy Staff
Pre-Conditions	The Pharmacy Staff is authenticated
Post Conditions	The medicine record is created, updated, or deactivated as requested. The change is recorded in the audit log.

Main Flow	1. The Pharmacy Staff navigate to the Inventory Management module. 2. The Pharmacy Staff selects Add New Medicine. 3. The system displays the medicine creation form. 4. The Pharmacy Staff enters medicine name, category, unit, stock quantity, low-stock threshold, and expiry date. 5. The Pharmacy Staff submits the form.
	6. The system validates the data and creates the medicine record. 7. The medicine is available for prescribing and dispensing.
Alternative Flows	The Pharmacy Staff selects an existing medicine record and clicks Edit. The Pharmacy Staff modifies the required fields. The Pharmacy Staff saves the changes.

Use Case ID	21
Use Case Name	Track Stock Levels and Expiry
Description	Monitors stock levels, expiry dates, and generate alerts for low stock or expired medicines.
Actor	Pharmacy Staff
Pre-Conditions	The Pharmacy Staff is authenticated. Medicine records exist in the system.
Post Conditions	Inventory reflects accurate stock levels. Expired medicines are marked unavailable. Low stock alerts are generated as appropriate.

Main Flow	1. The Pharmacy Staff navigate to the Inventory module. 2. The system displays a list of all medicines with current stock levels and expiry dates. 3. The system highlights medicines that are below the low-stock threshold. 4. The system flags medicines that are expired or near expiry. 5. The Pharmacy Staff reviews the flagged items. 6. The Pharmacy Staff updates stock quantities as new supplies arrive. 7. The system marks expired medicines as unavailable
Alternative Flows	Stock level for medicine falls below the configured threshold. The system generates low stock alert notification. The Pharmacy Staff is notified to reorder stock.

Use Case ID	22
Use Case Name	Generate Invoice
Description	Enables Admin to create invoices based on consultation and medicine charges.
Actor	Admin
Pre-Conditions	The admin is authenticated. A consultation has been completed and medicines dispensed.
Post Conditions	The invoice is generated with status 'Pending'. The invoice is retrievable for future reference.

Main Flow	1. The admin navigates to the Billing module. 2. The admin searches for the patient or selects from a completed consultation. 3. The system auto-populates consultation charges and medicine costs. 4. The admin reviews the invoice details. 5. The admin confirms and generates the invoice. 6. The system creates the invoice with status 'Pending'. 7. The system assigns an invoice number and timestamps the record
Alternative Flows	The admin notices an error in the auto-populated charges. The admin edits the applicable line items. The admin confirms the corrected invoice.

Use Case ID	23
Use Case Name	Record Payment
Description	Allow Admin to record payments and update invoice status.
Actor	Admin
Pre-Conditions	The admin is authenticated. A 'Pending' invoice exists for the patient.
Post Conditions	The payment is recorded and the invoice status is updated to 'Paid'. The payment is logged in the audit log.
Main Flow	1. The admin opens the pending invoice.

	2. The system displays the invoice total and line items. 3. The admin confirms that cash payment has been received. 4. The admin records the payment. 5. The system updates the invoice status to 'Paid'. 6. The system records the payment with the date, amount, and Admin ID. 7. The system logs the payment event in the audit log
Alternative Flows	The patient is unable to pay the full amount. The admin records the amount paid. The system keeps the invoice status as 'Pending' with a partial amount noted. The remaining balance is tracked for future collection. The admin searches for a prior invoice by patient name, invoice number, or date. The system retrieves and displays the historical invoice. The admin views or reprints the invoice as required.

Use Case ID	24
Use Case Name	Generate Clinic Reports
Description	Provides reports on clinic operations such as patient visits and consultations.
Actor	Super Admin, Admin
Pre-Conditions	The user is authenticated with the appropriate role. Historical appointment and consultation data exists.
Post Conditions	The clinic report is generated and displayed. The report can be exported for offline use.

Main Flow	1. The user navigates to the Reports module. 2. The user selects the Clinic Reports category. 3. The user specifies the report parameters (date range, doctor, report type). 4. The system compiles and generates the report.
	5. The system displays the report including patient counts, consultation counts, and queue statistics. 6. The user can export the report in a supported format.
Alternative Flows	Report generation exceeds the acceptable time threshold. The system displays a timeout notification. The user is prompted to reduce the date range and regenerate

Use Case ID	25
Use Case Name	Generate Pharmacy Reports
Description	Generate reports related to medicine usage and inventory.
Actor	Super Admin, Pharmacy staff
Pre-Conditions	The user is authenticated. Dispensing and inventory data exists.
Post Conditions	The pharmacy report is generated and available for review or export.
Main Flow	1. The user navigates to the Pharmacy Reports section. 2. The user selects a type of report (stock usage, expiry report, dispensing history). 3. The user specifies the date range or other filter parameters. 4. The system compiles and displays the pharmacy report. 5. The user can export the report in a supported format.

Table 7-26: Use case of View Audit Logs

Use Case ID	26
Use Case Name	View Audit Logs
Description	Allows Super Admin to monitor system activities for security and compliance.
Actor	Super Admin
Pre-Conditions	The Super Admin is authenticated. Audit log data exists.
Post Conditions	The Super Admin has reviewed the required audit log entries. Exported audit logs are available for compliance or investigation purposes.
Main Flow	1. The Super Admin navigates to the Audit Log section. 2. The system displays recent audit log entries. 3. The Super Admin applies filters (date range, user, action type, module). 4. The system returns filtered audit log entries. 5. The Super Admin reviews individual log entries for details. 6. The Super Admin can export the filtered audit log.
Alternative Flows	No log entries match the applied filters. The system displays: 'No audit log entries found for the selected criteria.' The Super Admin adjusts the filters.

Use Case ID	27
Use Case Name	Book Appointment Without Login
Description	Allows patients to book appointments without creating an account
Actor	Patient

Pre-Conditions	The clinic self-service portal is accessible. Doctor schedules are configured and available.
Post Conditions	The appointment is created with status 'Scheduled'. The patient receives confirmation.
Main Flow	1. The patient navigates to the clinic's self-service booking page. 2. The system displays the appointment booking form. 3. The patient enters personal details (name, phone number, and date of birth). 4. The patient selects the preferred doctor, date, and available time slot.
	5. The patient submits the booking request. 6. The system validates the details and checks slot availability. 7. The system creates the appointment with status 'Scheduled'. 8. The system sends a booking confirmation notification to the patient
Alternative Flows	The selected slot is taken at the time of submission. The system displays available alternative slots. The patient selects an alternative slot and resubmits.

Use Case ID	28
Use Case Name	Interact with Clinic Chatbot
Description	Provides automated responses to patient queries using a knowledge base.
Actor	Patient

Pre-Conditions	The chatbot interface is accessible on the clinic website or portal. Approved knowledge source documents are uploaded and indexed
Post Conditions	The patient receives accurate, approved information. No medical diagnosis or prescription advice is provided by the chatbot.
Main Flow	1. The patient opens the chatbot interface. 2. The chatbot greets the patient and presents query options. 3. The patient types a query (e.g., clinic hours, doctor's availability, health information). 4. The system processes the query against the predefined approved knowledge base. 5. The chatbot displays a relevant response to the patient. 6. The patient may ask follow-up questions within the allowed scope.
Alternative Flows	The patient asks for a diagnosis, prescription recommendation, or information not in the knowledge base. The chatbot responds: 'I'm sorry, I am unable to provide medical diagnoses or prescriptions. Please consult a doctor.' The chatbot offers help with other approved topics.

Use Case ID	29
Use Case Name	Manage Chatbot Knowledge Base
Description	Allows authorized users to update chatbot information sources.
Actor	Doctor
Pre-Conditions	The authorized user is authenticated. The Knowledge Base Management section is accessible.

Post Conditions	The chatbot knowledge base is updated with approved information. The upload or removal event is logged in the audit log.
Main Flow	1. The Doctor or Super Admin navigates to the Chatbot Knowledge Base section. 2. The system displays a list of currently uploaded knowledge source documents. 3. The user selects Upload New Document. 4. The user selects an approved health-related PDF document. 5. The system uploads and indexes the document for chatbot use. 6. The system confirms successful upload and indexing. 7. The chatbot knowledge base is updated to include the new document.
Alternative Flows	The user selects an existing document and clicks Remove. The system prompts confirmation. Upon confirmation, the document is removed from the knowledge base. The chatbot no longer references the removed document.

Use Case ID	30
Use Case Name	Mark Prescription for External Dispensing
Description	Allow pharmacy staff to mark prescriptions for off-site delivery.
Actor	Pharmacy Staff
Pre-Conditions	The Pharmacy Staff is authenticated. A prescription with status 'Pending Dispense' exists.
Post Conditions	The prescription is flagged for external dispensing. The Pharmacy Staff proceeds with identity verification.

Main Flow	1. The Pharmacy Staff opens a pending prescription. 2. The Pharmacy Staff selects the option Mark for External Dispensing. 3. The system displays the external dispensing form. 4. The Pharmacy Staff confirms the patient's off-site status. 5. The system flags the prescription for external dispensing processing. 6. The system proceeds with patient identity verification.
Alternative Flows	Prescription Not Eligible: The prescription has already been dispensed or cancelled. The system prevents action and displays an appropriate status message

Use Case ID	31
Use Case Name	Verify Patient Identity for External Dispensing
Description	Ensures correct patient identity before external medicine delivery.
Actor	Pharmacy Staff
Pre-Conditions	The prescription is flagged for external dispensing. The Pharmacy Staff is authenticated.
Post Conditions	Patient identity is verified. The system allows the dispensing step to proceed.
Main Flow	1. The system displays the patient identity verification form.
	2. The Pharmacy Staff selects the verification method: phone number, clinic ID, or QR code scan. 3. The Pharmacy Staff enters or scans the identity information. 4. The system validates the provided identity against the patient record. 5. Upon successful verification, the system proceeds to the dispensing step.

Alternative Flows	Identity Verification Failure: The provided identity does not match the patient record. The system displays: 'Identity verification failed. Please verify and try again.' The Pharmacy Staff may attempt an alternative verification method. After repeated failures, the dispensing process is cancelled and logged.

Use Case ID	32
Use Case Name	Complete External Dispensing
Description	Finalizes external dispensing, updates inventory, and records delivery details.
Actor	Pharmacy staff
Pre-Conditions	The Pharmacy Staff is authenticated. Patient identity verification is completed. Sufficient inventory stock exists.
Post Conditions	Inventory is updated immediately. Prescription status is set to 'Dispensed (External)'. A delivery record is generated and stored. The event is recorded in the audit log for compliance.
Main Flow	1. The system displays prescription details for external dispensing. 2. The Pharmacy Staff confirms the medicines to be dispensed. 3. The Pharmacy Staff selects the payment method.

	4. The Pharmacy Staff confirms the dispensing action. 5. The system deducts dispensed quantities from the inventory immediately. 6. The system updates the prescription status to 'Dispensed (External)'. 7. The system generates a delivery record including patient ID, staff ID, date/time, and payment method. 8. The system logs the external dispensing event in the audit log
Alternative Flows	Insufficient Stock: One or more medicines are out of stock. The system highlights the unavailable items. The Pharmacy Staff dispenses available items and notes partial fulfilment. The remaining items are flagged for follow-up.

1.7 Activity Diagrams

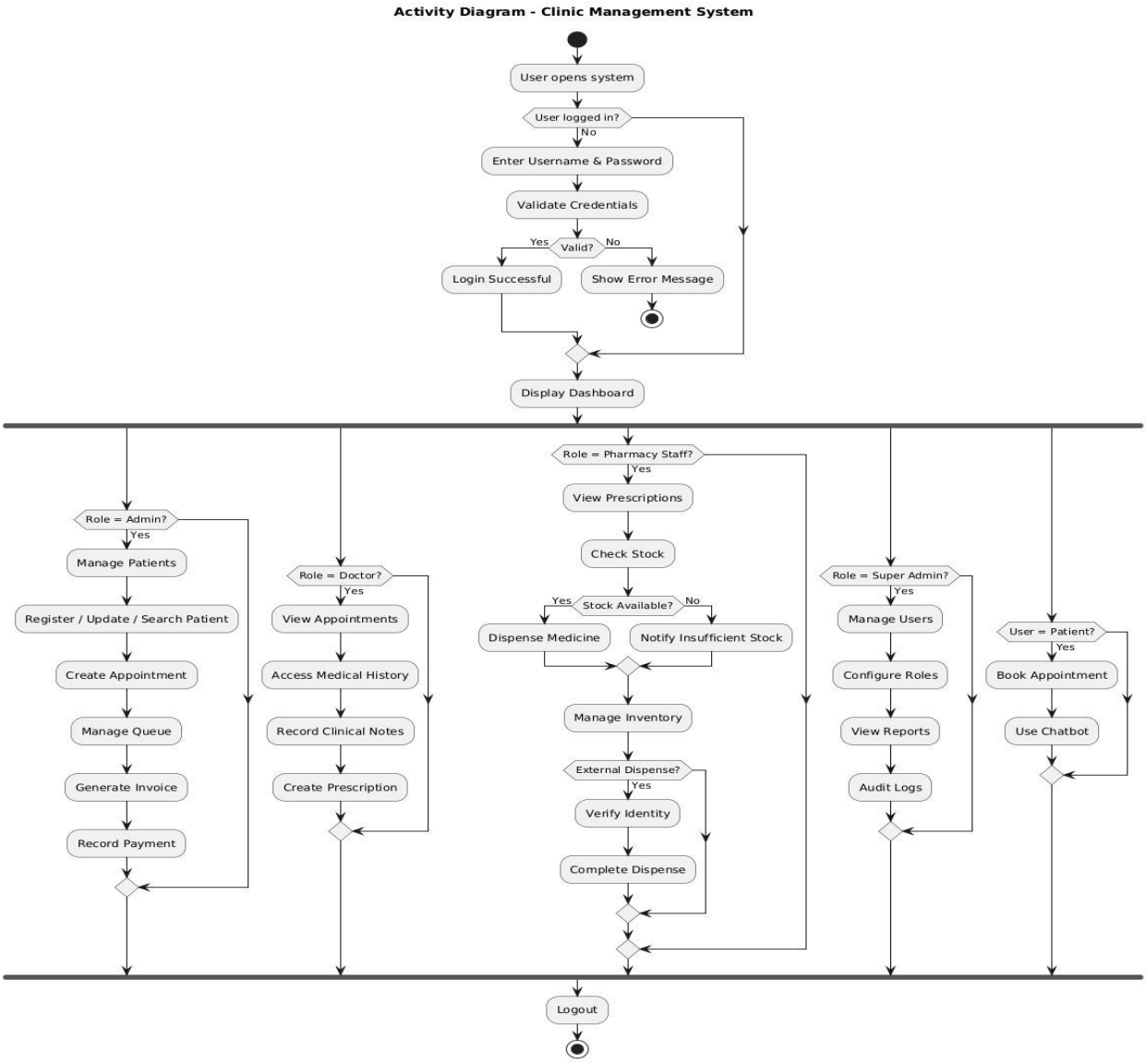
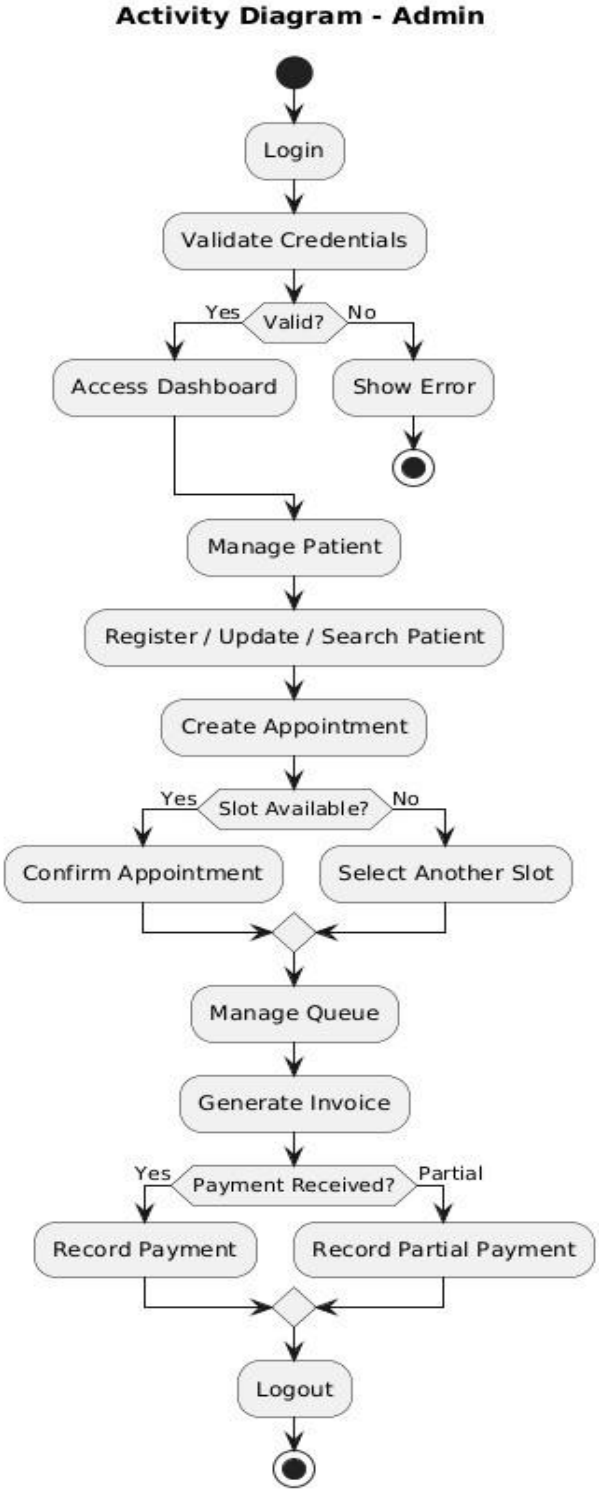
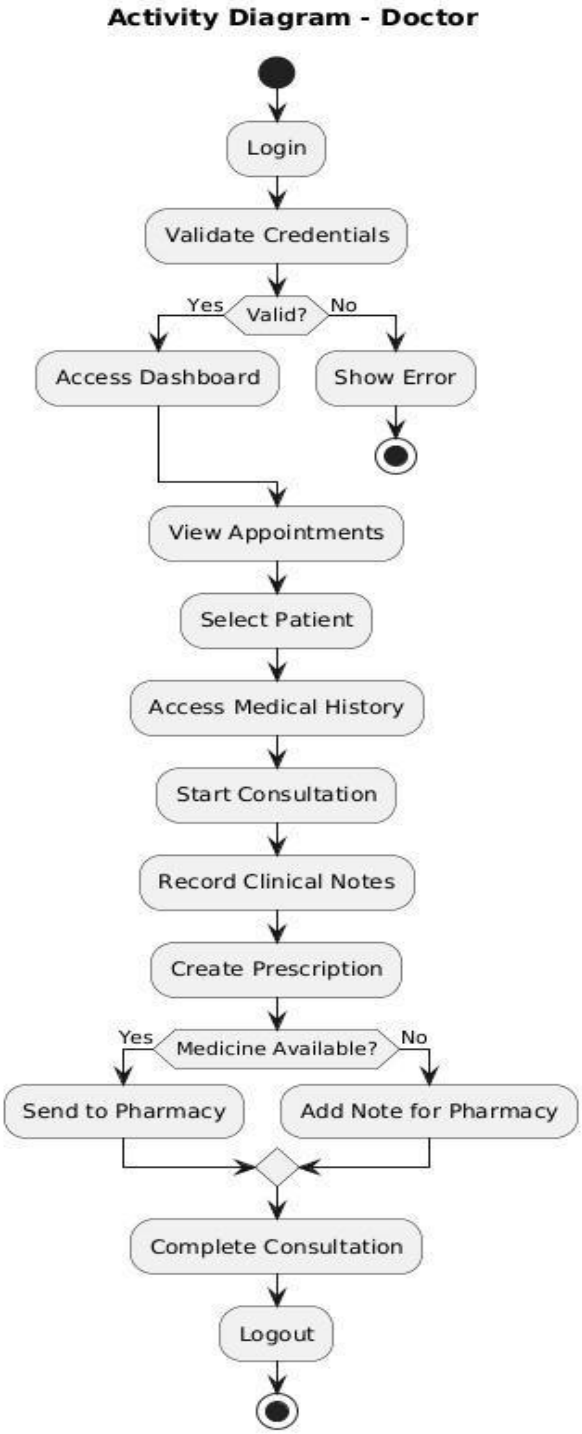
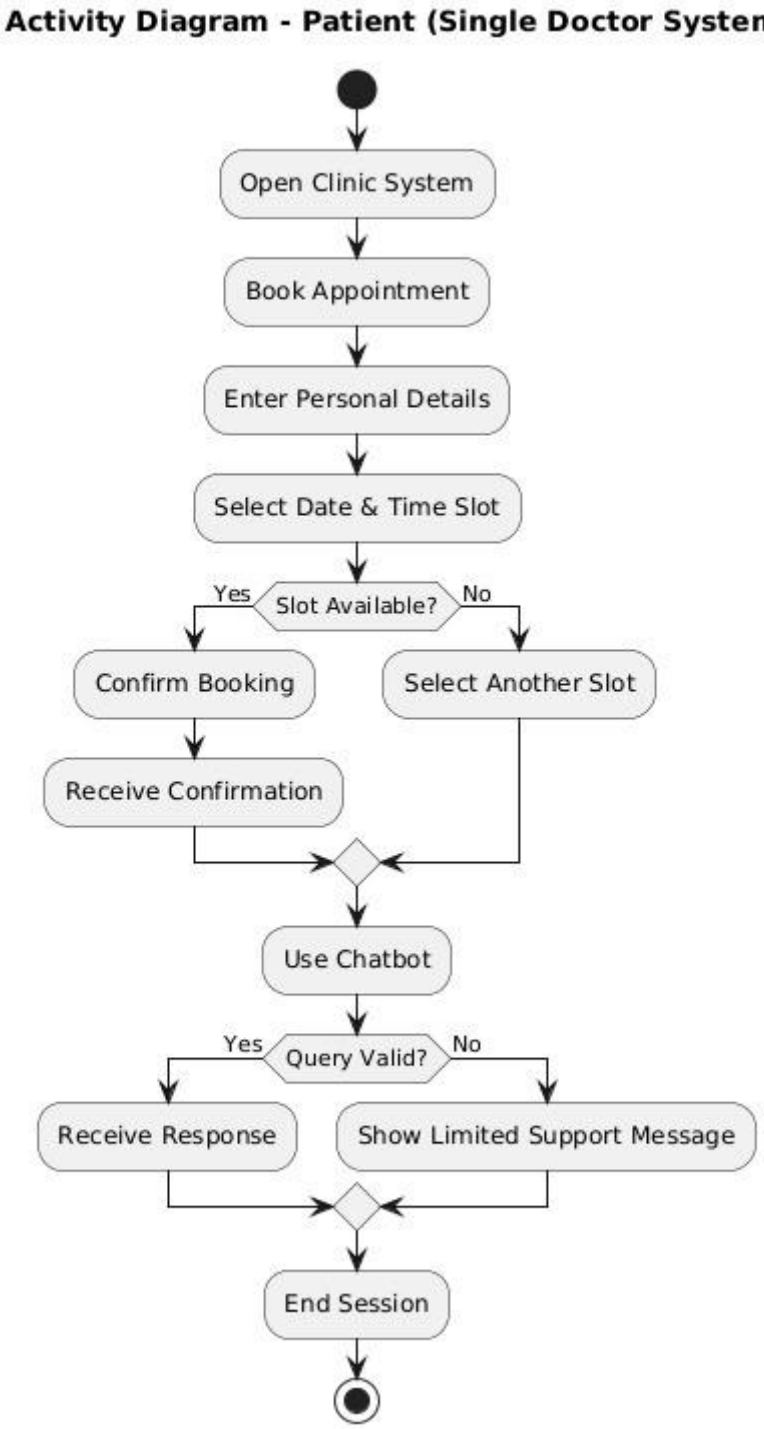
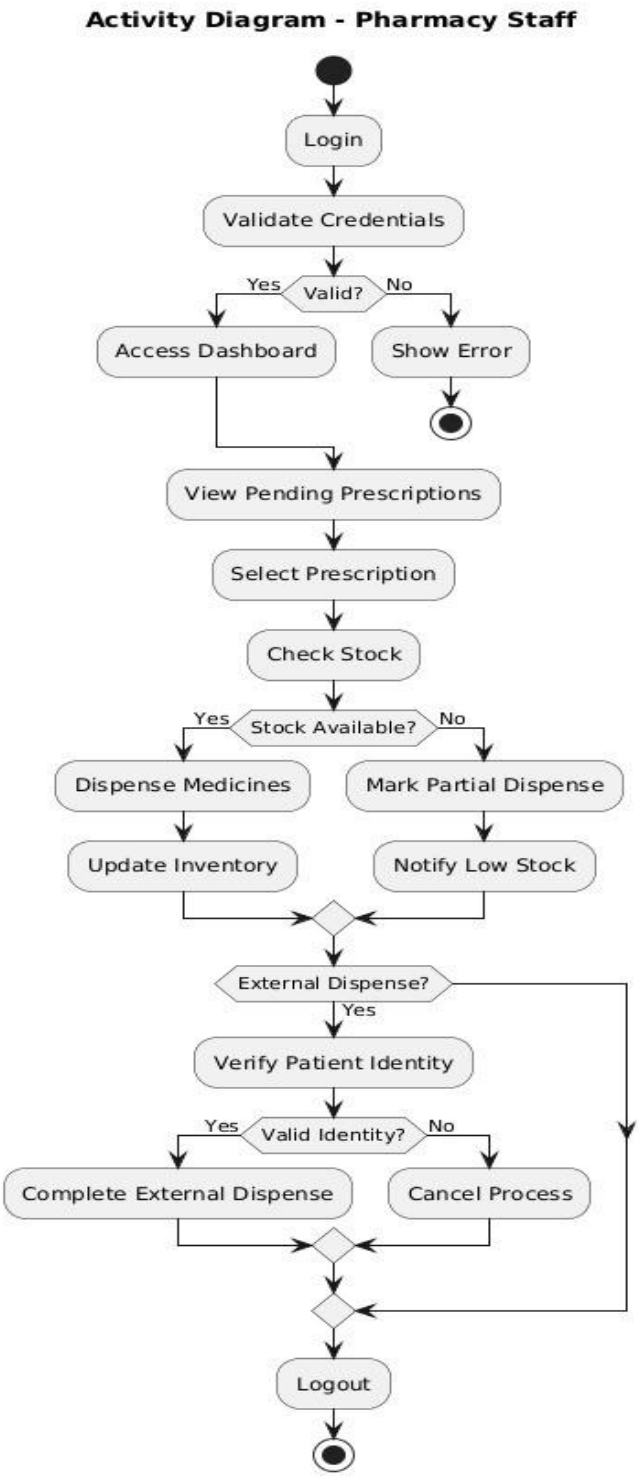


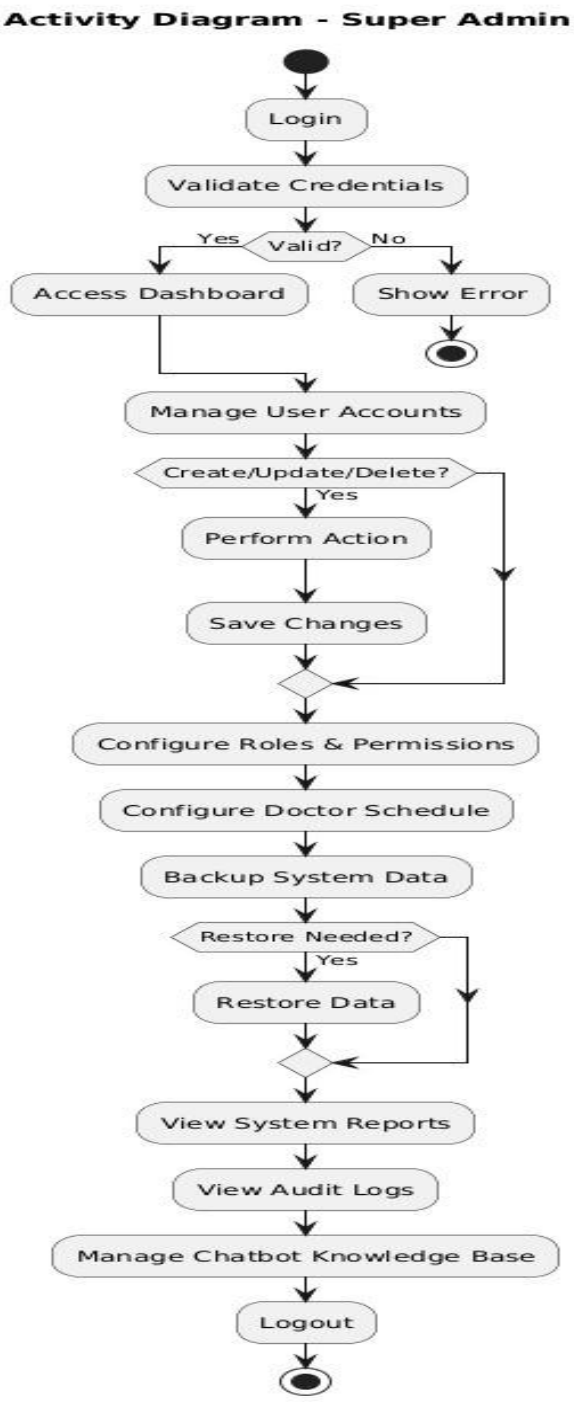
Figure 25 : Activity Diagrams


Activity Diagram - Admin
Login
Validate Credentials
Yes
Valid?
No
Access Dashboard
Show Error
Manage Patient
Register / Update / Search Patient
Create Appointment
Yes
Slot Available?
No
Confirm Appointment
Select Another Slot
Manage Queue
Generate Invoice
Yes
Payment Received?
Partial
Record Payment
Record Partial Payment
Logout


Activity Diagram - Doctor
Login
Validate Credentials
Yes
Valid?
No
Access Dashboard
Show Error
View Appointments
Select Patient
Access Medical History
Start Consultation
Record Clinical Notes
Create Prescription
Yes
Medicine Available?
No
Send to Pharmacy
Add Note for Pharmacy
Complete Consultation
Logout


Activity Diagram - Patient (Single Doctor System)
Open Clinic System
Book Appointment
Enter Personal Details
Select Date & Time Slot
Slot Available?
Yes
No
Confirm Booking
Select Another Slot
Receive Confirmation
Use Chatbot
Query Valid?
Yes
No
Receive Response
Show Limited Support Message
End Session


Activity Diagram - Pharmacy Staff
Login
Validate Credentials
Valid?
Yes
No
Access Dashboard
Show Error
View Pending Prescriptions
Select Prescription
Check Stock
Stock Available?
Yes
No
Dispense Medicines
Mark Partial Dispense
Update Inventory
Notify Low Stock
External Dispense?
Yes
Verify Patient Identity
Valid Identity?
Yes
No
Complete External Dispense
Cancel Process
Logout


Activity Diagram - Super Admin
Login
Validate Credentials
Valid?
Yes
No
Access Dashboard
Show Error
Manage User Accounts
Create/Update/Delete?
Yes
Perform Action
Save Changes
Configure Roles & Permissions
Configure Doctor Schedule
Backup System Data
Restore Needed?
Yes
Restore Data
View System Reports
View Audit Logs
Manage Chatbot Knowledge Base
Logout

1.8 External Interface Requirements

1.8.1 User Interface Requirements

A role-based, responsive web interface shall be provided by the system, designed specifically for clinic and pharmacy workflows.

General UI Requirements

- Separate dashboards shall be provided for:
  - Doctor (patient records, prescriptions)
  - Pharmacist (inventory, dispensing, alerts)
  - Super Admin (user management, reports)
- The interface shall be responsive and accessible on desktops and tablets.
- Consistent layouts and navigation menus shall be used across all modules.

Clinic-Specific UI Features

- A patient registration form shall be provided, including:
  - Personal details
  - Medical history
- An appointment booking interface shall be provided, including:
  - Date and time slot selection
  - Availability display
- The doctor's dashboard shall display:
  - Daily appointments
  - Patient history
  - Prescription creation panel

Pharmacy-Specific UI Features

- An inventory dashboard shall be provided, displaying:
  - Medicine name, quantity, batch number, expiry date
  - Visual indicators shall be provided for:
    - Low stock medicines
    - Near-expiry medicines
- A prescription view screen shall be provided for pharmacists to:
  - Verify doctor prescriptions
  - Dispense medicines

Alert & Notification UI

- Pop-up and dashboard alerts shall be provided for:

    - Stock alerts
    - Expiry alerts
- Appointment reminders shall be provided through a notification panel accessible to all roles.

Accessibility Requirements

- Readable fonts and proper color contrast shall be ensured
- The UI shall be simple for non-technical users.
- Minimal steps shall be required to complete key tasks (e.g., booking, dispensing).

1.8.2 Hardware Interface Requirements

- The system is a web-based application and does not require specialized hardware.
    - The following hardware dependencies shall apply:
    - A server or cloud hosting environment shall be provided with at least:
        - 2 vCPU
        - 4 GB RAM
        - 50 GB storage
    - Clinic staff devices (computers or tablets) shall:
        - Support a modern web browser
        - Have stable internet connectivity
    - A printer (optional) may be used for:
        - Printing prescriptions
        - Printing reports
- No direct integration with specialized medical devices shall be required in this version

1.8.3 Software Interface Requirements

Integration with the following software components shall be provided:

- Database System:
    - relational database management system (e.g., MySQL) shall be used by the system for storing:
        - Patient records
        - Appointment details
        - Prescription data
        - Medicine inventory

- Email Service:

Integration with an email service (SMTP) shall be provided by the system for:

  - Password reset notifications
  - Sending patient bills

- SMS Notification Service:

Appointment confirmation messages shall be sent by the system.

- Chatbot Module:

A chatbot interface shall be included in the system to:

  - Handle patient queries
  - Provide appointment guidance
  - Respond to frequently asked questions

- Web Browser Compatibility:

The system shall be made accessible through modern web browsers such as:

  - Google Chrome
  - Microsoft Edge
  - Mozilla Firefox

- Application Server:

Deployment shall be carried out on a web application server compatible with the backend framework used.

1.8.4 Communication Interface Requirements

- All client-server communications shall be conducted over HTTPS using TLS 1.2 or higher to ensure secure data transmission.
- RESTful APIs shall be used for communication between frontend and backend components.
- Data exchanged between components shall be in JSON format.
- Operation shall occur over standard TCP/IP networks; no proprietary communication protocols shall be required.
- Real-time updates shall be supported for:
  - Appointment status
  - Stock and expiry alerts
- All communication involving sensitive data (e.g., patient records) shall be encrypted and securely handled.

1.9 Data Requirements

This chapter defines the data entities, their attributes, relationships, and storage requirements for the OneCare Clinic Integrated Pharmacy Management System.

1.9.1 Data Entities and Attributes

1.9.1.1 Patient

All personal and medical information for registered patients is stored.

Table 6: Data Entities

Field	Data Type	Constraints	Description
patient_id	VARCHAR (10)	PRIMARY KEY, UNIQUE, NOT NULL	Unique patient identifier
first_name	VARCHAR (100)	NOT NULL	Patient's first name
last_name	VARCHAR (100)	NOT NULL	Patient's last name
date_of_birth	DATE	NOT NULL	Patient date of birth
gender	ENUM('Male','Female','Other')	Male/Female/Other	Patient gender
contact_number	VARCHAR (15)	NOT NULL	Primary contact number
email	VARCHAR (150)	UNIQUE	Email address
address	TEXT	NULLABLE	Residential address
blood_group	VARCHAR (5)	NULLABLE	Patient blood group
registered_date	DATETIME	NOT NULL, DEFAULT NOW ()	Date of registration

1.9.1.2 User (System Users)

Stores credentials and role assignments for all system users including Doctors, Pharmacists, and Admins.

Field	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
username	VARCHAR (50)	UNIQUE, NOT NULL	Login username
password_hash	VARCHAR (255)	NOT NULL	Bcrypt/Argon2 hashed password
role	ENUM ('Doctor','Pharmacist','Admin','Super Admin')	Doctor/Pharmacist/Admin/Super Admin	User role for RBAC
email	VARCHAR (150)	UNIQUE, NOT NULL	Email for notifications/2FA
is_active	BOOLEAN	DEFAULT TRUE	Account active status
failed_attempts	INT	DEFAULT 0	Consecutive failed logins
locked_until	DATETIME	NULLABLE	Account lockout expiry
last_login	DATETIME	NULLABLE	Timestamp of last login

1.9.1.3 Appointment

Manages all appointment booking data between patients and the doctor.

Field	Data Type	Constraints	Description
appointment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique appointment ID
patient_id	VARCHAR (10)	FOREIGN KEY → Patient	Associated patient

appointment_date	DATE	NOT NULL	Date of appointment
appointment_time	TIME	NOT NULL	Scheduled time slot
status	ENUM (`Scheduled`,`Cancelled`,`Completed`,`No-show`)	Pending/Confirmed/Cancelled/Completed	Appointment status
reason	TEXT	NULLABLE	Reason for visit
created_at	DATETIME	DEFAULT NOW ()	Booking timestamp

1.9.1.4 Prescription

Records digital prescriptions issued by the doctor to patients.

Field	Data Type	Constraints	Description
prescription_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique prescription ID
patient_id	VARCHAR (10)	FOREIGN KEY → Patient	Associated patient
doctor_id	INT	FOREIGN KEY → User	Issuing doctor
appointment_id	INT	FOREIGN KEY → Appointment, NULLABLE	Linked appointment
clinical_notes	TEXT	NULLABLE	Doctor's consultation notes
issued_date	DATETIME	NOT NULL, DEFAULT NOW ()	Date prescription was created
status	ENUM ('Issued','Dispensed','Cancelled')	Issued/Dispensed/Cancelled	Prescription fulfilment status

1.9.1.5 Prescription Item

Stores individual medicine line items linked to a prescription.

Field	Data Type	Constraints	Description
item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique item identifier
prescription_id	INT	FOREIGN KEY → Prescription	Parent prescription
medicine_id	INT	FOREIGN KEY → Medicine	Prescribed medicine
dosage	VARCHAR (100)	NOT NULL	Dosage instruction (e.g. 500mg)
frequency	VARCHAR (100)	NOT NULL	Intake frequency (e.g. twice daily)
duration_days	INT	NOT NULL	Treatment duration in days
quantity	INT	NOT NULL	Total quantity to dispense

1.9.1.6 Medicine (Inventory)

Maintains the pharmacy's medicine catalogue and stock levels.

Field	Data Type	Constraints	Description
medicine_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique medicine identifier
name	VARCHAR (150)	NOT NULL	Medicine name
generic_name	VARCHAR (150)	NULLABLE	Generic / INN name
category	VARCHAR (100)	NOT NULL	Drug category
unit	VARCHAR (50)	NOT NULL	Unit of measure (tablet/ml/etc.)
quantity_in_stock	INT	NOT NULL, DEFAULT 0	Current available stock
reorder_level	INT	NOT NULL	Minimum stock before alert

expiry_date	DATE	NOT NULL	Medicine expiry date
is_quarantined	BOOLEAN	DEFAULT FALSE	Expired/quarantined flag
unit_price	DECIMAL (10,2)	NOT NULL	Price per unit

1.9.1.7 Audit Log

Records all system user actions for security and compliance purposes.

Field	Data Type	Constraints	Description
log_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique log entry identifier
user_id	INT	FOREIGN KEY → User	User who performed the action
action_type	VARCHAR (100)	NOT NULL	Type of action performed
entity_type	VARCHAR (100)	NULLABLE	Affected data entity
entity_id	VARCHAR (50)	NULLABLE	ID of affected record
description	TEXT	NULLABLE	Detailed action description
ip_address	VARCHAR (45)	NULLABLE	Client IP address
timestamp	DATETIME	NOT NULL, DEFAULT NOW ()	Timestamp of the action

1.9.1.8 External Dispensing

Tracks prescriptions fulfilled outside the clinic's pharmacy

Field	Data Type	Constraints	Description
external_dispense_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for external dispensing

prescription_id	INT	FOREIGN KEY → Prescription	Prescription being dispensed externally
patient_id	VARCHAR (10)	FOREIGN KEY → Patient	Patient receiving the external dispense
staff_id	INT	FOREIGN KEY → User	pharmacist performing the external dispense
dispense_date	DATETIME	NOT NULL, DEFAULT NOW ()	Date and time of external dispense
quantity_dispensed	INT	NOT NULL	Total quantity of medicine dispensed
status	ENUM('Dispensed','Partially Dispensed')	NOT NULL	Fulfilment status
delivery_method	ENUM('Pick-up','Delivery')	NOT NULL	How medicine is delivered
payment_method	ENUM('Cash')	NOT NULL	Payment method (aligns with BR-011)
audit_log_id	INT	FOREIGN KEY → Audit Log	Link to audit trail for compliance

1.9.2 Data Relationships

The key relationships between data entities are as follows:

- A Patient can have many Appointments (one-to-many).
- An Appointment can have one Prescription (one-to-one).
- A Prescription can have many Prescription Items (one-to-many).
- A Prescription Item references one Medicine from the inventory (many-to-one).
- A User (Doctor) can issue many Prescriptions (one-to-many).
- All User actions are recorded in the Audit Log (one-to-many).
- A Prescription can have zero or one External Dispense (optional one-to-one).
- An External Dispense references one Patient.
- External Dispense is performed by one User (Pharmacist).

- External Dispense updates one or more Medicine stock levels in Inventory.
- All External Dispense actions are recorded in the Audit Log.

1.9.3 Data Storage and Retention

- All data shall be stored in a MySQL 8.0 relational database deployed via Docker.
- Patient medical records shall be retained for a minimum of 7 years in accordance with applicable health data retention policies.
- Audit log records shall be retained for a minimum of 90 days in a centralized logging system.
- Automated database backups shall be performed every 24 hours and stored in secure cloud storage.
- Expired medicine records shall be flagged as quarantined but not deleted, to maintain historical dispensing traceability.
- External dispensing records shall be retained for a minimum of 7 years along with Prescription and Patient data.
- Inventory updates due to external dispensing shall maintain historical records for auditing purposes.
- Delivery and payment details for external dispensing shall be stored in the database and linked to the corresponding Prescription.

1.9.4 Data Integrity and Validation Rules

- Patient IDs must be unique and system-generated at registration.
- All prescription items must reference a valid, non-quarantined medicine in the inventory.
- Appointment dates must not be set in the past.
- Stock quantity must never fall below zero; negative stock updates shall be rejected by the system.
- All monetary values (unit prices, invoices) shall be stored as DECIMAL (10,2) to prevent floating point errors.
- Foreign key constraints shall be enforced at the database level to prevent orphaned records.
- External Dispense quantity must not exceed the remaining Prescription quantity.
- Patient identity must be verified (phone number, clinic ID, or QR code) before external dispensing.
- External Dispense actions must create corresponding Audit Log entries.

- Stock updates from external dispensing must enforce non-negative inventory; insufficient stock shall block the dispense.
- Prescription status shall reflect partial or full external dispensing.

1.10 Alternative Solutions Considered

Table 7: Alternative Solutions

Alternative	**Description**	**Reason for Rejection**
Manual / Paper-Based System	Continue managing patient records, appointments, and pharmacy stock using paper files and registers.	Prone to human errors; difficult to maintain records; no real-time updates; poor inventory tracking; time-consuming.
Phone-Based Appointment Booking	Patients book appointments via phone calls without any digital system.	No proper scheduling system: high chance of missed or overlapping appointments; no record tracking; inefficient communication.
Separate Clinic and Pharmacy Systems	Use two different systems for patient management and pharmacy operations.	Lack of integration; data duplication; inefficient workflow between doctor and pharmacist; increased operational complexity.
Spreadsheet-Based System	Use tools like spreadsheets to manage patient data and medicine inventory.	Not suitable for multi-user access; lacks automation; no real-time alerts (stock/expiry); high risk of data inconsistency.
Mobile Application Only	Develop only a native mobile app instead of a web-based system.	Higher development cost; requires separate apps for different platforms; less accessible for clinic staff compared to web system.

Manual + Partial Digital System	Use a combination of manual records and basic digital tools.	Data inconsistency; duplication of work; increased chances of errors; inefficient workflow.
Admin Approval for Appointment Booking	After a patient has booked an appointment, the administrator must manually review and approve each	Causes delays in appointment confirmation; increases admin workload; inefficient for small clinics with limited staff; real-time
	booking before confirmation.	booking without approval is faster and more user-friendly.
Online Payment for Appointment Booking	Require patients to make online payments (card or digital payment) when booking appointments.	Not suitable for rural areas where most patients do not use online payments; may reduce accessibility; cash-based payment at the clinic is more practical and convenient.

1.11 Feasibility Study

This chapter assesses the feasibility of the OneCare Clinic Integrated Pharmacy Management System across four dimensions: technical, economic, operational, and schedule feasibility.

1.11.1 Technical Feasibility

Technical feasibility evaluates whether the proposed system can be built using available technologies, tools, and the development team's existing skill sets.

1.11.1.1 Technology Stack Assessment

Table 8: Feasibility Study

Component	**Proposed Technology**	**Justification**
Frontend	React.js	Component-based, responsive design, widely supported
Backend	Spring Boot	Scalable, modular, well documented, suitable for REST APIs.
Database	MySQL 8.0	Relational model suitable for clinic records; Docker compatible
Authentication	JWT + bcrypt/Argon2	Stateless secure authentication; industry standard.
Containerization	Docker	Consistent deployment environment, OS-independent.
Notifications	SMS API (e.g. e-SMS or Twilio)	Reliable SMS delivery for appointment reminders
API Documentation	Swagger / Open API	Standardized, auto-generated API documentation

1.11.1.2 Technical Feasibility Assessment

- The development team has prior experience with the proposed technology stack through coursework and projects.
- All tools and frameworks are open-source, eliminating licensing costs.

- Modular system architecture separates frontend, backend, and database layers.
- Docker ensures environment consistency across clinic devices.
- Offline operations, including queue updates, appointment logging, and external dispensing, introduce moderate technical complexity but are achievable via local caching and service worker strategies.
- External dispensing functionality interacts with inventory, patient verification, and audit logging; the system design supports these workflows without introducing new dependencies.

Conclusion: The project is technically feasible. The chosen technologies are mature, widely documented, and well within the competence of the development team.

1.11.1.3 Economic Feasibility

Economic feasibility examines whether the system can be developed and operated within acceptable financial boundaries relative to the benefits it provides. 12.2.1 Cost Estimate

Cost Item	Estimated Cost (LKR)
Development (student project - no labor cost)	0
Cloud hosting (e.g. AWS / Digital Ocean – 12 months)	~60,000 – 120,000
Domain name registration (1 year)	~3,000 – 5,000
SSL certificate (Let's Encrypt – free)	0
SMS notification service (Twilio / local provider)	~10,000 – 20,000 per year
Software licenses (open-source stack)	0
Total Estimated Annual Operational Cost	~73,000 – 145,000 LKR

12.2.2 Benefit Analysis

- Elimination of paper-based record management reduces stationery and storage costs.
- Reduction in appointment management errors decreases patient no-shows, improving revenue consistency.
- Automated inventory alerts prevent stock-outs and reduce losses from expired medicines.
- Digital prescriptions eliminate re-dispensing errors, reducing potential liability.

- Estimated productivity gain of 30–40% in administrative tasks based on similar clinic system deployments.

Conclusion: The system is economically feasible. The operational costs are modest relative to the efficiency gains and error reduction benefits for a small clinic environment.

1.11.2 Operational Feasibility

Operational feasibility assesses whether the proposed system will be accepted and successfully used by the clinic's staff within the day-to-day working environment.

1.1.3.1 User Readiness

- Clinic staff are assumed to have basic computer literacy (ASM-001).
- The system is designed for ease of learning: a new staff member shall be able to perform basic tasks within 30 minutes of first use (NFR-U002).
- All critical tasks require no more than three navigation steps (NFR-U003), reducing the learning curve.

1.1.3.2 Organizational Impact

- The transition from manual to digital workflows will require a one-time data migration effort for existing patient records and medicine inventory.
- An initial training session for all staff roles (Doctor, Pharmacist, Admin) is planned prior to system go-live.
- The system does not require changes to the clinic's physical infrastructure beyond provision of computer workstations, which are already available.
- The Super Admin role ensures ongoing user management without dependence on external technical support for routine tasks.

Conclusion: The system is operationally feasible. The intuitive interface design and multilingual support directly address the identified operational barriers.

1.11.3 Schedule Feasibility

Schedule feasibility evaluates whether the system can be delivered within the planned timeline.

1.11.3.1 Estimated Project Timeline

Phase	Description	Duration	Milestone
1	Requirements Analysis & SRS Documentation	3 weeks	SRS approved

2	System Design (Architecture, Database, UI Mockups)	3 weeks	Design document
3	Frontend Development (React.js, core modules)	6 weeks	UI prototype
4	Backend Development (APIs, Auth, DB integration)	6 weeks	API tested
5	Integration Testing & Bug Fixing	3 weeks	Test reports
6	User Acceptance Testing (UAT) with clinic staff	2 weeks	UAT sign-off
7	Deployment, Data Migration & Training	2 weeks	System go-live
8	Buffer Time	3 weeks	Contingency
Total		~28 weeks	

12.4.2 Schedule Risk Factors

- Four-member team enables parallel development of frontend and backend.
- Academic examination periods may cause schedule interruptions and have been accounted for in the timeline estimates.
- Modular architecture allows phased delivery, mitigating full-system delays.
- Scope has been clearly bounded (excluding online payments, multi-clinic support, and telemedicine) to prevent scope creep.

Conclusion: The schedule is feasible. The 28-week timeline is achievable for a four-member team given the defined scope, provided that parallel development tracks are maintained and scope is strictly controlled.

1.12 RISK ANALYSIS

This section identifies and evaluates potential risks associated with the development of the OneCare Clinic Integrated Pharmacy Management System. The analysis focuses on technical, resource, schedule, requirements, and external risks that may impact project execution and delivery. Appropriate mitigation strategies have been defined to minimize the likelihood and impact of each identified risk.

Risk ID	Description	Category	Likelihood	Impact	Risk Level	Mitigation Strategy
R1	Unequal distribution of tasks among team members	Resource	High	Medium	High	Tasks shall be clearly defined and allocated based on individual competencies, and progress shall be monitored regularly.
R2	Ineffective Communication among team members	Resource	Medium	High	High	Regular team meetings shall be conducted, and collaboration tools (e.g., messaging platforms, task boards) shall be used to ensure continuous communication.
R3	Insufficient technical	Technical	Medium	High	High	Required technical

	expertise in specific development areas					knowledge shall be acquired through documentation, training resources, and guidance from experienced individuals.
R4	Integration issues between frontend and backend components	Technical	Medium	High	High	API contracts shall be defined early, and integration testing shall be performed incrementally throughout development.
R5	High occurrence of software defects during development	Technical	High	Medium	High	Code reviews, unit testing, and continuous debugging practices shall be implemented throughout the development lifecycle.
R6	Delays due to academic workload and	Schedule	High	Medium	High	A realistic project schedule shall be prepared, and

	internship commitments					tasks shall be prioritized and distributed to balance workload effectively.
R7	Scope creep due to Uncontrolled addition of new features	Requirements	Medium	High	High	All requirements shall be strictly managed according to the approved SRS, and any changes shall follow a formal change control process.
R8	Misunderstanding or ambiguity in system requirements	Requirements	Medium	High	High	Requirements shall be reviewed regularly with stakeholders, and clarifications shall be documented and approved.
R9	Version control conflicts and improper code management	Technical	Medium	Medium	Medium	A standardized branching strategy shall be followed, and team members shall be trained

						in version control best practices.
R10	Dependency on a single team member for critical tasks (key person risk)	Resource	Medium	High	High	Knowledge sharing shall be encouraged, and critical tasks shall be documented and distributed among multiple team members.
R11	Data loss during development	Technical	Low	High	Medium	Regular backups shall be maintained, and version control systems shall be used to preserve code and data integrity.
R12	Deployment and Environment configuration issues	Technical	Medium	Medium	Medium	Deployment shall be tested in a staging environment prior to production release, and configuration steps shall be documented.

R13	Inaccurate estimation of task duration	Schedule	High	Medium	High	Tasks shall be broken down into smaller units, and progress shall be tracked against defined milestones.
R14	Network connectivity issues affecting development activities	External	Medium	Medium	Medium	Alternative internet sources (e.g., mobile hotspots) shall be used, and offline work shall be planned where possible.
R15	Data security and patient privacy risks	Technical	Medium	High	High	Security measures such as authentication, authorization, and data encryption shall be implemented, following standard security practices.
R16	System downtime or performance degradation	Technical	Medium	High	High	System performance shall be monitored, and load handling strategies shall be

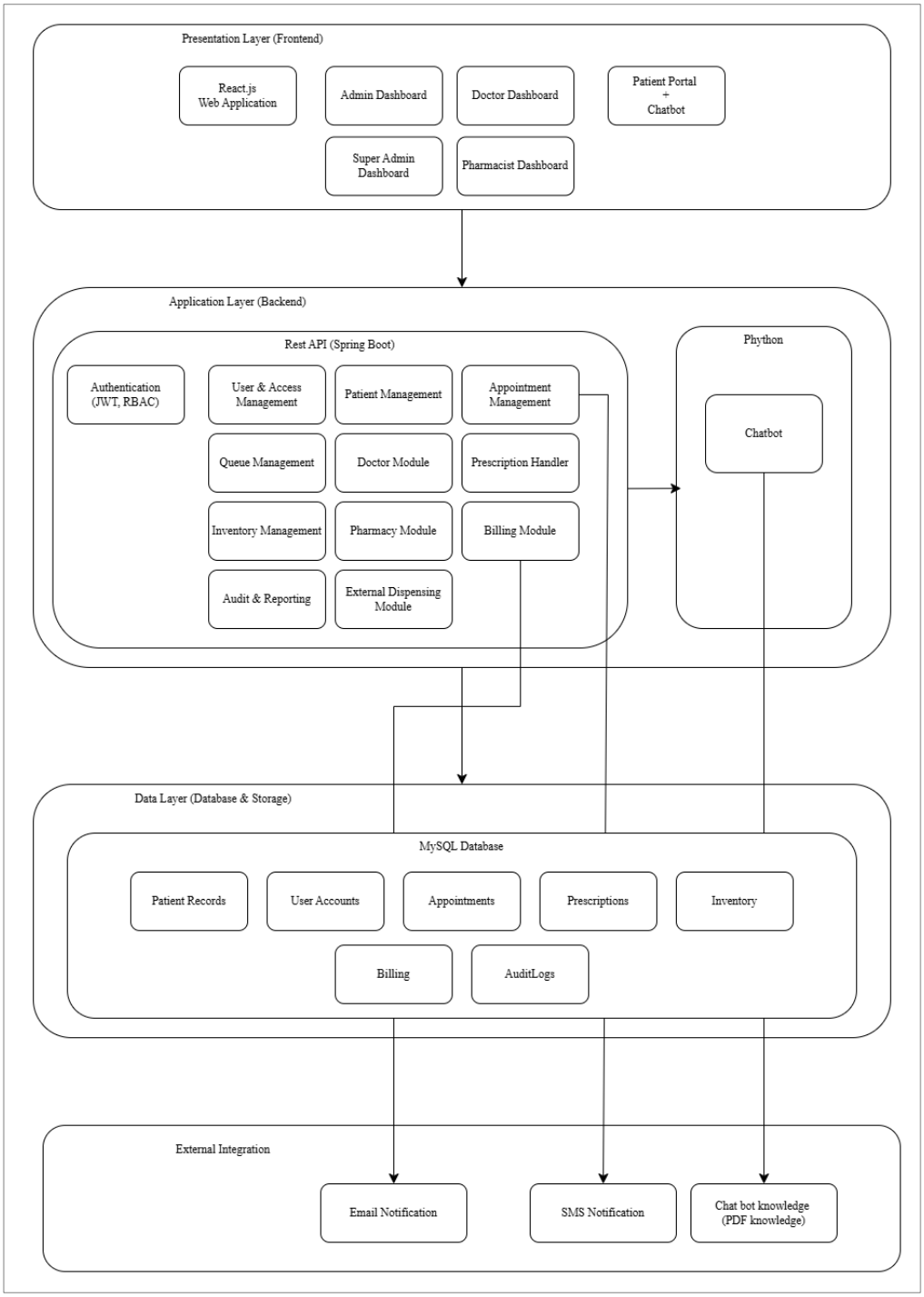
						implemented to ensure availability.
R17	Failure of third-party services	External	Medium	Medium	Medium	Reliable service providers shall be selected, and
	(e.g., SMS, APIs)					Fallback mechanisms or alternatives shall be considered.
R18	Resistance from users in adopting the new system	Operational	Medium	Medium	Medium	User training and system demonstrations shall be conducted to improve usability and acceptance.

ANNEX 2: SYSTEM DESIGN SPECIFICATION

2.1 Architectural Design

2.1.1 High-Level Architecture

A Three-Tier Layered Architecture is adopted by the OneCare Clinic Integrated Pharmacy Management System, consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database). Separation of concerns, scalability, security, and maintainability are ensured through this architectural approach.



2.1.2 Architecture Overview

Presentation Layer (Frontend Components)

- React.js web application serving all interfaces
- Role-based dashboards for Admin, Doctor, Pharmacist, and Super Admin
- Patient self-service portal with chatbot integration
- All communicate via REST API

Application Layer (Backend Services)

The backend is organized around a Spring Boot REST API gateway with several key functional modules:

- **Security**: Authentication & Authorization using JWT and role-based access control (RBAC)
- **Core Business Modules**:
    - User & Access Management
    - Patient Management
    - Appointment Management
    - Queue Management
    - Doctor Module
    - Pharmacy Module
    - Inventory Management
    - Billing Module
    - Chatbot Service
    - Prescription Handler

Data Layer (MySQL Database)

- Central MySQL database storing all operational data
- Organized tables for patients, users, appointments, prescriptions, inventory, billing records, audit logs, and chatbot knowledge base

External Services (Third-Party Integrations)

- SMS Notification Service (for appointment reminders)
- Email Service (SMTP for confirmations and reports)
- Chatbot Knowledge Base Storage (PDF documents)

2.1.3 Cloud-Native Architecture

The OneCare system is designed as a cloud-native application with containerization, enabling scalability and deployment flexibility.

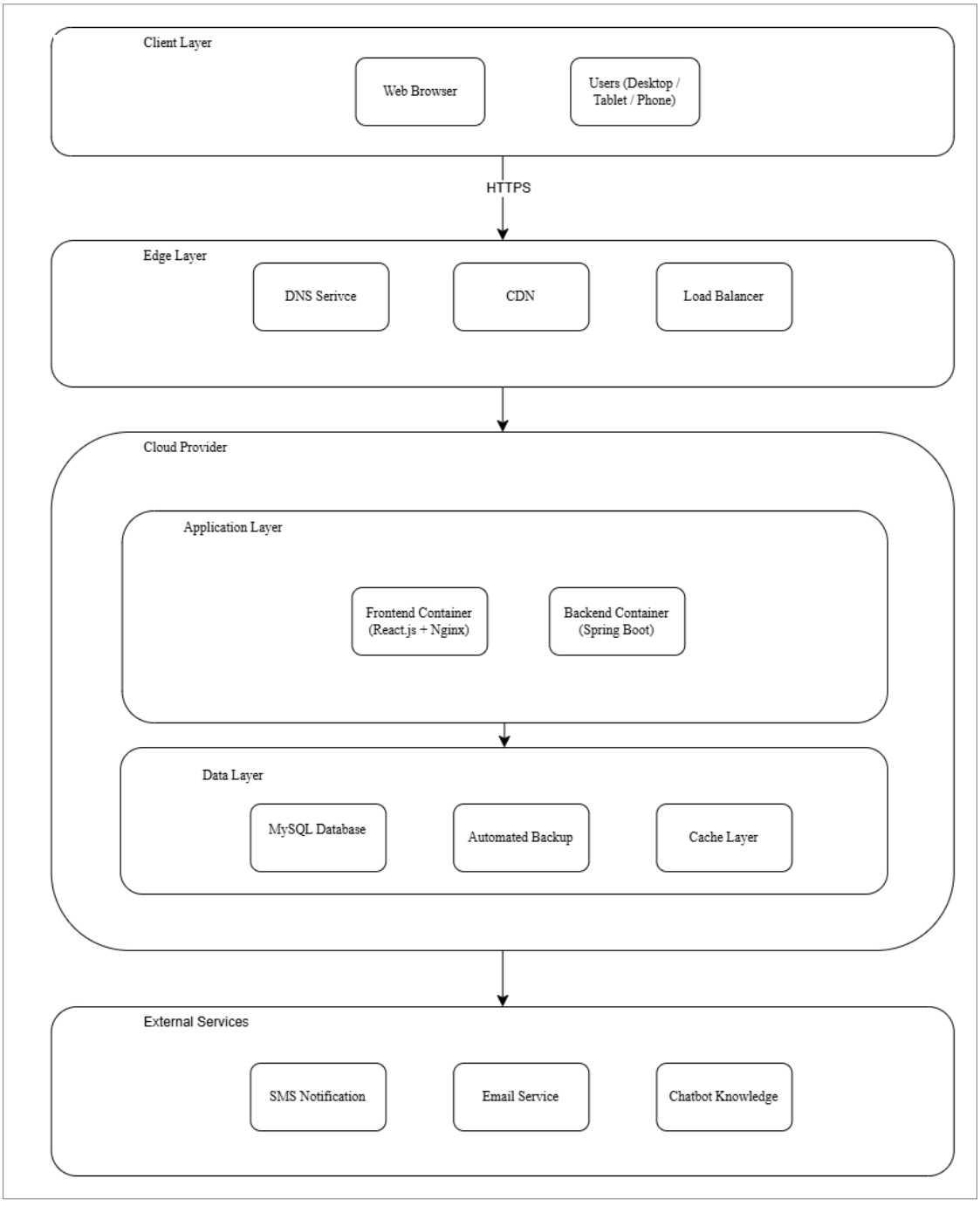


Figure 26 :Cloud Native Diagram

2.1.4 Justification for architecture choice

2.1.4.1 Why Three-Tier Layered Architecture?

- **Separation of Concerns:** Clear division between presentation, business logic, and data access enables independent development and testing.
- **Scalability:** Each layer can be scaled independently. Frontend via CDN, backend via load balancing, database via optimization.
- **Security:** Clear enforcement points for authentication and authorization. Data access control isolated in data layer.
- **Maintainability:** Modular design allows feature additions without major refactoring. Clear API contracts between layers.
- **Cost-Effectiveness:** Leverages open-source technologies with no licensing fees. Cloud-native design reduces infrastructure costs.
- **Technology Alignment:** Matches proposed stack (React, Spring Boot, MySQL). All technologies are mature and widely supported.

2.1.5 Why NOT Microservices?

For a clinic system supporting ~25 patients/day, microservices architecture is not justified because:

- Operational complexity increases (service discovery, distributed transactions)
- Higher infrastructure costs and deployment complexity
- Overkill for the system scale and requirements
- Development team expertise better suited for layered monolithic backend

2.1.6 Scalability and Maintainability Considerations

1.1.3.3 Scalability Measures

- **Horizontal Scaling**: Stateless backend services can be scaled behind a load balancer
- **Performance Optimization**: API response ≤2 seconds, Dashboard load ≤3 seconds
- **Database Optimization**: Indexing on frequently queried fields, connection pooling
- **Caching Strategy**: Redis for frequently accessed data (optional)
- **Offline Capability**: Service Workers and Indexed DB for 5-minute offline operations

1.1.3.4 Maintainability Measures

- **Modular Architecture**: Separated modules per feature with clear responsibilities
- **Code Quality Standards**: ≥70% unit test coverage, SonarQube analysis
- **API Documentation**: Swagger/Open API auto generated from annotations
- **Logging & Monitoring**: Centralized logging with 90-day retention
- **Version Control**: Git-based with CI/CD pipeline

2.2 Database Design

2.2.1 ER Diagram

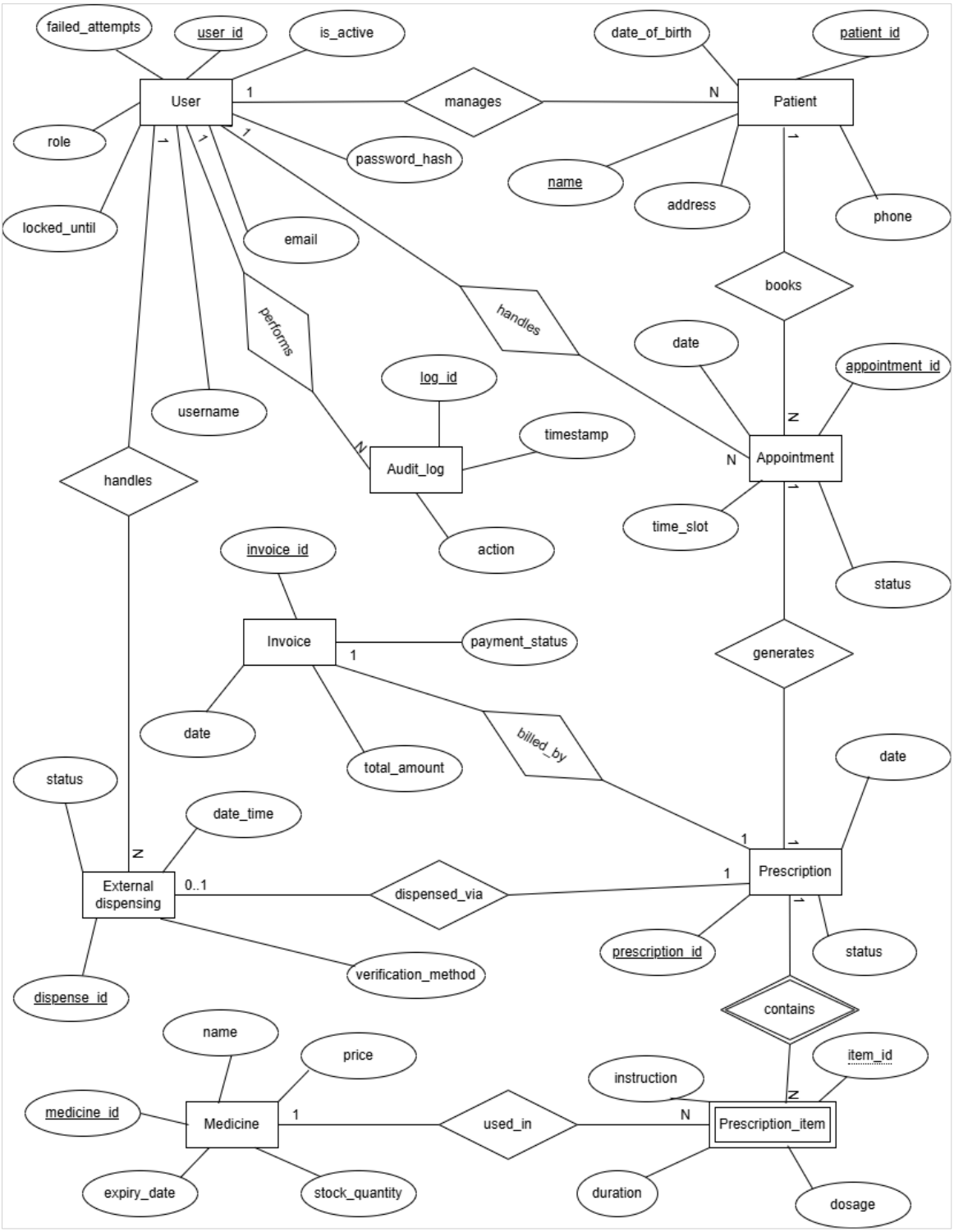


Figure 27: ER Diagram

2.2.2 Normalization Explanation

Normalization is a systematic process used in relational database design to minimize data redundancy and improve data integrity. It involves decomposing tables into smaller, well-structured relations based on defined rules known as Normal Forms (1NF, 2NF, and 3NF).

1.1.3.5 First Normal Form(1NF)

A relation is said to be in First Normal Form (1NF) when all attributes contain atomic (indivisible) values, repeating groups are eliminated, and each record is uniquely identifiable using a primary key.

A poorly designed Patient table in the proposed system is illustrated as follows:

Before 1NF

patient_id	name	address	contact_number	date_of_birth
P001	Arun	jaffna	0756737217,0728605573	15.10.1997

After 1NF

patient_id	name	address	contact_number	date_of_birth
P001	Arun	jaffna	0728605573	15.10.1997
P001	Arun	jaffna	0756737217	15.10.1997

In the normalized structure, each contact number is stored as a separate record, ensuring atomicity and eliminating repeating groups.

In the proposed system, a poorly designed Prescription Item table is illustrated as follows:

Before 1NF

prescription_id	Medicines
1	Paracetamol 500mg, Amoxicillin 250mg
2	Ibuprofen 400mg

After 1NF

item_id	prescription_id	medicine	dosage
1	1	Amoxicillin	250mg
2	1	Paracetamol	500mg
3	2	Ibuprofen	400mg

In the normalized structure, each medicine is stored as a separate record, ensuring atomicity and eliminating repeating groups.

1.1.3.6 Second Normal Form(2NF)

The relation is in Second Normal Form (2NF) if it is already in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. This eliminates partial dependencies, which occur when a non-key attribute depends only on part of a composite key.

In the Prescription Item table, the following structure is observed:

Before 2NF

prescription_id	medicine_id	medicine_name	price	dosage	duration
1	M01	Paracetamol	5.00	500mg	5days
2	M01	Paracetamol	5.00	250mg	3days

After 2NF

Prescription Item table – item_id, prescription_id, medicine_id, dosage, duration, instruction
Medicine table – medicine_id, name, price, expiry_date, stock_quantity

In the unnormalized structure, attributes such as medicine_name and price depend only on medicine_id rather than the full composite key. This results in redundancy. By separating medicine details into a dedicated Medicine table, partial dependency is removed.

1.1.3.7 Third Normal Form(3NF)

A relation is in Third Normal Form (3NF) if it is already in 2NF and contains no transitive dependencies. This means that non-key attributes must depend only on the primary key and not on other non-key attributes.

Before 3NF

Prescription table - prescription_id, date, status, doctor_id, doctor_name, doctor_email
The attribute doctor_name depends on doctor_id rather than on prescription_id. This represents transitive dependency.

After 3NF

After normalization, the Prescription table stores only prescription_id, date, status, and doctor_id. Doctor-related attributes are maintained in the User table, where each user is

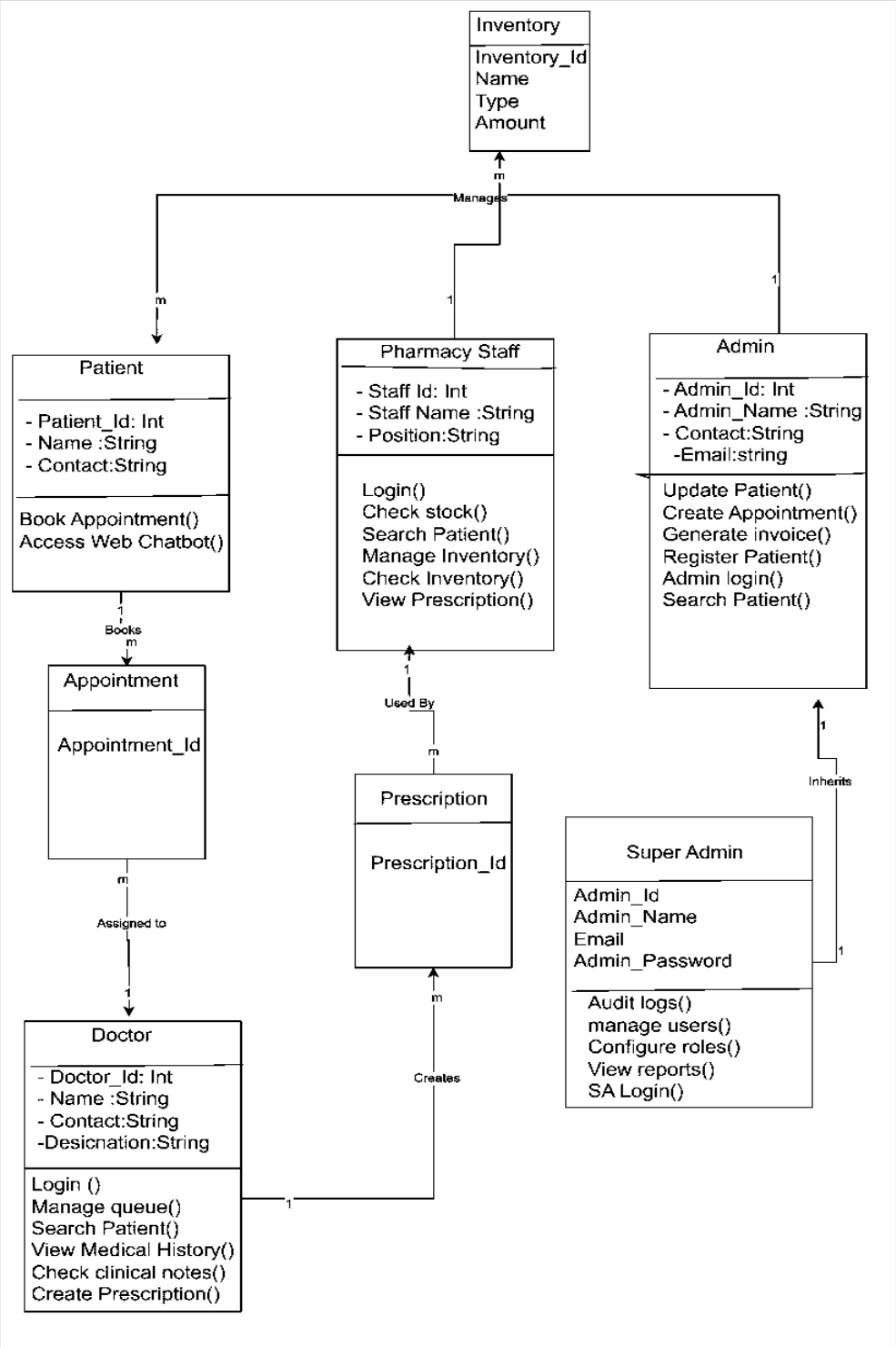
assigned a role. The doctor_id acts as a foreign key referencing the corresponding user record.

In the unnormalized structure, the attributes doctor_name and doctor_email depend on doctor_id rather than directly on prescription_id, creating a transitive dependency. This dependency is removed by storing all user-related details in the User table and referencing them through doctor_id as a foreign key.

Thus, normalization ensures that the database structure is efficient, consistent, and free from update anomalies, supporting reliable data management within the system.

2.3 Class Diagram

This class diagram represents a Clinic Management System designed to handle patient care, appointments, prescriptions, and pharmacy operations. It models the structure of the system by showing the key classes, their attributes, methods, and relationships. The system involves multiple roles such as Patient, Doctor, Admin, Pharmacy Staff, and Super Admin, each responsible for different functionalities.



2.3.1 Description of the Class Diagram

The system is centered around managing patients and medical services efficiently:

- Patient

  Individuals receiving medical services are represented by the Patient entity. Appointments may be booked by patients, and access to basic system features such as chatbot support is provided.

- Appointment

  Scheduling between patients and doctors is handled by the Appointment entity. Multiple appointments may be booked by a patient, and each appointment is assigned to a doctor.

- Doctor

  The diagnosis of patients is performed by a single doctor in the system. Patient queues are managed, prescriptions are created, and medical history and clinical notes are accessed by the doctor.

- Prescription

  Prescriptions are created by doctors after consultation. They contain treatment or medicine details and are later used by pharmacy staff.

- Pharmacy Staff

  Medicine-related operations such as stock checking, prescription viewing, and inventory updates are managed by pharmacy staff. Medication is dispensed based on prescriptions.

- Inventory

  Details of medicines such as name, type, and quantity are stored in the Inventory entity. It is managed by both Admin and Pharmacy Staff to ensure drug availability.

- Admin

  Administrative tasks such as patient registration, appointment creation, invoice generation, and patient detail updates are handled by the admin role.

- Super Admin

  A higher-level role is defined as Super Admin, through which advanced privileges such as user management, role configuration, report viewing, and audit log maintenance are provided.

2.3.2 Relationships Overview

- A Patient may have multiple Appointments (1 → many).
- Each Appointment is assigned to a single doctor in the system.

- A Doctor creates multiple Prescriptions.
- Prescriptions are used by Pharmacy Staff to dispense medicine.
- Inventory is managed by both Admin and Pharmacy Staff.
- Super Admin extends the admin class with additional control features.

2.4 Sequence Diagram

The sequence diagrams are used to illustrate how various users, including Admin, Doctor, Pharmacy Staff, and Super Admin, interact with the system to perform different operations. The step-by-step flow of messages between actors, the system, and databases is presented in a time-ordered manner.

Key functionalities such as patient registration, appointment handling, prescription management, inventory control, billing, user management, and audit logging are covered in these diagrams. The way requests are sent, processed, and responses are returned is clearly depicted, ensuring proper coordination between system components.

Overall, the dynamic behavior of the system is demonstrated through sequence diagrams, and the interaction between different modules to support efficient clinic management is effectively represented.

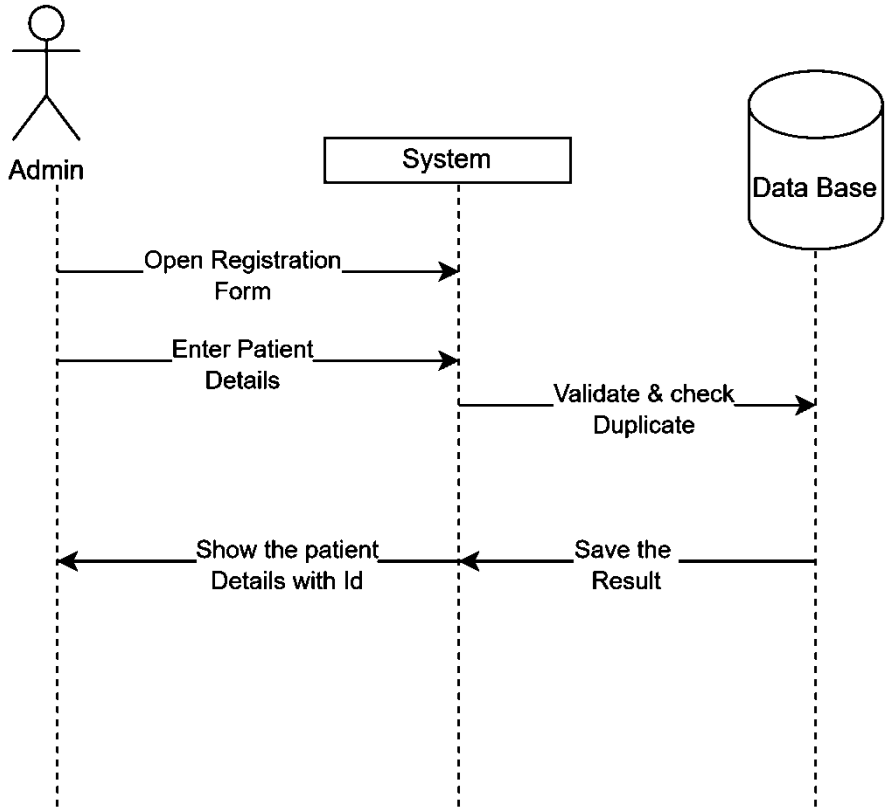
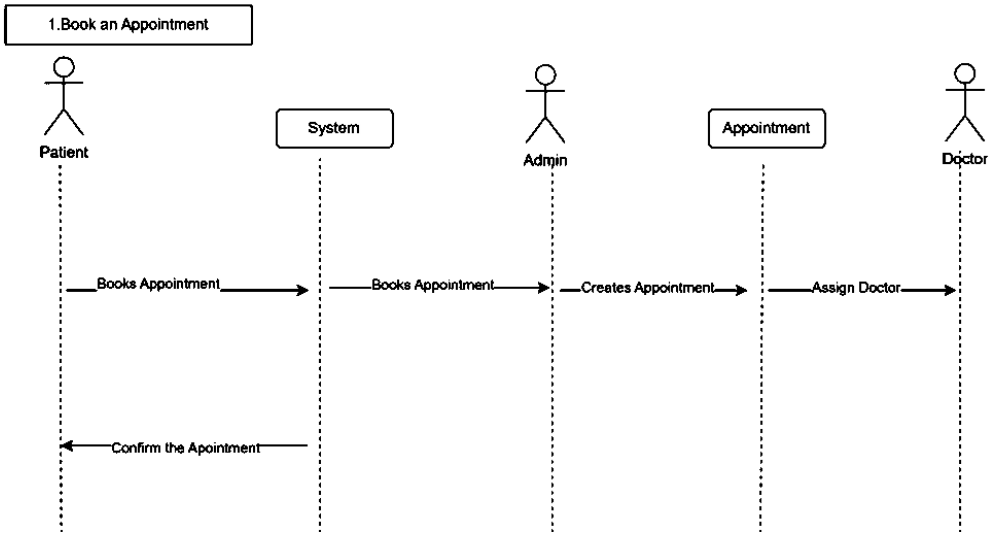
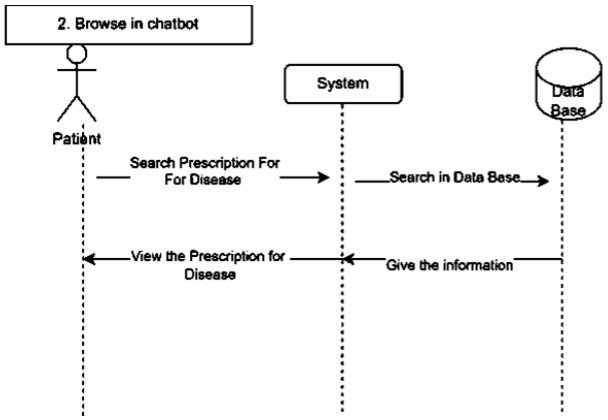


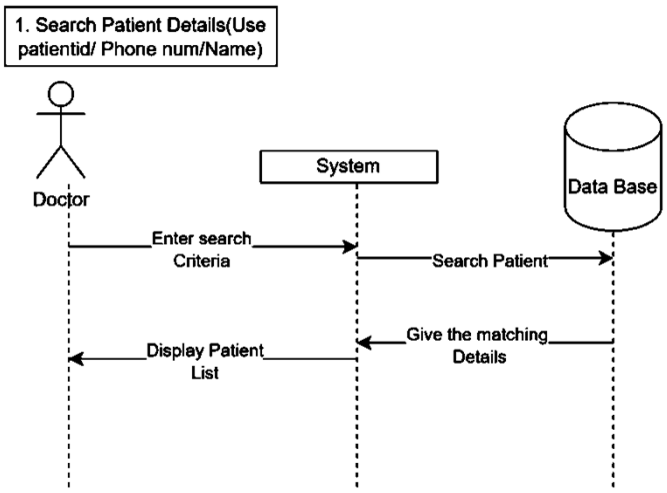
Figure 28: Sequence Diagrams

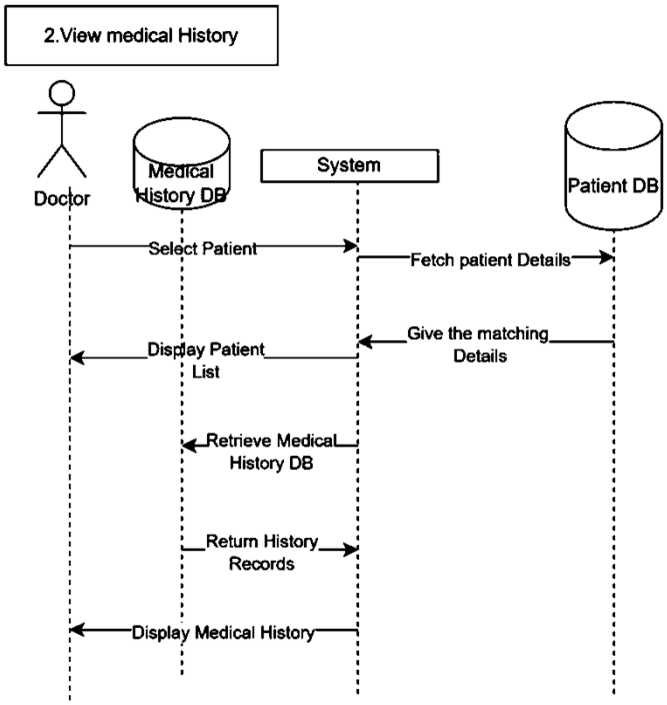
2.4.1 Sequence Diagram for Patient



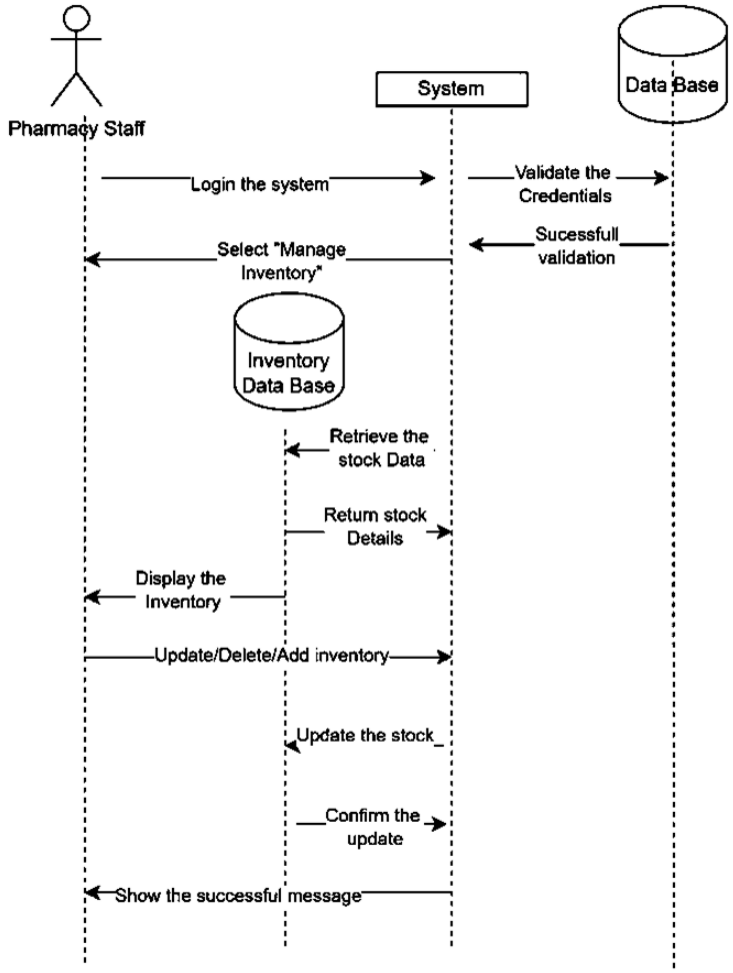


2.4.2 Sequence Diagram for Doctor

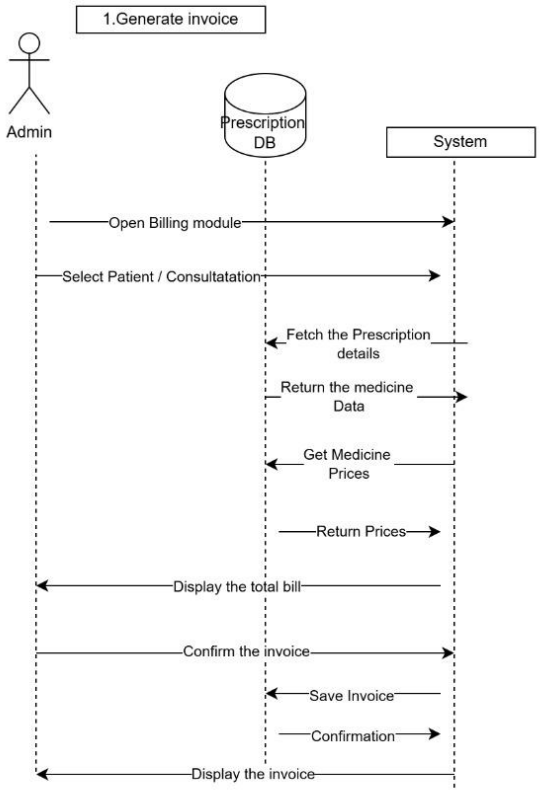


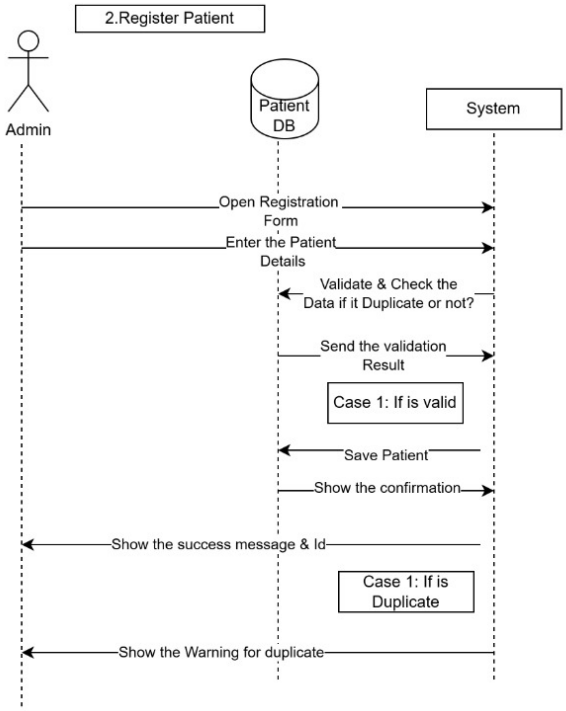


2.4.3 Sequence Diagram for Pharmacy Staff

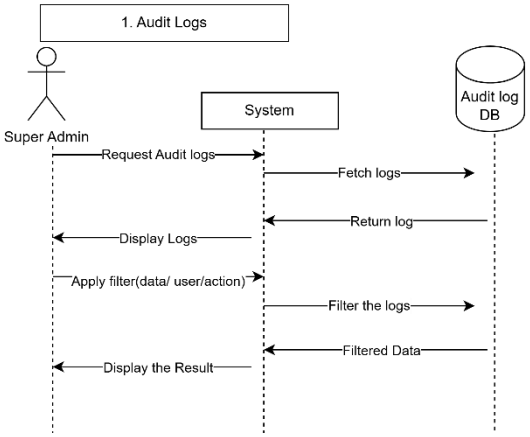


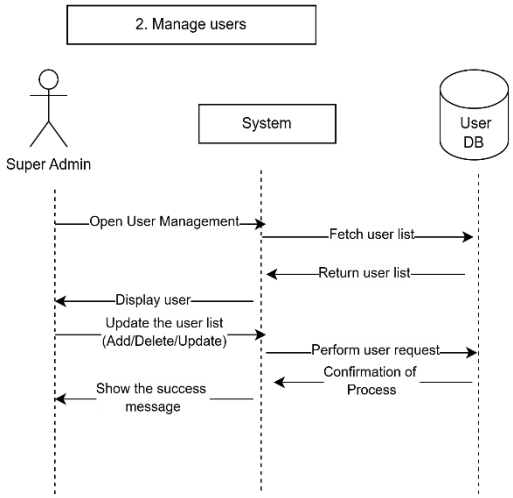
2.4.4 Sequence Diagram for Admin

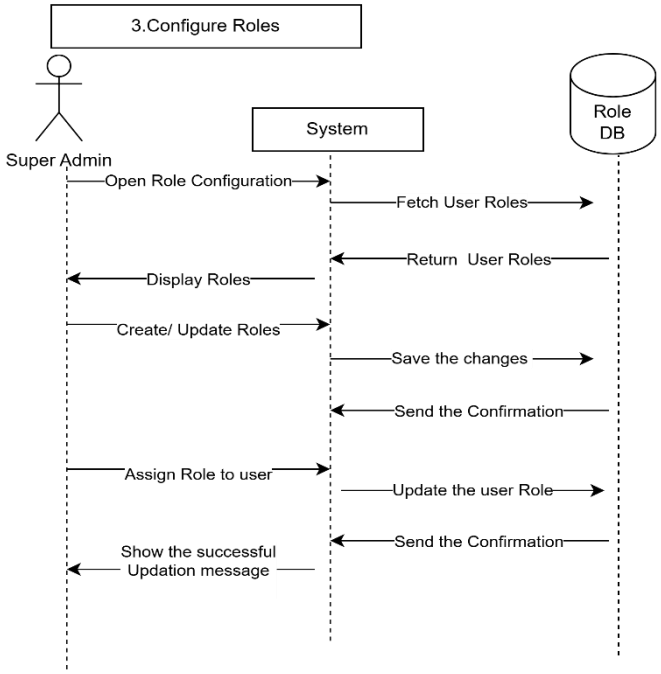




2.4.5 Sequence Diagram for Super Admin






3.Configure Roles
Super Admin
System
Role
DB
Open Role Configuration
Fetch User Roles
Return User Roles
Display Roles
Create/ Update Roles
Save the changes
Send the Confirmation
Assign Role to user
Update the user Role
Send the Confirmation
Show the successful
Updation message

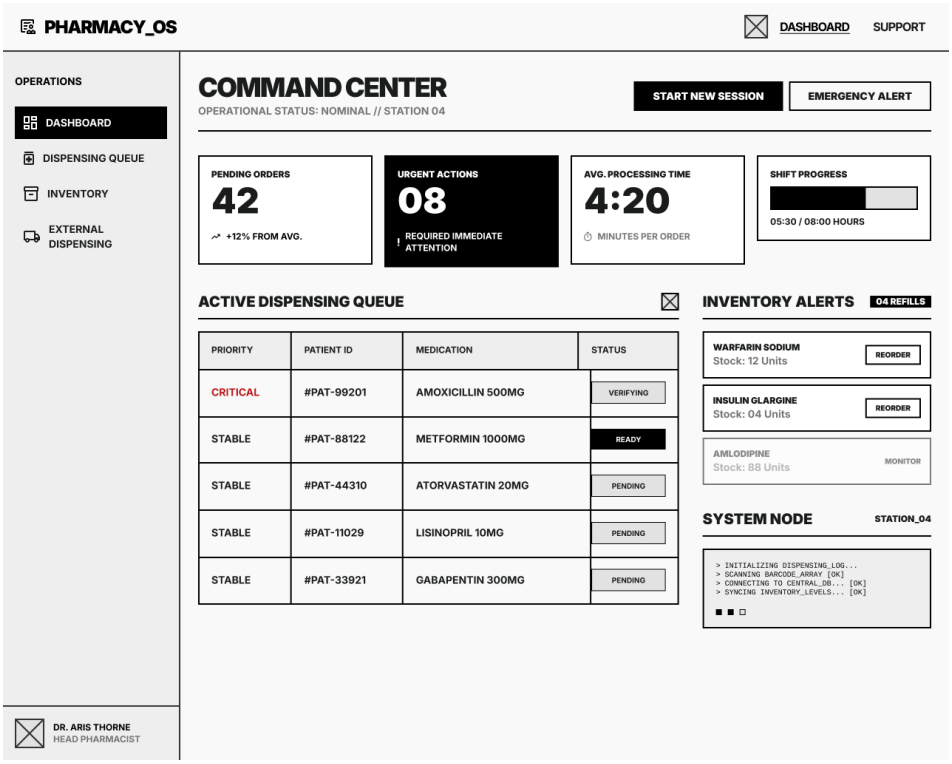
UI/UX Design

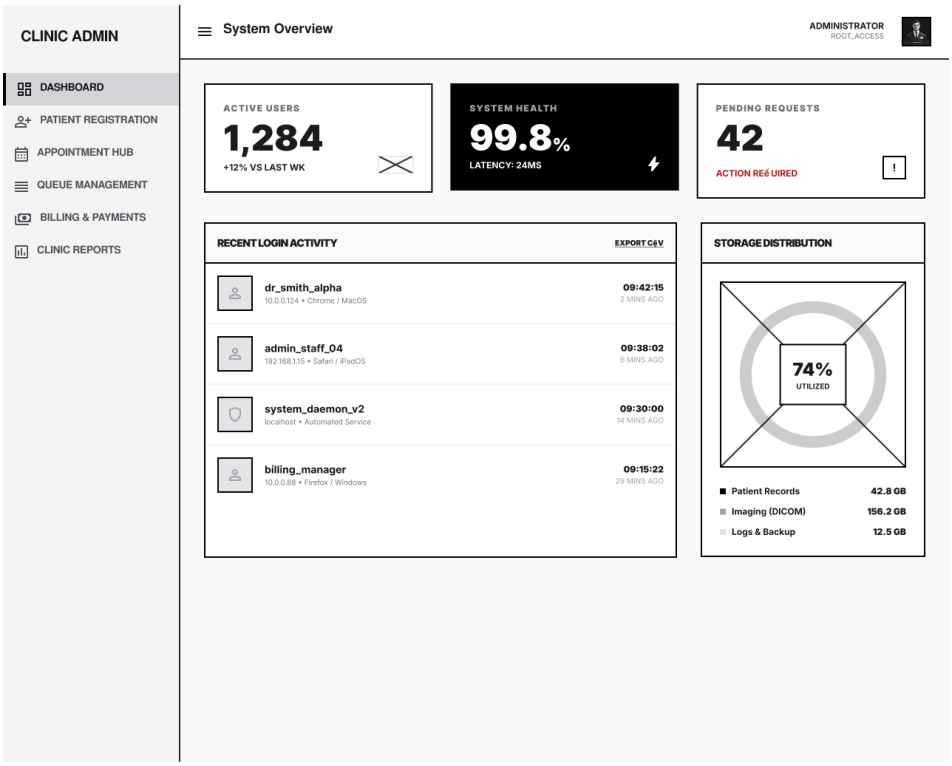
2.4.6 Wireframes and Screen Mock-ups

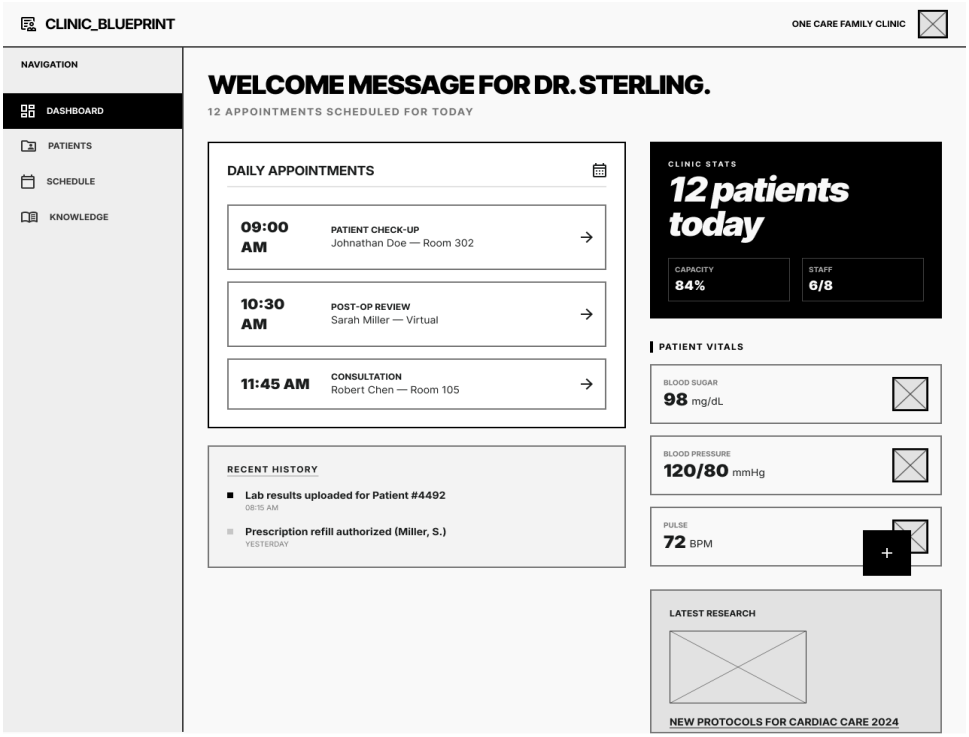
The user interface of the OneCare Clinic Management System is designed using a user-centered approach to ensure usability, efficiency, and clarity across different user roles.

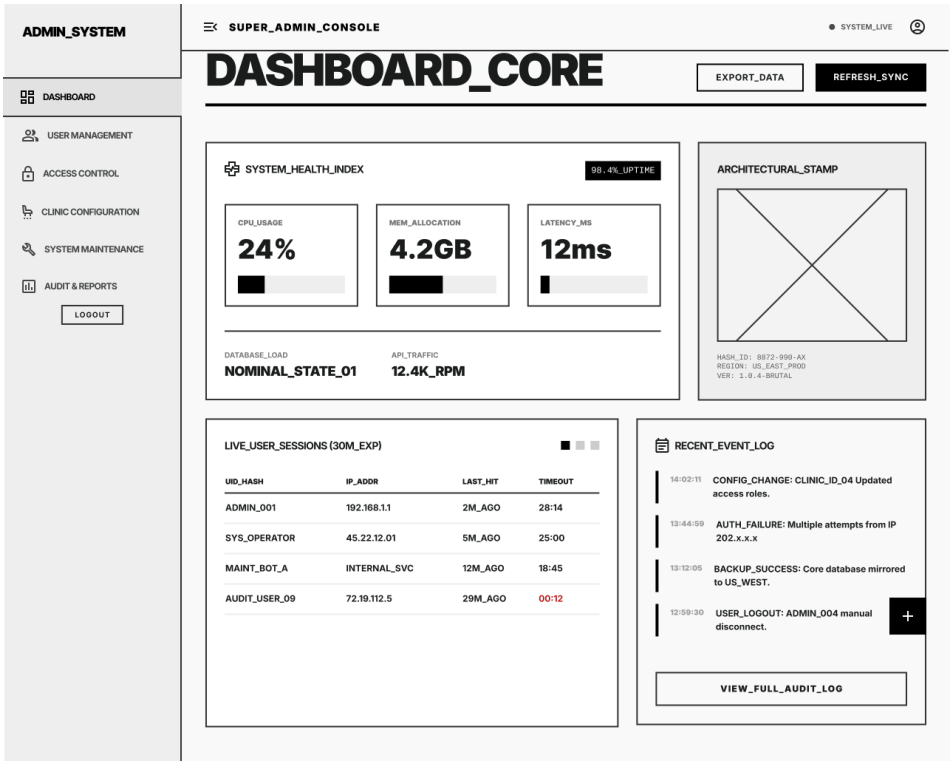
Wireframes and high-fidelity mock-ups have been created using Figma to visualize the layout, structure, and interaction of key system components. These designs provide a clear representation of user interfaces before implementation.




PHARMACY_OS
DASHBOARD
SUPPORT
OPERATIONS
DASHBOARD
DISPENSING QUEUE
INVENTORY
EXTERNAL DISPENSING
COMMAND CENTER
OPERATIONAL STATUS: NOMINAL // STATION 04
START NEW SESSION
EMERGENCY ALERT
PENDING ORDERS
42
+12% FROM AVG.
URGENT ACTIONS
08
REQUIRED IMMEDIATE ATTENTION
AVG. PROCESSING TIME
4:20
MINUTES PER ORDER
SHIFT PROGRESS
05:30 / 08:00 HOURS
ACTIVE DISPENSING QUEUE
PRIORITY
PATIENT ID
MEDICATION
STATUS
CRITICAL
#PAT-99201
AMOXICILLIN 500MG
VERIFYING
STABLE
#PAT-88122
METFORMIN 1000MG
READY
STABLE
#PAT-44310
ATORVASTATIN 20MG
PENDING
STABLE
#PAT-11029
LISINOPRIL 10MG
PENDING
STABLE
#PAT-33921
GABAPENTIN 300MG
PENDING
INVENTORY ALERTS
04 REFILLS
WARFARIN SODIUM
Stock: 12 Units
REORDER
INSULIN GLARGINE
Stock: 04 Units
REORDER
AMLODIPINE
Stock: 88 Units
MONITOR
SYSTEM NODE
STATION_04
> INITIALIZING DISPENSING_LOG...
> SCANNING BARCODE_ARRAY [OK]
> CONNECTING TO CENTRAL_DB... [OK]
> SYNCING INVENTORY_LEVELS... [OK]
DR. ARIS THORNE
HEAD PHARMACIST


CLINIC ADMIN
System Overview
ADMINISTRATOR
ROOT_ACCESS
DASHBOARD
PATIENT REGISTRATION
APPOINTMENT HUB
QUEUE MANAGEMENT
BILLING & PAYMENTS
CLINIC REPORTS
ACTIVE USERS
1,284
+12% VS LAST WK
SYSTEM HEALTH
99.8%
LATENCY: 24MS
PENDING REQUESTS
42
ACTION REQUIRED
RECENT LOGIN ACTIVITY
EXPORT CSV
dr_smith_alpha
10.0.0.124 • Chrome / MacOS
09:42:15
2 MINS AGO
admin_staff_04
192.168.1.15 • Safari / iPadOS
09:38:02
6 MINS AGO
system_daemon_v2
localhost • Automated Service
09:30:00
14 MINS AGO
billing_manager
10.0.0.88 • Firefox / Windows
09:15:22
29 MINS AGO
STORAGE DISTRIBUTION
74%
UTILIZED
Patient Records
42.8 GB
Imaging (DICOM)
156.2 GB
Logs & Backup
12.5 GB


CLINIC_BLUEPRINT
ONE CARE FAMILY CLINIC
NAVIGATION
DASHBOARD
PATIENTS
SCHEDULE
KNOWLEDGE
WELCOME MESSAGE FOR DR. STERLING.
12 APPOINTMENTS SCHEDULED FOR TODAY
DAILY APPOINTMENTS
09:00 AM
PATIENT CHECK-UP
Johnathan Doe — Room 302
10:30 AM
POST-OP REVIEW
Sarah Miller — Virtual
11:45 AM
CONSULTATION
Robert Chen — Room 105
RECENT HISTORY
Lab results uploaded for Patient #4492
08:15 AM
Prescription refill authorized (Miller, S.)
YESTERDAY
CLINIC STATS
12 patients today
CAPACITY
84%
STAFF
6/8
PATIENT VITALS
BLOOD SUGAR
98 mg/dL
BLOOD PRESSURE
120/80 mmHg
PULSE
72 BPM
LATEST RESEARCH
NEW PROTOCOLS FOR CARDIAC CARE 2024


ADMIN_SYSTEM
SUPER_ADMIN_CONSOLE
SYSTEM_LIVE
DASHBOARD
USER MANAGEMENT
ACCESS CONTROL
CLINIC CONFIGURATION
SYSTEM MAINTENANCE
AUDIT & REPORTS
LOGOUT
DASHBOARD_CORE
EXPORT_DATA
REFRESH_SYNC
SYSTEM_HEALTH_INDEX
98.4%_UPTIME
CPU_USAGE
24%
MEM_ALLOCATION
4.2GB
LATENCY_MS
12ms
DATABASE_LOAD
NOMINAL_STATE_01
API_TRAFFIC
12.4K_RPM
ARCHITECTURAL_STAMP
HASH_ID: 8872-990-AX
REGION: US_EAST_PROD
VER: 1.0.4-BRUTAL
LIVE_USER_SESSIONS (30M_EXP)
UID_HASH
IP_ADDR
LAST_HIT
TIMEOUT
ADMIN_001
192.168.1.1
2M_AGO
28:14
SYS_OPERATOR
45.22.12.01
5M_AGO
25:00
MAINT_BOT_A
INTERNAL_SVC
12M_AGO
18:45
AUDIT_USER_09
72.19.112.5
29M_AGO
00:12
RECENT_EVENT_LOG
14:02:11
CONFIG_CHANGE: CLINIC_ID_04 Updated access roles.
13:44:59
AUTH_FAILURE: Multiple attempts from IP 202.x.x.x
13:12:05
BACKUP_SUCCESS: Core database mirrored to US_WEST.
12:59:30
USER_LOGOUT: ADMIN_004 manual disconnect.
VIEW_FULL_AUDIT_LOG

2.4.7 Key Screens

1. Login Interface

- Provides secure authentication for system users
- Includes input validation and error feedback
- Supports role-based access redirection after login

2. Role-Based Dashboard

- Displays summary information relevant to each role (Admin, Doctor, Pharmacist)
- Provides quick navigation to frequently used modules
- Includes alerts such as upcoming appointments and low inventory

3. Patient Management Interface

- Allows administrators to register and update patient information
- Displays patient records in a structured and searchable format

4. Appointment Management Interface

- Enables appointment scheduling, rescheduling, and cancellation
- Includes calendar-based and list-based views

5. Doctor Consultation Interface

- Allows doctors to view patient history and record clinical notes
- Supports prescription creation and submission

6. Pharmacy and Inventory Interface

- Displays prescriptions for dispensing
- Allows pharmacists to manage medicine inventory and stock levels

Each interface is designed with consistency in layout, typography, and component usage to ensure a seamless user experience.

2.4.8 User Flow and Navigation Design

The system navigation is designed to ensure smooth and efficient task completion with minimal user effort.

2.4.8.1 Primary User Flow

User → Login → Role-Based Dashboard → Select Module → Perform Action → Logout

Detailed Interaction Flow

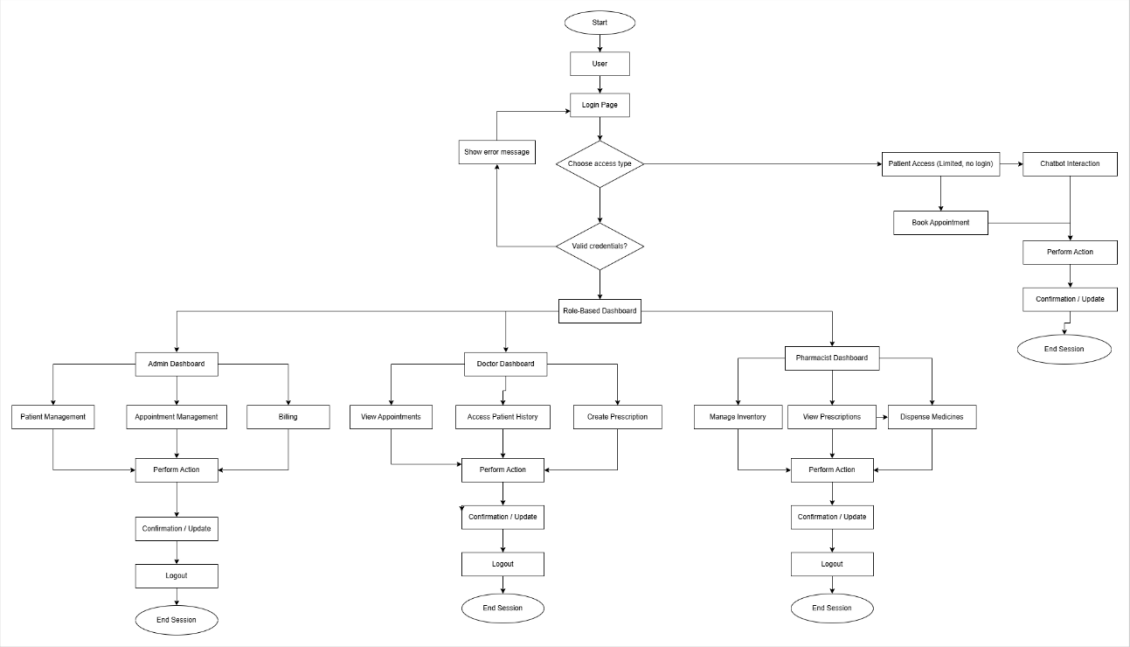


Figure 29: User Flow Diagram

Navigation Structure

- Top navigation bar for global actions (profile, logout)
- Sidebar menu for module-based navigation
- Breadcrumbs for tracking user location within the system

This structure ensures:

- Reduced navigation complexity
- Faster task completion
- Clear system feedback

2.4.9 Usability Considerations

The user interface of the system has been designed with a strong focus on usability to ensure efficient and error-free interaction. Several usability principles have been considered during the design process:

- **Consistency:** Uniform layouts, colors, and components are maintained across all interfaces to reduce the learning curve for users.
- **Simplicity:** Interfaces are designed to be clean and uncluttered, allowing users to complete tasks with minimal effort.
- **Feedback Mechanisms:** Immediate system feedback is provided through alerts, validation messages, and confirmations to inform users about the outcome of their actions.

- **Error Prevention and Recovery:** Input validation and guided workflows are implemented to prevent errors, while clear error messages assist users in correcting mistakes.
- **Efficiency:** Frequently used features are easily accessible through dashboards and navigation menus, reducing the time required to perform tasks.

These usability considerations ensure that the system is intuitive, user-friendly, and suitable for users with varying levels of technical expertise, including administrative staff, medical professionals, and patients.

2.4.10 Accessibility Considerations (WCAG 2.1 Compliance)

The system is designed to comply with Web Content Accessibility Guidelines (WCAG) 2.1 to ensure usability for all users, including those with disabilities.

1. Visual Accessibility
    - Text and background color contrast ratio maintained at a minimum of 4.5:1
    - Avoidance of color-only indicators for important actions or errors
2. Keyboard Accessibility
    - All interactive elements (forms, buttons, navigation) are accessible via keyboard
    - Logical tab order is maintained across all screens
3. Screen Reader Support
    - Form inputs include descriptive labels
    - Buttons and interactive components have meaningful names
    - ARIA attributes are used where necessary
4. Error Handling and Feedback
    - Clear and descriptive error messages are displayed
    - Errors are not indicated by color alone; text explanations are provided
    - Inline validation helps users correct issues before submission
5. Responsive Design
    - Interfaces are optimized for desktops, tablets, and mobile devices
    - Layouts adapt dynamically to different screen sizes

The accessibility features implemented in the system are aligned with internationally recognized standards to ensure inclusivity and usability for all users. The design follows the guidelines specified in the Web Content Accessibility Guidelines (WCAG) 2.1, which define best practices for improving accessibility across web-based applications. These

guidelines ensure that the system is perceivable, operable, understandable, and robust for users with diverse abilities.

2.5 Security Design

2.5.1 Authentication Mechanism

A credential-based authentication mechanism is implemented using JSON Web Tokens (JWT) combined with username and password authentication. Secure identity verification, session management, and access control are enforced throughout the system.

1.1.3.8 Credential-Based Authentication

Authentication is performed using a username and password. Upon successful authentication, a JWT token is issued and used for subsequent requests.

1.1.3.9 Password Policy

The system enforces a strong password policy to enhance security.

Rule	**Specification**
Minimum length	12 characters
Complexity	Uppercase, lowercase, numeric, and special character required
Hashing algorithm	Argon2id
First-login reset	Mandatory password change after reset
Password history	Last 5 passwords cannot be reused
Password expiry	Every 90 days

1.1.3.10 Password Storage

Passwords are securely stored using the Argon2id hashing algorithm with the following configuration:

- Algorithm: Argon2id
- Salt Length: 16 bytes (random per user)
- Time Cost: 2 iterations
- Memory Cost: 65,536 KB
- Parallelism: 1
- Hash Format: $argon2id$v=19$m=65536,t=2,p=1$<salt>$<hash>

1.1.3.11 Login Flow

The authentication process follows a secure sequence:

1. Credentials are submitted via POST /api/auth/login.
2. Input validation is performed (null checks, length validation).
3. The user record is retrieved from the database.
4. The password is verified using Argon2id hashing.

Successful Authentication

- A JWT token is generated.
- Login activity is recorded in the Audit Log (timestamp and client IP).
- The token is returned to the client.

Failed Authentication

- Failed attempt counter is incremented.
- If failed attempts ≥ 5:
    - The account is locked for 15 minutes.
    - Lockout event is recorded in the Audit Log.
    - A lockout response is returned.
- Otherwise:
    - An "Invalid username or password" message is returned.

1.1.3.12 Authentication Flow Diagram

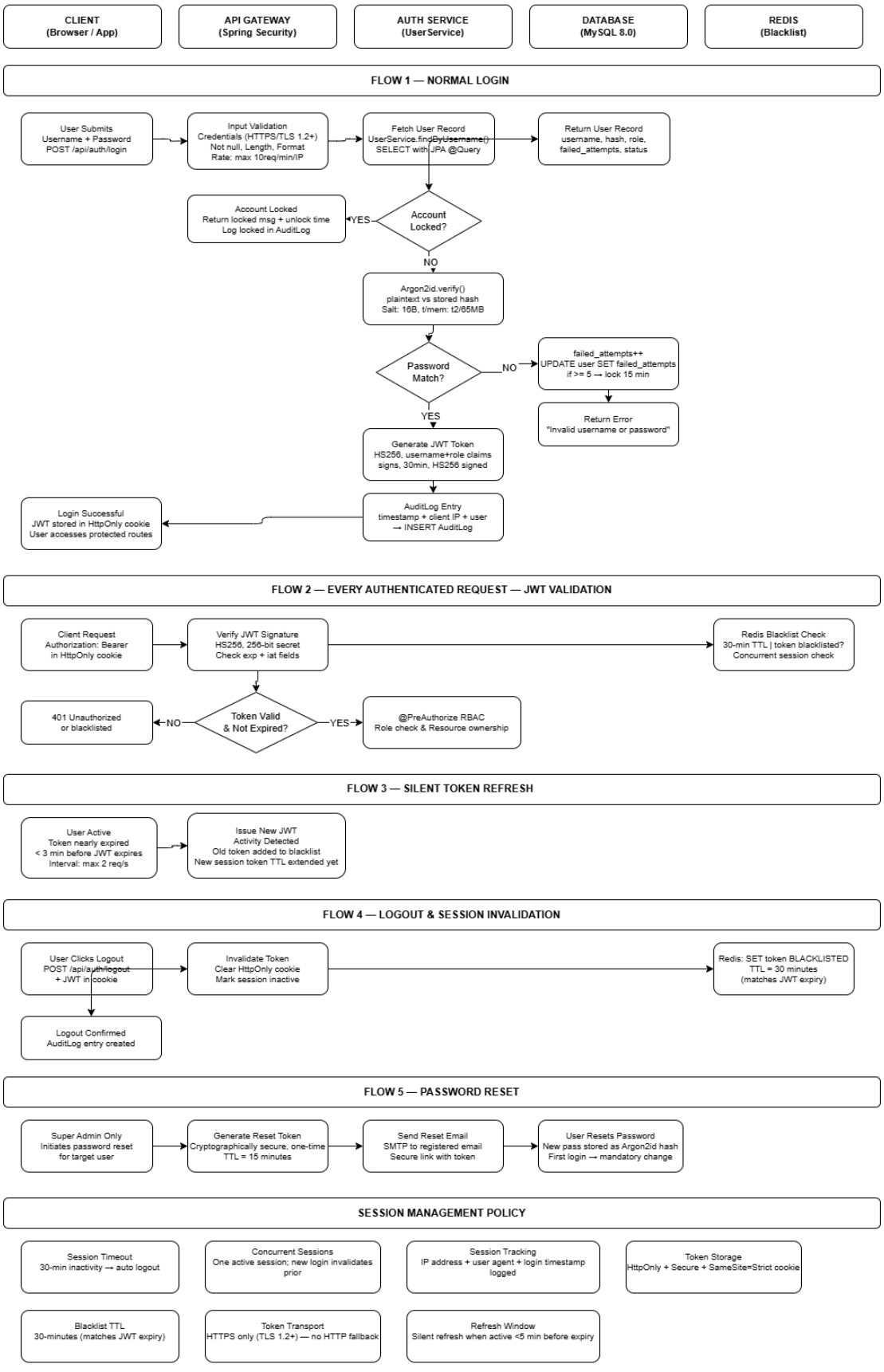


Figure 30: Authentication Flow Diagram

1.1.3.13 JWT Token Structure

JWT tokens are issued as Bearer tokens and transmitted via the HTTP Authorization header. Tokens are stored securely in cookies.

Token Components

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "sub": "user_123",
  "username": "doctor_name",
  "role": "DOCTOR",
  "email": "doctor@clinic.com",
  "iat": 1712282400,
  "exp": 1712284200,
  "iss": "onecare-clinic",
  "aud": "onecare-app"
}
```

Signature:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  SECRET_KEY
)
```

1.1.3.14 Token Security Properties

- Signing: HMAC-SHA256 (256-bit key)
- Key storage: Secure environment variables
- Transport: HTTPS (TLS 1.2+)
- Storage: httpOnly, Secure, Same Site=Strict cookie
- Token refresh: Automatic refresh before expiry (5 minutes prior)
- Token revocation: Redis-based blacklist (30-minute TTL)

1.1.3.15 Session Management

The system enforces strict session control policies:

- Session timeout: 30 minutes of inactivity
- Concurrent sessions: Single active session per user
- Session tracking: IP address, user agent, login timestamp
- Logout behavior: Token blacklisted and session invalidated

1.1.3.16 Password Reset Flow

The password reset process is controlled and secure:

1. Reset is initiated by a Super Administrator.
2. A time-limited reset token is generated.
3. A reset link is sent via email (SMTP).
4. The link remains valid for 30 minutes and is single use.
5. The user is required to change the password upon first login.
6. The reset event is recorded in the Audit Log.

1.1.3.17 Authentication Mechanisms Summary

- Credential storage: Argon2id hashing (16-byte salt, 65536 KB memory)
- Token type: JWT (Bearer, HS256)
- Token expiry: 30-minute session timeout
- Token transport: HTTPS + HTTP Only Secure cookie
- Account lockout: 15 minutes after 5 failed attempts
- Password reset: Email-based, 30-minute expiry, one-time use
- Rate limiting: Maximum 10 authentication requests per minute per IP

2.5.2 Authorization Levels

1.1.3.18 Role Definitions

Five system roles are defined with strictly controlled permissions to enforce separation of duties.

Role	**Key Responsibilities**
Super Administrator	User management, role configuration, audit log access, system backup and restore, chatbot knowledge base management
Administrator	Patient registration and updates (non-clinical), appointment and queue management, billing and payments
Doctor	Appointment viewing, medical history access, clinical note recording, prescription creation
Pharmacist	Prescription dispensing, inventory management, and external dispensing operations

Patient	Appointment booking (without authentication) and chatbot interaction

To enforce proper separation of responsibilities:

- Doctors are restricted from modifying pharmacy inventory.
- Pharmacists are restricted from modifying patient medical records.
- Administrators are limited to non-clinical patient data only.

These restrictions are enforced at multiple layers within the system.

1.1.3.19 Permission Matrix

Table 9: Permission Matrix

Module	Action	Super Admin	Admin	Doctor	Pharmacist	Patient
User Management	Create User	✓	✗	✗	✗	✗
User Management	Edit/Deactivate User	✓	✗	✗	✗	✗
User Management	Reset Password	✓	✗	✗	✗	✗
Clinic Configuration	Configure Schedules	✓	✗	✗	✗	✗
Clinic Configuration	Manage Roles & Permissions	✓	✗	✗	✗	✗
Patient Management	Register Patient	✓	✓	✗	✗	✗
Patient Management	Edit Non-Clinical Information	✓	✓	✗	✗	✗
Patient Management	View Patient Profile	✓	✓	✓	✓*	✗
Appointment Management	Create Appointment	✓	✓	✗	✗	✓ (self)
Appointment Management	Reschedule/Cancel	✓	✓	✗	✗	✗

Appointment Management	View Appointments	✓	✓	✓ (own)	✗	✓ (own)
Queue Management	Manage Queue	✓	✓	✓	✗	✗
Doctor Module	Access Medical History	✓	✗	✓	✗	✗
Doctor Module	Record Clinical Notes	✗	✗	✓	✗	✗
Doctor Module	Create Prescription	✓	✗	✓	✗	✗
Pharmacy	View Prescriptions	✓	✗	✓	✓	✗
Pharmacy	Dispense Medicines	✓	✗	✗	✓	✗
Inventory	Manage Medicines	✓	✗	✗	✓	✗
Inventory	View Stock Levels	✓	✗	✓ (read-only)	✓	✗
External Dispensing	Mark External Dispense	✓	✗	✗	✓	✗
External Dispensing	Verify Patient Identity	✓	✗	✗	✓	✗
Billing	Generate Invoice	✓	✓	✗	✗	✗
Billing	Record Payment	✓	✓	✗	✗	✗
Reports	View Clinic Reports	✓	✓	✗	✗	✗
Reports	View Pharmacy Reports	✓	✗	✗	✓	✗
Chatbot	Manage Knowledge Base	✓	✗	✓	✗	✗

Audit	View Audit Logs	✓	✗	✗	✗	✗
System	Backup & restore	✓	✗	✗	✗	✗

* Pharmacists are limited to non-clinical patient information only.

1.1.3.20 RBAC Implementation

Authorization is enforced using a multi-layered approach to ensure secure access control.

- **Endpoint-Level Security:** Access control is enforced using framework-level annotations.
- **Service-Level Authorization:** Business rules are enforced within the service layer to prevent unauthorized operations.
- **Data-Level Security:** Access restrictions are enforced at the data layer using filtered queries.

2.5.3 Data Protection Approach

1.1.3.21 Encryption at Rest

Sensitive data is protected at rest using strong encryption mechanisms at both the database and application levels.

Data Category	**Encryption Method**	**Algorithm**	**Key Size**
Patient Records	Database-level encryption (InnoDB TDE)	AES-256	256-bit
Authentication Credentials	Secure hashing	Argon2id	—
Sensitive Fields (e.g., contact details, healthcare ID)	Column-level encryption	AES-256-CBC (PKCS5 padding)	256-bit
Prescription / Clinical Notes	Column-level encryption	AES-256-CBC	256-bit
Backup Files	File-level encryption	AES-256-GCM	256-bit
Audit Logs	Encrypted database storage	AES-256 (TDE)	256-bit

External Dispensing Records	Column-level encryption	AES-256-CBC	256-bit

Implementation Details

- Transparent Data Encryption (TDE) is enabled at the database level to secure stored data files.
- Sensitive fields are encrypted at the application layer using secure cryptographic libraries.
- Encryption keys are stored separately from application source code and managed through secure configuration mechanisms (e.g., environment variables or a secrets manager).
- A key rotation policy is enforced, where encryption keys are rotated periodically and securely archived to support long-term data access requirements.

1.1.3.22 Encryption in Transit

All data transmitted between clients and servers is secured using industry-standard transport encryption protocols.

- Protocol: HTTPS with TLS 1.2 or higher
- Cipher Suites: Strong Cipher suites (e.g., AES-256-GCM-based)
- Certificate: Valid SSL/TLS certificate issued by a trusted Certificate Authority
- HSTS: HTTP Strict Transport Security enforced
- Perfect Forward Secrecy: Enabled using ephemeral key exchange mechanisms

This ensures that data cannot be intercepted or modified during transmission.

1.1.3.23 Data Access Control

Access to data is restricted based on user roles and responsibilities to ensure confidentiality and integrity.

- **Patient medical records** are accessible only to authorized medical personnel.
- **Prescription data** is accessible to the issuing doctor and authorized pharmacy staff for dispensing purposes.
- **Non-clinical patient data** is accessible to administrative staff for operational use.
- **Financial and billing data** is restricted to authorized administrative roles.
- **Audit logs** are accessible only to highly privileged system roles.
- **External dispensing records** are accessible only to authorized pharmacy personnel.

- **Chatbot knowledge base content** is manageable only by authorized roles.
- **Sensitive data masking** is applied in reports, where partial values (e.g., contact numbers, identification numbers) are displayed instead of full data.

2.5.4 Input Validation and Sanitization Strategy

1.1.3.24 Input Validation Strategy

Input validation is enforced across multiple layers to ensure data integrity and prevent invalid or malicious input.

Layer 1 - Client-Side Validation

Validation is performed at the user interface level to provide immediate feedback and reduce invalid submissions:

- Required field checks are enforced.
- Email format validation is applied.
- Phone number format validation follows the Sri Lankan format (+94-XX-XXX-XXXX).
- Date validation ensures that appointment dates are not set in the past.
- Numeric constraints are enforced (e.g., stock quantity ≥ 0, price > 0).
- Inline validation messages are displayed prior to form submission.

Layer 2 - Server-Side Validation

Server-side validation is implemented using Bean Validation (JSR-380) to ensure that all incoming data is validated regardless of client-side enforcement.

Layer 3 - Business Logic Validation

Additional validation rules are enforced within the service layer:

- Doctor availability is verified before confirming appointments.
- Appointment slot conflicts are prevented to avoid double-booking.
- Prescriptions are validated against available and approved medicines.
- Inventory updates are restricted to prevent negative stock levels.
- External dispensing quantities are validated against remaining prescription limits.
- Patient identity verification is required before dispensing medications externally.

1.1.3.25 Sanitization Strategy

Sanitization mechanisms are applied to prevent injection attacks and ensure safe data handling.

Cross-Site Scripting (XSS) Prevention

- User-supplied inputs are sanitized before storage and rendering.

- Special characters such as <, >, &, ", and ' are escaped.
- Frontend rendering frameworks automatically escape output values, providing an additional layer of protection.

SQL Injection Prevention

- All database operations are executed using parameterized queries.
- Object-relational mapping frameworks are used to eliminate direct SQL string construction.

1.1.3.26 CSRF Protection

Cross-Site Request Forgery protection is implemented to secure state-changing operations.

- CSRF tokens are generated per session and included in all state-changing requests (e.g., POST, PUT, DELETE).
- Built-in security framework protections are enabled for all secured endpoints.
- Public endpoints are excluded from CSRF enforcement but are protected through alternative mechanisms such as rate limiting.

1.1.3.27 Rate Limiting

Rate limiting is applied to prevent abuse, brute-force attacks, and denial-of-service scenarios.

Endpoint Category	Limit	Enforcement Mechanism
Authentication endpoints	10 requests per minute per IP	Security filters with distributed counters
Public booking endpoints	20 requests per minute per IP	API gateway rate limiting
General API endpoints	100 requests per minute per user	API gateway rate limiting

Rate limiting policies are enforced dynamically and monitored to ensure system stability and availability.

2.5.5 Encryption Standards for data at rest and in transit

1.1.3.28 Encryption Standards

This section provides a consolidated overview of the encryption mechanisms implemented within the system.

Standard	Usage	Algorithm	Key Length

AES-256-CBC	Column-level encryption (patient data, prescriptions)	AES	256-bit
AES-256-GCM	Backup file encryption	AES	256-bit
TLS 1.2+	Client–server communication	TLS with secure key exchange	—
Argon2id	Password hashing	Argon2id	128-bit output
HS256	JWT token signing	HMAC-SHA256	256-bit
RSA-2048	SSL/TLS certificates	RSA	2048-bit

1.1.3.29 Security Architecture Diagram

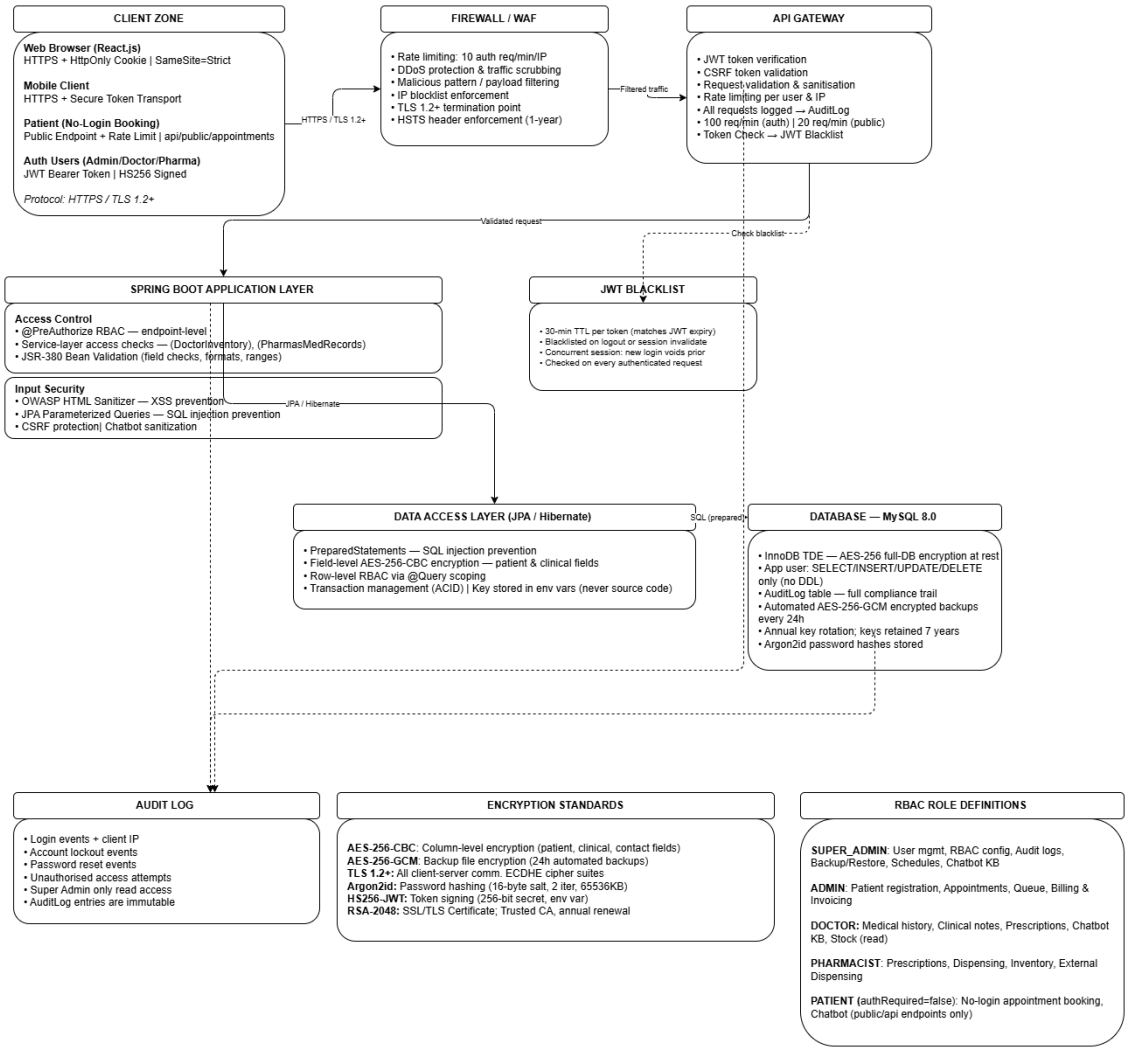


Figure 31: Security Architecture Diagram

1.1.3.30 Security Testing and Vulnerability Management

Security testing and vulnerability management practices are implemented to ensure continuous monitoring, detection, and mitigation of security risks.

Activity	**Tool / Method**	**Frequency**
Dependency vulnerability scanning	Automated dependency analysis tools	Every build
Static code analysis	Code quality and security analysis tools	Every build
Dynamic security testing	Automated application security scanning	Before production releases

Penetration testing	External or internal security assessment	Periodically
Authentication and access monitoring	Centralized logging and alerting systems	Continuous
Security patch management	Regular updates and vulnerability fixes	As required
Incident response	Defined response procedures (detection, containment, recovery)	Triggered upon incidents

2.6 Deployment Design

2.6.1 Hosting Plan

The OneCare Clinic Management System shall be deployed using containerized architecture to ensure portability, scalability, and ease of management. The system shall initially be hosted in a local development environment and can be extended to a cloud-based infrastructure (AWS) for production deployment.

2.6.2 Hardware Requirements

The minimum hardware requirements for system deployment are defined as follows:

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB SSD
- Network: Stable internet connection

2.6.3 Network Diagram

The system architecture shall include client devices communicating with the backend services through a secure network. The backend services shall interact with the database and external services such as SMS and email gateways.

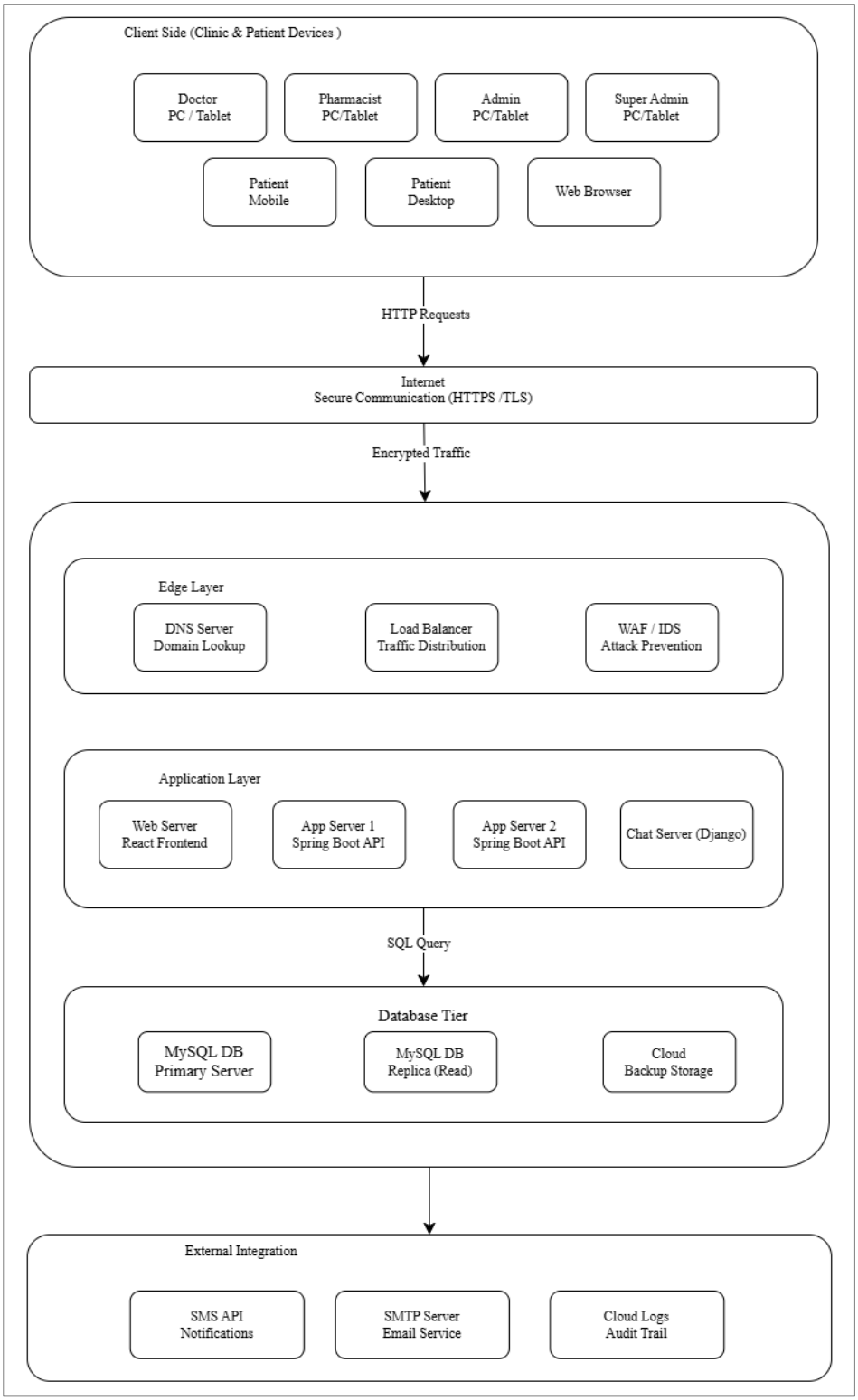


Figure 32: Network Diagram

2.6.4 Environment specification

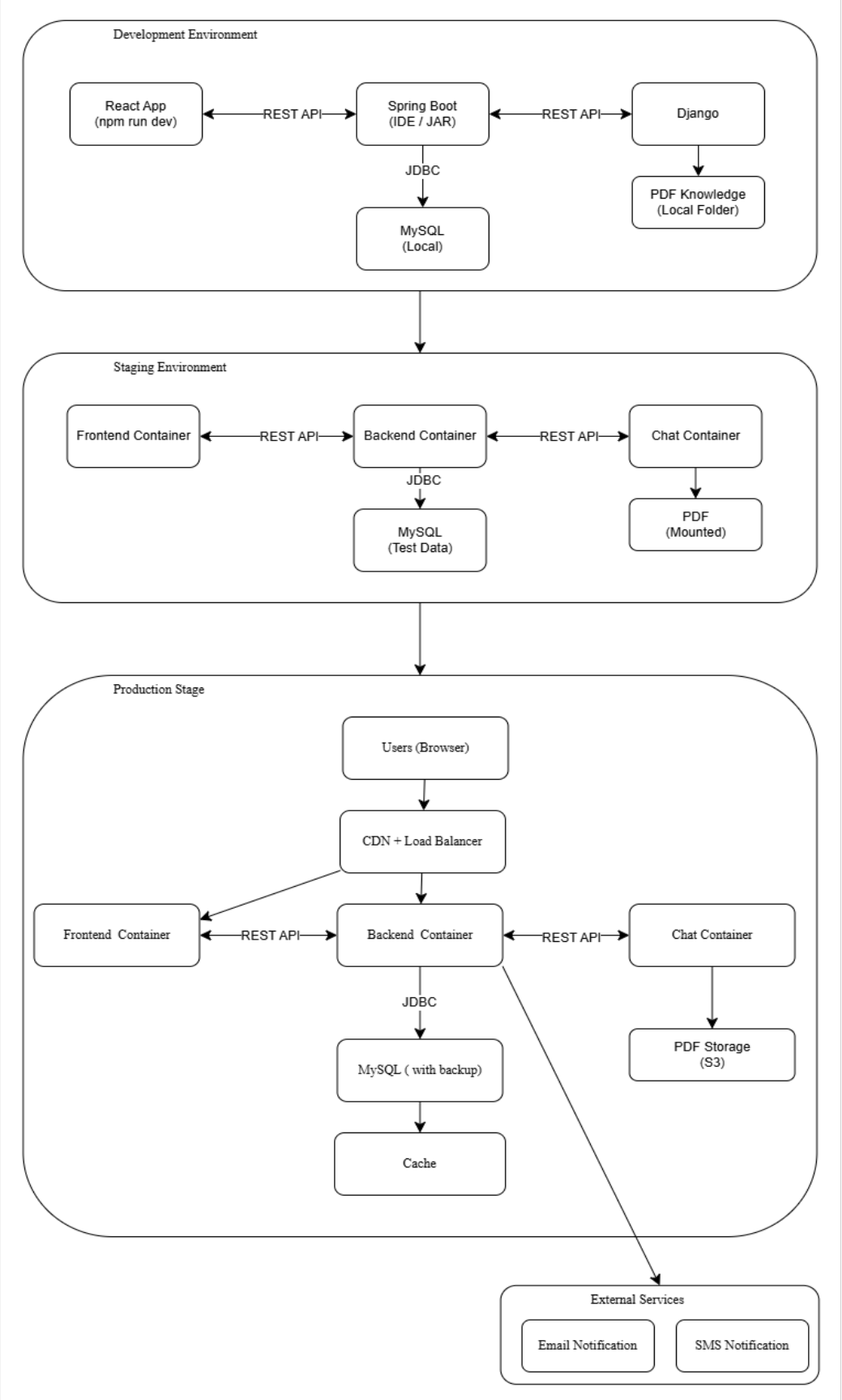


Figure 33: Environment Specification

2.6.4.1 Development Environment

The development environment shall be used by developers for coding, debugging, and initial testing. The system shall run locally using Docker Compose, enabling all services (frontend, backend, and database) to be executed in isolated containers.

2.6.4.2 Staging Environment

The staging environment shall replicate the production environment as closely as possible and shall be used for quality assurance (QA), integration testing, and user acceptance testing (UAT). Test data shall be used instead of real patient data to ensure data privacy.

2.6.4.3 Production Environment

The production environment shall represent the live system used in real-world clinic operations. It shall be designed to ensure high availability, security, and reliability.

The following operational targets and measures shall be enforced:

- System availability shall be maintained at **≥ 99%**
- Disaster recovery shall be completed within **≤ 4 hours (RTO)**
- Automated daily backups shall be performed with geo-redundancy
- Real-time monitoring and alerting mechanisms shall be implemented
- Centralized logging shall be maintained with a retention period of 90 days
- Role-Based Access Control (RBAC) shall be enforced across all modules
- Audit logging shall be implemented for compliance and traceability

2.6.5 CI/CD Pipeline

A Continuous Integration and Continuous Deployment (CI/CD) pipeline shall be implemented to automate the build, testing, and deployment processes.

The pipeline shall include the following stages:

1. **Source Control**

   Code shall be managed using Git-based repositories (GitHub).

2. **Build Stage**

   The application shall be built automatically upon code commits.

3. **Testing Stage**

   Automated unit and integration tests shall be executed to ensure code quality.

4. **Containerization**

   Docker images shall be created for the frontend and backend services.

5. **Deployment**

   The application shall be deployed to the target environment (staging or production).

6. **Monitoring**

   Deployed services shall be monitored for performance, errors, and uptime.

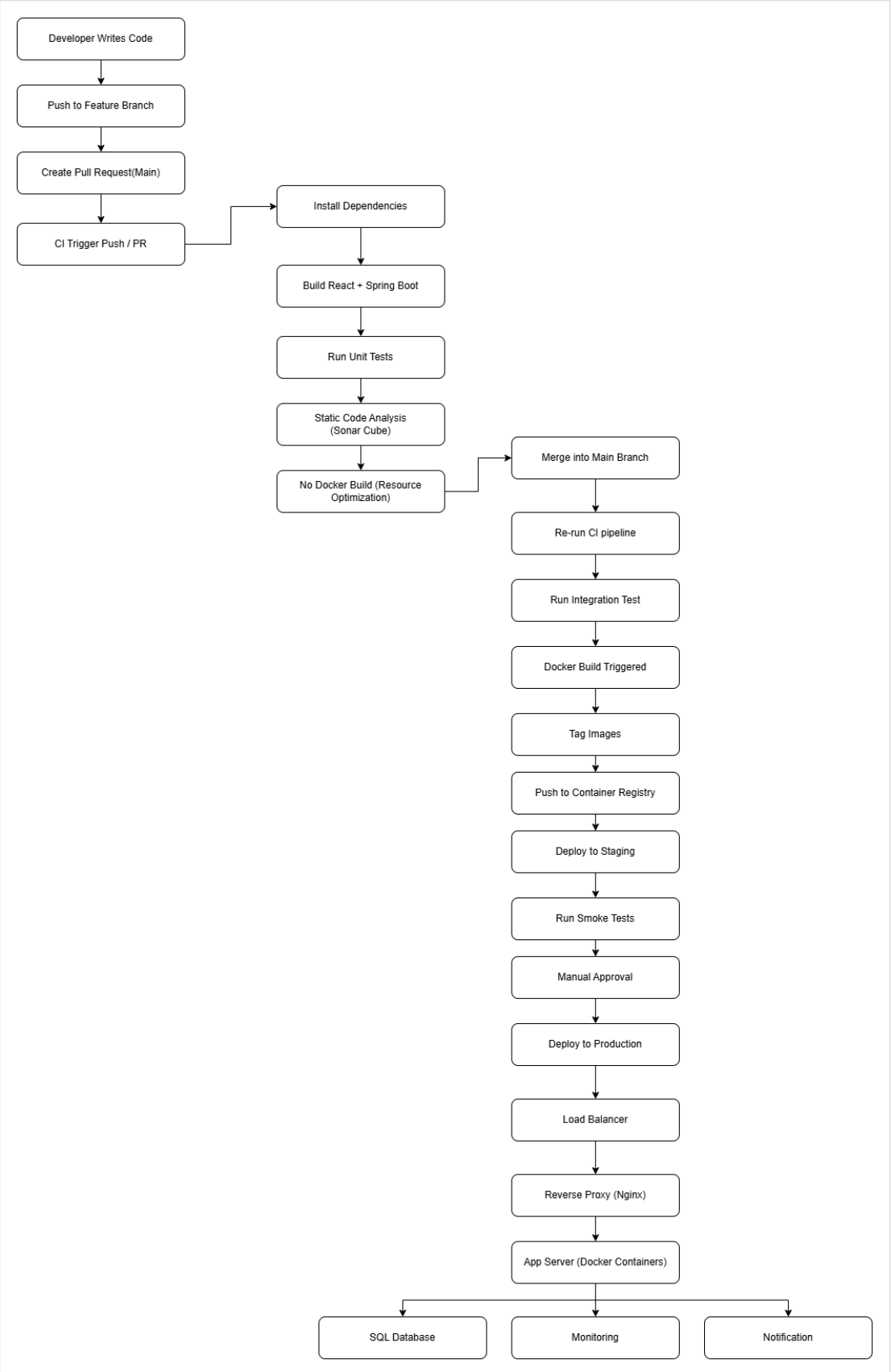


Figure 34: CI-CD Diagram

ANNEX 3: PROOFREADING CERTIFICATION

This appendix clarifies a minor timing detail in the external review. Due to time zone differences between Sri Lanka and Australia, feedback received on 30 April (Sri Lanka time) is recorded as 1 May in the reviewer's local time.



To,
The OneCare Development Team
Faculty of Science
University of Kelaniya
Sri Lanka

Review of SRS (v1.0) and SDS (v1.5) — OneCare Clinic Management System

Dear Team,

I have spent a good amount of time going through both your System Requirements Specification and your Software Design Specification, and I want to start by saying this the work you have put in is genuinely impressive for a final-year project. The security chapter in the SDS in particular shows a level of depth that many working developers do not bother with. The normalization walkthrough, the RBAC design, the Argon2id configuration, these things tell me you understand what you are building and why. You should be proud of that.

That said, I would not be doing my job if I glossed over the problems I found. Some of them are serious enough that they would cause the system to fail at runtime the moment certain features are used. I want to walk you through them honestly, not as criticism, but because catching these now — on paper — is infinitely better than catching them after you have written the code.

Let me start with the most pressing ones.

The billing module is specified in FR-035 through FR-038 with a reasonable level of detail invoices, payment recording, status tracking, history retrieval. But when you get to Chapter 10 of the SRS where the data model lives, there is simply no Invoice entity. No table, no fields, nothing. The billing requirements float in the document without any database structure to hold them. Before you write a single line of backend code for billing, that entity needs to be designed and added to the data model. At minimum you will need fields for the invoice ID, patient, appointment reference, consultation charge, medicine total, overall amount, payment status, who generated it, and when.

The second issue will cause database errors the moment your pharmacy module goes live. Your functional requirements describe prescription statuses as "Pending Dispense", "Fully Dispensed", "Partially Dispensed", and "Dispensed (External)". But if you look at Table 10-4 in the SRS; the actual Prescription entity; the ENUM is defined as only ('Issued', 'Dispensed', 'Cancelled'). Those two sets of values share almost nothing in common. When the system tries to write "Pending Dispense" into a column that only accepts "Issued", MySQL will reject it. Please align the ENUM in the schema with the statuses defined in your requirements before the database is created.

While you are fixing the prescription table, please also look at appointment statuses. FR-018 says the statuses are Scheduled, Completed, Cancelled, and No-show. The description comment in Table 10-3 says Pending, Confirmed, Cancelled, Completed. The actual ENUM in the same table says Scheduled, Cancelled, Completed, No-show. That is three different sets of values within a single document. Pick one and make it consistent everywhere in the requirements, the use cases, and the schema.

There is also a quiet but significant problem with anonymous appointment booking. FR-043 allows patients to book without logging in, which is a good usability decision for a clinic. However, your Appointment table has patient_id defined as NOT NULL with a foreign key to the Patient table. An anonymous patient has no patient_id. The database will reject every single anonymous booking with a constraint violation. You need to decide what happens at that moment does the system create a provisional patient record, does it allow a nullable patient_id for self-service

Figure 35: Proof Reading Certificate - Page 01

bookings, or does something else happen? There is currently no requirement or use case that answers this, and there needs to be one.

One more critical issue , and this one is a contradiction at the architectural level. FR-003 says that role-based permissions shall be "configurable" by the Super Admin. That word — configurable implies the Super Admin can change who can access what at runtime, through the system interface. But the SDS shows a hardcoded permission matrix with no supporting database table for permissions, no role_permissions entity, and a JWT payload that simply encodes a static role name. A hardcoded matrix cannot be configured at runtime. You need to make a clear decision: are permissions fixed by design, or are they genuinely dynamic? If dynamic, you need a permissions table and the access control layer needs to check it. If fixed, FR-003 needs to be reworded. Either answer is fine, but you cannot claim both in the same document set.

Beyond those critical items, there are a number of technical decisions in the SDS that I would encourage you to revisit.

The JWT configuration uses HS256, which is a symmetric signing algorithm — the same key signs and verifies tokens. For a single server this is workable, but your own SDS describes horizontal scaling and potential AWS deployment. When you run multiple instances behind a load balancer, every instance must share that secret, and that is a growing security risk. RS256, which uses a private key to sign and a public key to verify, is the standard approach for distributed deployments and is just as easy to implement in Spring Security.

There is also an inconsistency in your encryption choices that goes the wrong way. You have chosen AES-256-GCM for backup files and AES-256-CBC for patient records and prescriptions. GCM is the more secure of the two; it provides authenticated encryption, meaning it protects both the confidentiality and integrity of the data. CBC protects only confidentiality and is sensitive to how you manage initialisation vectors. The most sensitive data in your system — medical records and prescriptions ;should be using GCM, not the other way around.

On the topic of Redis: Section 1.4.1 describes it as optional infrastructure, but Section 6.1.7 makes your entire token revocation mechanism depend on it. If Redis is not running and a user logs out or a token is compromised, there is no way to blacklist that token for the rest of its 30-minute lifetime. Either Redis is required ;in which case remove the word "optional" — or you need a fallback, such as a database-backed blacklist that Redis sits in front of as a cache.

There is a straightforward contradiction on password resets between the two documents. The SRS says password reset happens via a time-limited email link, with no mention of who initiates it. The SDS says the reset must be initiated by the Super Admin. These cannot both be true. More practically; if only the Super Admin can reset passwords, what happens when the Super Admin is locked out? There needs to be a documented recovery path, and the two documents need to say the same thing.

NFR-R001 asks for 99% monthly uptime, which works out to about seven hours of acceptable downtime per month. Your deployment design in the SDS describes a single Docker Compose node with no failover and no replica database. A single machine will regularly exceed that seven-hour budget just from planned maintenance. Either bring the uptime target down to something achievable with a single-node setup 95% is a more honest figure or describe a basic high-availability configuration with a standby node.

Your usability requirements include Tamil language support for all patient-facing pages, which is a thoughtful inclusion for a Sri Lankan clinic. But the SDS says nothing about how this will actually be implemented. Tamil script has specific font requirements. React applications need an internationalisation library like react-i18next to handle string externalisation cleanly. Translation files need to be structured. None of this is in the SDS, and if it is not designed now, it will be bolted on later in a way that creates maintenance problems. Please add at least a brief i18n design section before frontend work begins.

Figure 36: Proof Reading Certificate - Page 02

Finally, the offline capability described in the SDS is using Service Workers and IndexedDB to handle five minutes of disconnection is a meaningful feature for a clinic with unreliable internet. But medical data is particularly sensitive to synchronisation conflicts. If a doctor marks a consultation complete offline, and the admin has concurrently updated the same record on the server, what happens when the device reconnects? The SDS does not say. This is a design decision that needs an answer before implementation, not after. Server-wins is the simplest approach and probably correct for most scenarios here; the important thing is that the answer is written down.

I also want to flag one smaller but important inconsistency. Section 9.1 of the SRS lists "Medical history" as a field on the patient registration form that Admin fills in. But Business Rule BR-006 explicitly restricts Admin to non-clinical information only. Medical history is clinical data — it should not appear on the Admin registration form. Doctors enter clinical notes during consultation, not Admin at registration. This needs to be corrected before UI development begins, otherwise the screen will be built incorrectly.

Similarly, the inventory dashboard in Section 9.1 lists "batch number" as a display field, but the Medicine entity in Chapter 10 has no batch_number column. Either the column needs to be added to the schema, or removed from the UI requirement. Given that batch numbers are important for pharmaceutical recalls, I would recommend adding it to the schema.

I want to close by repeating what I said at the start. The foundation here is genuinely strong. The architecture decision to use a layered monolith rather than microservices for a single-doctor clinic is correct and well-reasoned. The security thinking Argon2id, CSRF protection, rate limiting, audit logging are serious and appropriate. What needs attention now is consistency: making sure that what the requirements say, what the design says, and what the database schema says are all telling the same story. Right now they are not, and that gap, if left uncorrected, will cost you significantly more time during implementation than it costs you to fix it today.

I am happy to discuss any of these points further. Good luck with the build.

Yours sincerely,



Arebhy.G
Code Wizard Aotearoa
Senior Software Engineer
1st May 2026

Figure 37: Proof Reading Certificate - Page 03

ANNEX 4: CLIENT FEEDBACK REPORT

CLIENT MEETING & FEEDBACK REPORT

(Stage 4 – Final Design Validation)

1. Meeting Information

Project Title : OneCare Clinic Management System

Date : 30/04/2026

Time : 6.30 am

Location/Platform : OneCare Family Clinic - Chulipuram

2. Client Details

Client Name : Dr. V. Guruparan

Designation : Medical Officer

Organization : OneCare Family Clinic

Contact (Optional) : 0770563046

3. Participants

Name	Role
V. Guruparan	Client
S. Vipusa	Developer
R. Sweeskaran	Developer

4. Live System Demonstration

Feature	Demonstrated (✓)	Client Comment
Login / Authentication	☑	

Figure 38: Client Feedback Report - Page 01

Dashboard	☑	
Patient Management	☑	
Appointment Scheduling	☑	
Reports / Records	☑	
Other	☐	

5. Client Feedback Summary

5.1 UI/UX Feedback

Use medical symbols

Increase Font Size

Make proper alignments to page

5.2 Functionality Feedback

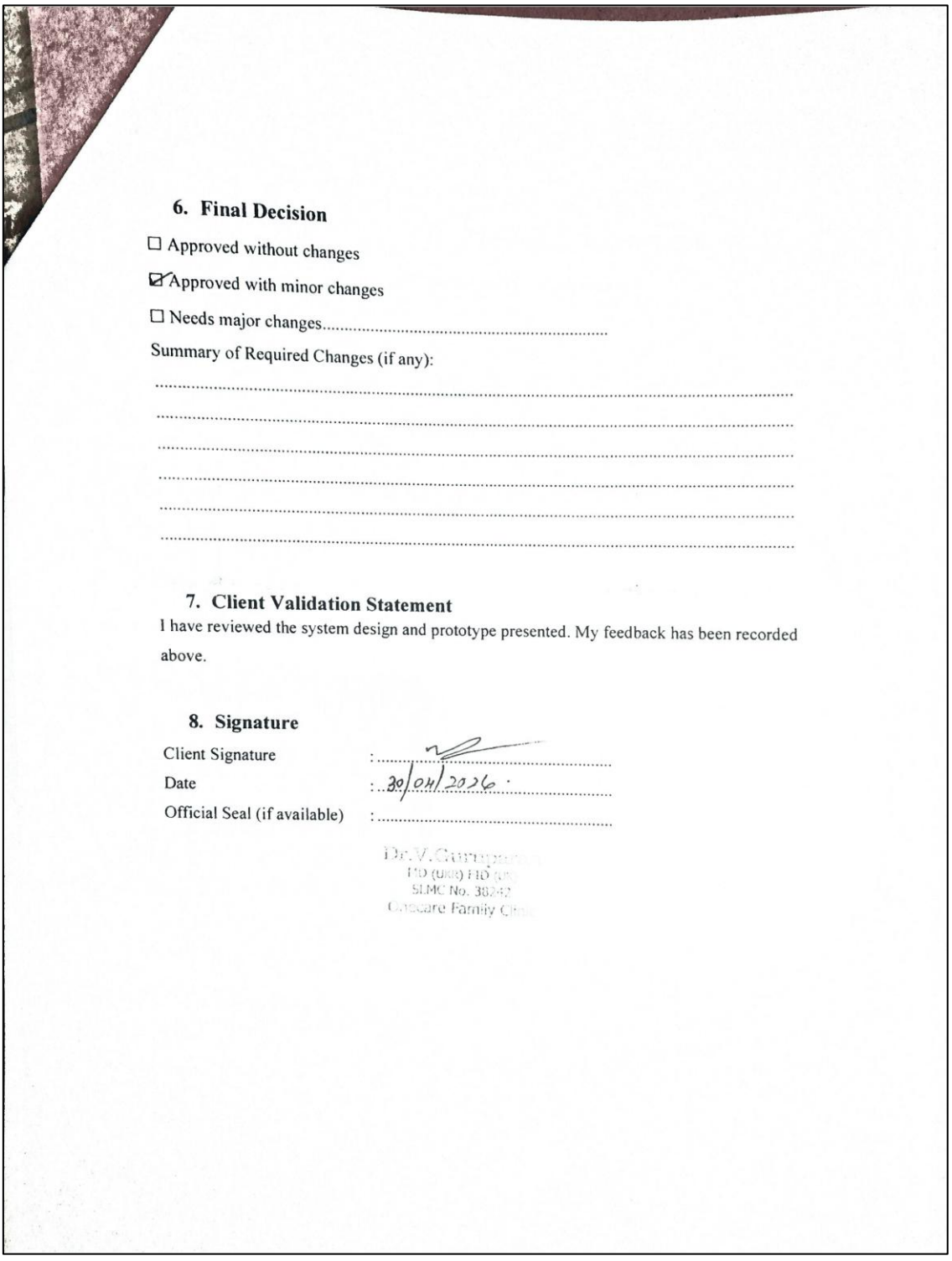
5.3 Suggested Improvements

provide simple notifications for actions

5.4 Missing Features

No multiple doctor support - do it fo future enhancement

Figure 39 Client Feedback Report - Page 02



6. Final Decision

☐ Approved without changes

☑ Approved with minor changes

☐ Needs major changes.....

Summary of Required Changes (if any):

.....

.....

.....

.....

.....

.....

7. Client Validation Statement

I have reviewed the system design and prototype presented. My feedback has been recorded above.

8. Signature

Client Signature : .....

Date : 30/04/2026 .....

Official Seal (if available) : .....

Dr. V. Ganeshan
MD (URO) FRCS (UK)
SLMC No. 38242
Oncocare Family Clinic

Figure 400: Client Feedback Report - Page 03